

Kubernetes Operator Project Ideas

Thu, Jul 23 at 12:48 AM

From quite a few days i am learning kubernetes in detail for CKA and also learning Go to contribute to open source CNCF projects, currently i am working as a platform engineer intern at eaglevew. This is for my background.

I want to make a project, when i send my resume after putting that project no recruiter could ignore me for giving me an interview chance. Project should be like that solves real problem that exist in kubernetes, or could add some feature to existing services....
so I got this while researching:

Operator kya hota hai, basics se

Kubernetes mein tu normally deployments, services, configmaps jaise built-in resources use karta hai — inko manage karne ka logic Kubernetes ke andar hi (kube-controller-manager) already likha hua hai. Jaise tu Deployment banata hai with replicas=3, controller continuously watch karta hai aur ensure karta hai ki 3 pods hamesha running rahein — agar ek crash ho jaye, naya spawn ho jaata hai automatically.

****Operator = custom controller + custom resource, jo yeh same pattern kisi bhi custom application ke liye implement karta hai.****

Do parts hote hain:

1. ****CRD (Custom Resource Definition)**** — ek naya "resource type" jo tu Kubernetes API mein define karta hai. Jaise `kubectl get pods` chalta hai, waise hi tu apna khud ka resource bana sakta hai — jaise `kubectl get databasebackups`
2. ****Controller**** — ek Go program jo continuously watch karta hai us resource ko, aur "desired state" vs "actual state" compare karke actions leta hai. Isko ****reconciliation loop**** kehte hain — ye core concept hai jisse CKA mein bhi indirectly touch hota hai (control plane architecture).

Problem kya solve karta hai

Real problem: Kubernetes ka default controller sirf generic infrastructure resources (pods, deployments, services) handle karta hai. Lekin ****application-specific operational knowledge**** (jaise "database ka backup kaise lena hai", "kis sequence mein failover karna hai", "kab restore trigger karna hai") — ye Kubernetes ko nahi pata, aur na hi ho sakta hai generically.

Operator pattern isko solve karta hai: tu apna khud ka "operational expertise" code mein encode kar deta hai, aur Kubernetes usko automatically, continuously execute karta hai — bina human intervention ke.

****Example concrete case:**** Tera "DatabaseBackup" operator socho —

- Tu ek YAML likhta hai: `kind: DatabaseBackup, schedule: daily, retention: 7days, target: postgres-prod`

- Operator ye dekh ke automatically: backup job trigger karega, S3 pe upload karega, purane backups delete karega retention policy ke hisaab se, agar backup fail ho to Slack alert bhejega, aur agar koi manually backup resource delete kare to cleanup bhi karega

Ye sab manual script/cron job se bhi ho sakta hai, but operator ka fayda: ****declarative, self-healing, Kubernetes-native lifecycle management.**** Agar cluster restart ho, node crash ho, resource accidentally delete ho — operator apne aap detect karke sahi state mein wapas le aata hai.

Kya companies isko use kar sakti hain?

Realistically — ****teri banayi hui specific operator ko koi company directly production mein use nahi karegi****, aur ye samajhna zaroori hai taaki expectation clear rahe. Reasons:

- Production-grade operator banane mein months lagte hain (edge cases, testing, security, RBAC scoping, upgrade paths)
- Companies apna khud ka internal tooling banati hain apni specific needs ke hisaab se
- Tera project ek ****portfolio/learning project**** hai, not a "sellable product"

****Lekin**** — agar tu genuinely polished bana le (proper testing, docs, README, real use case), to:

- GitHub pe open-source kar sakta hai, kuch chhote startups/side projects use kar sakte hain
- Ye interview mein tera "maine production-level thinking ke saath system design kiya" wala signal banta hai — yehi asli value hai, use hone se zyada important ****kaise socha aur banaya**** wo hai

Kya impact create karega (honestly)

Impact do level pe dekhna: ****industry impact**** vs ****career impact****.

- ****Industry impact: Minimal-to-zero**** — realistically tu kisi existing tool ko replace nahi kar raha, na hi koi naya massive gap fill kar raha hai. Ye chhota learning-scale project hai.
- ****Career impact: High**** — ye interview mein "K8s internals samajhta hai" ka strongest possible signal hai fresher/junior level pe. Recruiter ko impress karne ke liye industry-changing hone ki zaroorat nahi, technically deep hone ki zaroorat hai.

Brutal honesty: agar tu iska pitch "ye company production mein use karegi" wala banayega interview mein, wo overselling lagega aur senior interviewer turant pakad lega. Sahi framing: "Maine ye banaya taaki controller pattern, reconciliation loop, aur K8s API internals ko practically samjhoon — CKA prep ke saath directly align kiya."

Similar tools already existing?

Haan, bahut saare — aur ye important hai janna taaki tu apna operator differentiate kar sake:

- ****Kubebuilder / Operator SDK**** — ye tools hain jo operator banane mein help karte hain (scaffolding), khud operator nahi
- ****Prometheus Operator**** — real-world example, monitoring stack manage karta hai declaratively
- ****ArgoCD, FluxCD**** — GitOps operators, deployment lifecycle manage karte hain
- ****Velero**** — backup/restore operator (bahut similar tere DatabaseBackup idea se) — production-grade, widely used
- ****cert-manager**** — TLS certificate lifecycle operator

Iska matlab: tera exact idea (jaise backup operator) already solved hai production-grade tools se (Velero). ****Isse problem nahi hai**** — tu isko "learning reimplementation" ke tarah frame karega, jaise log LeetCode solve karte hain existing algorithms ke — goal solve karna nahi, samajhna hai.

****Suggestion:**** apna operator thoda unique twist de — jaise "cost-aware autoscaler operator" jo Prometheus metrics + AWS pricing API combine karke scale karta ho (tera FinOps EagleView experience se directly connect hota hai) — ye combination kam common hai aur tera actual work experience se bhi ganthta hai, jo interview mein extra credibility deta hai.

Detailed description file for this one:

Project: Cost-Aware Kubernetes Autoscaler Operator

One-line pitch

A custom Kubernetes Operator that scales workloads based on both ****application load (Prometheus metrics)**** and ****real-time AWS cost data**** — preventing autoscaling decisions that blindly spike cloud spend.

Why this specific project (not a generic operator)

Standard HPA (Horizontal Pod Autoscaler) scales purely on CPU/memory/custom metrics — it has zero awareness of cost. In real production environments, teams often need to balance performance against budget (this is literally FinOps — and it connects directly to your EagleView cost-anomaly-detection work, giving you a real story to tell in interviews without overclaiming).

This project sits at the intersection of three things recruiters care about for Platform/SRE roles: ****Kubernetes internals, Go engineering, and cost-conscious infrastructure thinking.****

Core Concept

You define a Custom Resource:

```
yaml
apiVersion: costops.karan.dev/v1alpha1
kind: CostAwareScaler
metadata:
  name: api-server-scaler
spec:
  targetDeployment: api-server
  minReplicas: 2
  maxReplicas: 10
  metricsSource:
    prometheusQuery: 'avg(rate(http_requests_total[5m]))'
    scaleUpThreshold: 100
    scaleDownThreshold: 20
  costConstraints:
    maxHourlyCostUSD: 5.00
    instanceType: "t3.medium"
  alerting:
    slackWebhookSecret: slack-webhook-secret
```

Your Operator watches this resource and:

1. Polls Prometheus for the defined query (load metric)
2. Calculates current estimated hourly cost (replicas × instance cost, pulled from AWS Pricing API or a static cost table for MVP)
3. Decides whether to scale up/down based on ****both**** load AND whether it stays within

maxHourlyCostUSD

4. If load says "scale up" but cost constraint blocks it → sends a Slack alert saying "wanted to scale to N replicas, blocked by cost ceiling" instead of silently doing nothing
5. Updates the resource's **status** field with current replica count, current estimated cost, last decision reasoning

Architecture Components

1. CRD (Custom Resource Definition) — **CostAwareScaler**

Defines the schema above. Built with Kubebuilder markers, generates OpenAPI validation automatically.

2. Controller / Reconciler

The core Go logic — implements the **Reconcile()** function that:

- Fetches the current **CostAwareScaler** object
- Fetches current replica count of target Deployment
- Queries Prometheus (via HTTP API, using **prometheus/client_golang**)
- Computes cost estimate
- Makes scaling decision
- Patches the target Deployment's replica count via **client-go**
- Updates **.status** subresource
- Requeues itself (e.g., every 30s) — this IS the reconciliation loop

3. Metrics Client

Small internal package that wraps Prometheus HTTP API queries — isolate this so it's testable independently.

4. Cost Estimator

For MVP: a static lookup table (**t3.medium = \$0.0416/hr**, etc. — hardcode common EC2 types). For v2: actually call AWS Pricing API for live rates.

5. Alerting Module

Simple Slack webhook POST when a scaling decision is blocked or when a threshold is crossed. Uses Kubernetes Secrets for the webhook URL (this alone is a good talking point — secret management, not hardcoding credentials).

6. Status Reporting

Kubernetes-native **status** subresource so **kubectl get costawarescaler** shows meaningful state — this is what makes it feel like a "real" K8s-native tool, not just a script.

Tech Stack

Component Tool
--- ---
Language Go
Scaffolding Kubebuilder
K8s client controller-runtime + client-go

Metrics	prometheus/client_golang
Local cluster	Kind
Testing	envtest (controller-runtime's test framework) + Ginkgo/Gomega
CI	GitHub Actions (lint, test, build image)
Container	Docker (multi-stage build for small image size)
Alerting	Slack Incoming Webhooks

Build Phases (so it's not overwhelming)

Phase 1 — Skeleton (Week 1)

- **kubebuilder init**, define the CRD, generate boilerplate
- Reconciler that just logs "reconcile called" — get the loop running against Kind cluster

Phase 2 — Real scaling logic (Week 2)

- Wire up Prometheus query (can fake metrics locally with a test exporter if no real app yet)
- Implement scale up/down logic against a dummy Deployment
- No cost logic yet — just prove the core scaling loop works

Phase 3 — Cost awareness (Week 3)

- Add static cost table
- Add the constraint logic (block scale-up if over budget)
- Update **.status** subresource properly

Phase 4 — Alerting + polish (Week 4)

- Slack webhook integration
- Proper error handling, retries, edge cases (target deployment doesn't exist, Prometheus unreachable, etc.)
- Unit tests with envtest

Phase 5 — Documentation + packaging (Week 5)

- Architecture diagram (draw.io or Excalidraw — shows reconciliation flow visually)
- README with setup instructions, demo GIF/video
- Helm chart or plain YAML manifests for installation
- Push image to Docker Hub / GHCR

Realistic total: ****4-6 weeks**** if you're doing this alongside CKA prep and internship, not full-time.

What This Demonstrates in an Interview (be precise about this, don't oversell)

Honest framing to use: **"I built this to deeply understand the controller/reconciliation pattern that Kubernetes itself is built on, and to combine it with cost-awareness since that's the FinOps problem I work on at EagleView."**

What it actually proves:

- You understand CRDs, controllers, reconciliation loops (core K8s architecture — this is what separates you from "I used kubectl" candidates)
- You can write idiomatic Go against real APIs (client-go, Prometheus client)

- You think about production concerns: secrets, retries, status reporting, observability
- You can connect infrastructure work to business impact (cost) — a Platform/SRE-flavored signal, not just "I can code"

What it does ****not**** prove, and don't claim it does:

- That it's production-ready or used by any company
- That you're a Go expert (you're not yet, and interviewers doing a code review will find out fast if you claim otherwise)

Stretch Goals (only after MVP works — don't scope-creep early)

- Multi-metric support (combine CPU + custom Prometheus query)
- Webhook validation (reject invalid CRD specs at admission time — teaches you admission controllers)
- Grafana dashboard showing scaling decisions over time
- Convert static cost table to live AWS Pricing API calls
- Publish as a Helm chart on Artifact Hub

Immediate Next Step

Install Kubebuilder + set up a Kind cluster locally, then run `kubebuilder init --domain karan.dev --repo github.com/<your-username>/cost-aware-scaler` and `kubebuilder create api --group costops --version v1alpha1 --kind CostAwareScaler`. This generates the skeleton — from there Phase 1 is just making the Reconciler log successfully on every change to the CR.

Show moreShow less

Real talk:

This is a **good project**, but I would **not** make it my primary portfolio project if the goal is:

"No recruiter should ignore my resume."

The reason is simple.

Recruiters spend **20-40 seconds** on a resume. They don't know whether your operator is technically deep or just another Kubebuilder tutorial. Every week people build CRUD apps, operators, and "mini-HPA" projects.

The project needs to immediately make a recruiter think:

"This person understands Kubernetes internals and solved a problem people actually complain about."

The operator idea is good for interviews.

It is **not unique enough** to stand out on GitHub.

Let's evaluate it like a senior platform engineer would.

What is good

It demonstrates

- CRDs
- Controllers
- Reconciliation loop
- client-go
- controller-runtime
- Go
- Kubernetes API
- Prometheus
- Status subresources
- RBAC
- Leader election
- Watches
- Finalizers (if implemented)

These are excellent skills.

What is weak

The "cost-aware autoscaler" itself.

Why?

Because companies already have

- HPA
- KEDA
- Cluster Autoscaler
- Karpenter
- Goldilocks
- Kubecost
- AWS Compute Optimizer
- Prometheus Adapter

Your operator becomes

another scaling controller.

Interviewers know this.

Even worse...

AWS Pricing API is **not what production systems actually use** for scaling.

Nobody is querying pricing APIs every 30 seconds.

Real systems already know

- node cost
- spot/on-demand cost
- reserved instances
- cluster budget

through Kubecost or FinOps platforms.

So the "live AWS pricing" feature sounds cooler than it actually is.

The bigger issue

You are preparing for

- CKA
- Go
- CNCF contribution
- Platform Engineering

That means your strongest selling point is

Kubernetes internals.

Don't dilute that.

Build something where Kubernetes internals are the hero.

Not AWS pricing.

What would impress me much more?

Here are projects I would rank.

Tier S (Resume changer)

1. Kubernetes Drift Detection Operator ★★★★★

This is my favorite.

Problem:

Someone manually edits Deployment.

Someone changes replicas.

Someone changes image.

Someone changes ConfigMap.

Git says one thing.

Cluster says another.

Operator continuously detects

Desired

vs

Actual

and creates Kubernetes Events, Slack alerts, Prometheus metrics, annotations.

Unlike ArgoCD

don't sync automatically.

Just detect and explain.

Example

```
Deployment nginx

Git:
replicas = 3

Cluster:
replicas = 5

Reason:
Manual kubectl scale detected.

Suggested Fix:
kubectl scale deployment nginx --replicas=3
```

Why good?

You learn

- Informers
- Watches
- DeepEqual
- Diffing
- Events
- OwnerReferences
- Status

- Conditions
- Finalizers

Interview value: huge.

2. Kubernetes Dependency Operator ★★★★★

One problem nobody teaches.

Deployment A depends on

- Redis
- Kafka
- PostgreSQL
- Secret
- ConfigMap

Normally pods crash.

Instead

```
kind: DependencyPolicy
```

Operator waits.

Checks readiness.

Then starts application.

Like systemd dependencies.

Real production problem.

3. Kubernetes Upgrade Advisor ★★★★★

Imagine

```
kubectl apply
```

Operator checks

- deprecated APIs
- incompatible versions
- resource limits
- PodSecurity violations
- bad probes
- missing PDB

- missing NetworkPolicy

and comments

This deployment will likely fail after upgrading to 1.34

That's extremely valuable.

4. Kubernetes Resource Optimizer ★★★★★

Not autoscaler.

Much better.

Reads

Prometheus

Looks at

CPU

Memory

OOMKills

Throttle

Recommends

Current:

CPU: 2 cores

Recommended:

700m

Much closer to what companies actually need.

Goldilocks exists.

Doesn't matter.

Reimplementing teaches lots.

5. Kubernetes Restart Analyzer ★★★★★

This one is amazing.

Continuously watches pods.

Whenever a restart happens

collect

- Events
- Previous logs

- Exit code
- OOM
- Node conditions
- Container status
- Probe failures

Automatically produce

Likely Cause:

OOMKilled

Evidence:

Memory limit exceeded.

Recommendation:

Increase limit from 512Mi to 1Gi.

Every Kubernetes engineer has debugged this.

Tier A

Secret Rotation Operator

Rotates

Secrets

TLS

Passwords

Triggers rollout restart.

Very useful.

Namespace Cleanup Operator

Deletes

old Jobs

completed Pods

orphan PVCs

old ConfigMaps

Huge issue in internal clusters.

PVC Auto Expansion Operator

When PVC >80%

Automatically resize

Very realistic.

Node Maintenance Operator

Drain

Cordon

Slack notifications

Maintenance windows.

Which one would recruiters remember?

Suppose recruiter sees

Resume A

Built Cost Aware Autoscaler

Resume B

Built Kubernetes Restart Root Cause Analyzer

Guess which one sounds cooler?

Second.

Because every Kubernetes engineer has suffered restart loops.

Immediately relatable.

If you REALLY want an operator

Then make something that existing operators don't explain well.

Example

PodCrashAnalyzer

CR

kind: CrashAnalysis

Operator

- watches Pods
- watches Events
- collects previous logs
- identifies

OOM

CrashLoop

ImagePullBackOff

DNS

Probe failures

Secret missing

PVC pending

Network issue

- updates Status

```
kubectl get analysis
```

NAME

frontend

CAUSE

OOMKilled

CONFIDENCE

98%

SUGGESTION

Increase memory limit.

This is awesome.

Another idea I like even more

A Kubernetes Event Correlation Engine.

Production problem.

One issue creates

500 events.

Nobody knows root cause.

Operator groups them.

NodeNotReady

↓

Pods Evicted

↓
PVC Pending
 ↓
Deployment Failed
 ↓
HPA Failed

Shows

Root Cause

↓

Impact Chain

This is genuinely interesting.

If I were mentoring you

Knowing your background

- Platform Engineering intern
- CKA
- Go
- CNCF contributions

I would tell you:

Don't build something because it's "an operator."

Build something because platform engineers actually lose hours debugging it.

That makes the operator merely the implementation detail.

My recommendation

I would rank them like this:

1. **Kubernetes Restart Analyzer Operator** ★★★★★
2. **Drift Detection Operator** ★★★★★
3. **Dependency Management Operator** ★★★★★☆
4. **Resource Recommendation Operator** ★★★★★☆
5. **Cost-Aware Autoscaler** ★★★☆☆

The first two are much more likely to spark a technical conversation in an interview because they solve day-to-day operational pain that almost every Kubernetes team recognizes. They also force you to work

deeply with the Kubernetes API, events, watches, reconciliation, and Go—exactly the areas you're trying to master.

can you tell me in detail in simple text about each project you talked about, so that may have a clear idea and could select the best one, also tell me what are the other solutions for each of them that exist and company uses them

Real talk:

Out of the projects I suggested, **only 2** have the potential to become a "this person really understands Kubernetes" project if you build them deeply:

- **Kubernetes Restart Analyzer Operator**
- **Kubernetes Drift Detection Operator**

The others are also good, but these two naturally force you to learn more Kubernetes internals and production debugging.

I'll explain every project from scratch, including:

- What real problem it solves
- How companies solve it today
- Why existing solutions aren't enough
- What your operator would do
- Whether it's worth building

1. Kubernetes Restart Analyzer Operator ★★★★★ (My #1 Recommendation)

The real-world problem

Imagine a production application.

Frontend
Backend
Redis
Postgres
Kafka

One day, an alert comes.

Backend restarted 143 times

Now an engineer has to answer:

Why?

This is much harder than people think.

Possible reasons:

```
OOMKilled
CrashLoopBackOff
ImagePullBackOff
Secret missing
ConfigMap missing
Database unavailable
Readiness probe failed
Liveness probe failed
DNS failure
TLS failure
Permission denied
PVC not mounted
Node pressure
Killed by kubelet
Application panic
Container exited with code 1
```

Finding the actual reason may take **30 minutes to several hours**.

What engineers actually do today

They run

```
kubectl describe pod
```

Then

```
kubectl logs
```

Then

```
kubectl logs --previous
```

Then

```
kubectl get events
```

Then

```
kubectl describe node
```

Then

```
kubectl top pod
```

Then check Grafana.

Then Prometheus.

Then Slack.

Then maybe Loki.

This is repetitive.

What your operator would do

Instead of humans investigating,
your operator continuously watches Pods.

Whenever it detects a restart,
it automatically gathers

```
Pod events
Container state
Exit code
Previous logs
Current logs
Node conditions
Memory usage
CPU usage
Probe failures
Image status
```

Then analyzes everything.

Produces something like

```
Analysis
Cause:
OOMKilled

Confidence:
97%

Evidence:
Container exited with code 137

Memory usage reached 512Mi

Recommendation:
Increase memory limit to 1Gi
```

Now

```
kubectl get podanalysis
```

shows

```
NAME
frontend
CAUSE
OOMKilled
STATUS
Resolved
```

Existing solutions

Companies use

- Grafana
- Prometheus
- Loki
- Elasticsearch
- Datadog
- Splunk
- New Relic
- Dynatrace

These collect data.

But none of them automatically explain

WHY

Kubernetes restarted the pod.

Human still investigates.

Why your project is still valuable

You're not replacing Grafana.

You're automating investigation.

Very realistic.

Difficulty

★★★★☆

Need to learn

- Watch API
- Events
- Logs
- Pod status
- Conditions
- Controller Runtime

Amazing learning.

2. Kubernetes Drift Detection Operator ★★★★★

Problem

Git says

```
replicas: 3
```

Someone runs

```
kubectl scale deployment app --replicas=10
```

Now

Git

```
3
```

Cluster

```
10
```

This is called

Configuration Drift

In production,

people accidentally modify

Deployment

Secrets

ConfigMaps

Ingress

Service

Resources

Images

Annotations

Labels

Nobody notices.

Months later

everything becomes inconsistent.

Your operator

Operator continuously compares

Desired

vs

Actual

If difference found

Deployment
Git
3 replicas
Cluster
10 replicas
Difference
replicas
Changed By
Unknown
Time
12:40 PM

Creates

Kubernetes Event

Slack notification

Prometheus metric

Status update

Existing solutions

Main one

ArgoCD

FluxCD

ArgoCD

continuously compares Git and cluster.

If different

either

Shows OutOfSync

or

Auto-syncs.

So why build yours?

Because

your operator can go much deeper.

For example

Instead of saying

```
OutOfSync
```

it can say

```
Reason
```

```
Horizontal Pod Autoscaler changed replicas.
```

```
Ignore.
```

or

```
kubectl edit was used.
```

```
Potential manual drift.
```

or

```
Deployment image changed.
```

```
Probably hotfix.
```

Explain

not just detect.

Difficulty

★★★★★

Excellent project.

3. Kubernetes Dependency Operator ★★★★★☆

Problem

Imagine

Application needs

```
Redis
Kafka
Postgres
```

But Kubernetes starts everything together.

Result

```
Backend starts
↓
Redis not ready
↓
Crash
↓
Restart
↓
Crash
↓
Restart
```

Developers often solve this using

```
sleep 30
```

Seriously.

Or

Init Containers.

Or

Retry forever.

Your operator

You write

```
DependencyPolicy
```

```
Backend
```

```
depends on
```

```
Redis
```

```
Kafka
```

```
Postgres
```

Operator checks

```
Redis Ready?
```

```
Kafka Ready?
```

```
Database Ready?
```

Only then

starts Backend.

Existing solutions

Mostly

Init Containers

Readiness Probes

Helm Hooks

Service Mesh

Custom scripts

No popular dedicated operator.

Interesting niche.

Difficulty

★★★★☆

4. Kubernetes Resource Recommendation Operator ★★★★★

Problem

Developers write

resources:

cpu

4

memory

8Gi

Because

"No idea."

Actual usage

CPU

300m

Memory

500Mi

Cluster wastes money.

Or opposite

CPU

100m

Need

2 CPU

Pods throttle.

Your operator

Reads

Prometheus

Computes

Average CPU

Peak CPU

Memory

OOM history

Recommends

Current

2 CPU

Suggested

600m

Updates
Status
Creates report

Existing solutions
Goldilocks
Kubecost
VPA
AWS Compute Optimizer
Google Recommender

Still excellent learning.

Difficulty
★★★★☆

5. Cost-Aware Autoscaler ★★★★★

Problem
Load high.
HPA says

Scale
10
Pods

Finance says
"No."

Operator decides

Stay at
7
Pods

Alert

Budget exceeded.

Existing solutions
Kubecost

Karpenter
Cluster Autoscaler
HPA
AWS
FinOps tools

Interesting
but
less Kubernetes-focused than others.

Difficulty
★★★★☆

6. Kubernetes Upgrade Advisor ★★★★★☆

Problem
Cluster upgrade

```
1.29
↓
1.32
```

Some APIs removed.
Pods fail.
Ingress fails.
PDB changes.
Admission changes.

Operator scans everything.
Produces

```
Deployment frontend
Issue
extensions/v1beta1 removed
Recommendation
Use networking.k8s.io/v1
```

Existing solutions

kubent
pluto
kubescape
Fairwinds Polaris

Mostly scanners.
Not operator-based.

Difficulty
★★★★★

7. Namespace Cleanup Operator ★★☆☆

Problem
Clusters become dirty.
Thousands of
Completed Jobs
Old PVCs
Unused ConfigMaps
Unused Secrets
Temporary Namespaces
Nobody cleans.

Operator
Deletes
according to policies.

Existing solutions
Mostly
CronJobs
Scripts
Kyverno policies

Good
but not resume-defining.

8. Secret Rotation Operator ★★★★★☆

Problem

Secrets never change.

Passwords become years old.

Security risk.

Operator

Rotates

TLS

Passwords

Database credentials

Triggers rollout restart.

Existing solutions

External Secrets Operator

Vault

cert-manager

AWS Secrets Manager

Very useful

but security-heavy.

Comparison

Project	Real Production Problem	Existing Solutions	Learning Value	Resume Impact
Restart Analyzer	Pod failures and debugging	Grafana, Loki, Datadog, New Relic	★★★★★★	★★★★★★
Drift Detection	Configuration drift	Argo CD, Flux CD	★★★★★★	★★★★★★
Dependency Operator	Startup ordering	Init Containers, Readiness Probes	★★★★★☆	★★★★★☆
Resource Recommendation	Wrong CPU/Memory requests	Goldilocks, VPA, Kubecost	★★★★★☆	★★★★★☆
Upgrade Advisor	Kubernetes version upgrades	kubent, Pluto, Polaris	★★★★★★	★★★★★☆
Secret Rotation	Secret lifecycle	Vault, External Secrets Operator	★★★★★☆	★★★★★☆
Cost-Aware Autoscaler	Cost-aware scaling	HPA, Karpenter,	★★★★★☆	★★★★★☆☆

Project	Real Production Problem	Existing Solutions	Learning Value	Resume Impact
		Kubecost		
Namespace Cleanup	Cluster hygiene	CronJobs, Kyverno, Scripts	★★★★☆☆	★★★★☆☆

If I were optimizing for your exact goal

Given your background:

- Platform Engineering intern
- Preparing for CKA
- Learning Go specifically for CNCF contributions
- Wanting a project that leads to strong Kubernetes interviews

I would build **one flagship project** instead of three average ones.

My choice would be:

1. **Kubernetes Restart Analyzer Operator** — because every Kubernetes engineer has debugged CrashLoopBackOffs, OOMKills, probe failures, and restart storms. It gives you a natural opportunity to demonstrate controllers, watches, events, logs, status subresources, and Kubernetes debugging.
2. **Kubernetes Drift Detection Operator** — if you're more interested in GitOps, reconciliation, and controller design than operational debugging.

These are the projects most likely to lead to an interviewer spending 20–30 minutes discussing your design decisions instead of just asking, "So you built another operator?"

Kubernetes Restart Analyzer Operator, as you recommended i am going to pick this one at first for building, but currently i have not completed what this tool does actually, i understand that when an alert of Backend restarted 143 times then engineer has to go through kubectl describe pods and there he can get all the thing under events and problem solved (i know my assumption is incorrect but i have this type of doubt for this), but overall engineer has to go through multiple commands and logs of different tools to verify what happened with particular pod why it is getting restarted again and again. First i need to understand what type of problem this tool will solve then need to understand step by step exactly how each specific problem is solved by engineer then only i can conclude that what problem this is going to solve and how much i can save for teams. Tell me in detail about these

Show moreShow less

Real talk:

This is exactly the right question.

Most people jump into coding the operator before understanding the operational workflow. That's why their projects end up being "toy operators."

Before writing even one line of Go, you should be able to answer:

"If I were an on-call Platform Engineer at 2 AM, how would I debug a restarting pod?"

Only then can you automate that workflow.

First misconception

You said:

"Engineer has to go through `kubectl describe pod` and problem solved."

This is **sometimes true**, but in many real incidents it's **not even close**.

Let's see why.

Imagine you're on-call

It's 2:15 AM.

PagerDuty rings.

Alert:

```
Service: payment-api
Pod Restarts: 143
Namespace: production
```

Management only knows one thing:

Customers cannot make payments.

They don't care whether it's OOM, probe failure, DNS, or Kafka.

They want

Fix it.

Now you start investigating.

Step 1

First question

Is Kubernetes restarting it?

Run

```
Bash
kubectl get pods
```

Output

NAME	READY	STATUS	RESTARTS
payment-api	0/1	CrashLoopBackOff	143

Good.

Now we know

- Pod exists
- It isn't deleted
- It keeps restarting

But...

We still don't know

WHY.

Step 2

Run

```
Bash
kubectl describe pod payment-api
```

You see

```
Events
Started container
Killing container
Back-off restarting failed container
```

That's all.

You still don't know

WHY.

CrashLoopBackOff is **not the cause**.

It is the symptom.

Think of it like

Doctor says

```
Patient has fever.
```

Great.

Why?

Virus?

Malaria?

Dengue?

COVID?

No idea.

CrashLoopBackOff is just

"The container keeps crashing."

Step 3

Now engineer checks logs

```
Bash
kubectl logs payment-api
```

Output

```
Connecting to database...
Database timeout
```

Maybe database problem.

Or maybe not.

Sometimes logs say

```
panic:
```

Easy.

Sometimes

```
Segmentation fault
```

Easy.

Sometimes

```
Killed
```

Not useful.

Sometimes

Nothing.

Literally nothing.

Why?

Because container crashed before writing logs.

Now what?

Step 4

Engineer checks previous logs

```
Bash
kubectl logs payment-api --previous
```

Very important.

Because Kubernetes already started a new container.

Current logs belong to new container.

Previous logs belong to crashed container.

Many beginners don't know this.

Example

Current logs

```
Starting...
Loading config...
```

Previous logs

```
panic
nil pointer dereference
```

Now we found it.

Step 5

Maybe logs don't help.

Now engineer checks exit code.

```
Bash
kubectl describe pod
```

Container Status

```
Last State
Terminated
Exit Code
137
```

What is 137?

Experienced engineers immediately think

OOMKilled

Memory exceeded.

No need to check application.

Increase memory.

Exit Code

1

Generic failure.

Need more investigation.

Exit Code

143

SIGTERM.

Maybe deployment rollout.

Maybe node drain.

Not application bug.

Exit Code

139

Segmentation fault.

Application issue.

Already you can see

Engineer needs Linux knowledge too.

Step 6

Suppose describe says

OOMKilled

Question

Why?

Need memory metrics.

Engineer opens

Grafana.

Checks

Memory graph.

400Mi

450Mi

500Mi

512Mi

OOM

Now root cause confirmed.

Step 7

Suppose

Not OOM.

Engineer checks Events.

Readiness probe failed

Interesting.

Now

Need to inspect probe.

YAML

readinessProbe:

httpGet:

path: /health

Engineer curls endpoint.

Bash

curl localhost:8080/health

Gets

500 Internal Server Error

Application unhealthy.

Step 8

Suppose

No probe issue.

Now engineer checks

```
Bash
kubectl get svc
```

Maybe Service missing.

Step 9

Checks

```
Bash
kubectl get endpoints
```

No endpoints.

Hmm.

Step 10

Checks

```
Bash
kubectl exec
```

Runs

```
Bash
nslookup postgres
```

Gets

```
No such host
```

DNS issue.

Step 11

Maybe DNS fine.

Now

```
Bash
telnet postgres 5432
```

Connection refused.

Database down.

Step 12

Maybe DB fine.

Now

Kafka.

Redis.

External API.

Vault.

S3.

IAM.

Secrets.

ConfigMaps.

Every dependency is checked.

Real investigation looks like this

Alert

↓

kubectl get pod

↓

kubectl describe

↓

logs

↓

previous logs

↓

events

↓

exit code

↓

metrics

↓

```

node
↓
dependencies
↓
network
↓
DNS
↓
application
↓
root cause

```

Notice

This isn't one command.

It's an investigation.

So what does YOUR operator automate?

This is the important part.

Imagine your operator watching every pod.

Pod restarted.

Immediately it gathers

- Pod object
- Events
- Previous logs
- Exit code
- Restart count
- Node conditions
- Container state
- Probe failures
- Resource limits
- Resource requests

Instead of engineer manually checking everything.

Then your operator applies rules.

Example

Rule

```
ExitCode == 137
AND
Reason == OOMKilled
```

↓

Diagnosis

```
Likely OOM
Confidence 99%
```

Rule

```
Readiness Probe Failed
AND
HTTP 500
```

↓

Diagnosis

```
Application unhealthy
```

Rule

```
ImagePullBackOff
```

↓

Diagnosis

```
Image cannot be downloaded
```

Recommendation

```
Verify image tag
Check registry credentials
```

Rule

```
Secret not found
```

↓

Recommendation

```
Create missing secret
payment-secret
```

Rule

PVC Pending

↓

Recommendation

StorageClass unavailable

Rule

Node Memory Pressure

Multiple pods restarted

↓

Recommendation

Node resource exhaustion

This is NOT AI

Many people think

"We'll use LLM."

Don't.

Version 1 should be deterministic.

Simple rule engine.

Example

```
Go
if exitCode == 137 &&
lastState.Reason == "OOMKilled" {

return Diagnosis{
Cause: "Out Of Memory",
Confidence: 99,
Recommendation: "Increase memory limit",
}
}
```

Simple.

Reliable.

Fast.

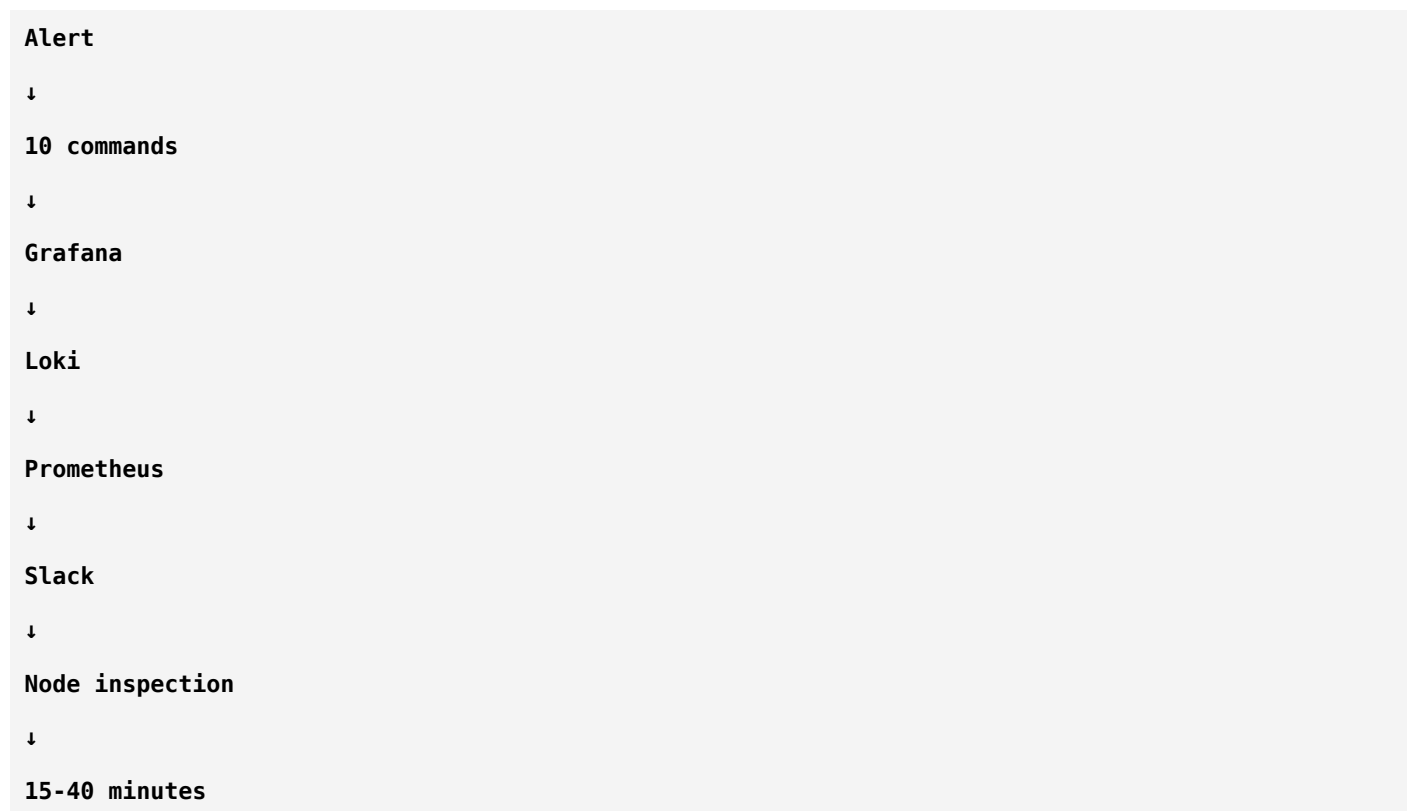
How much time does this save?

Let's be realistic.

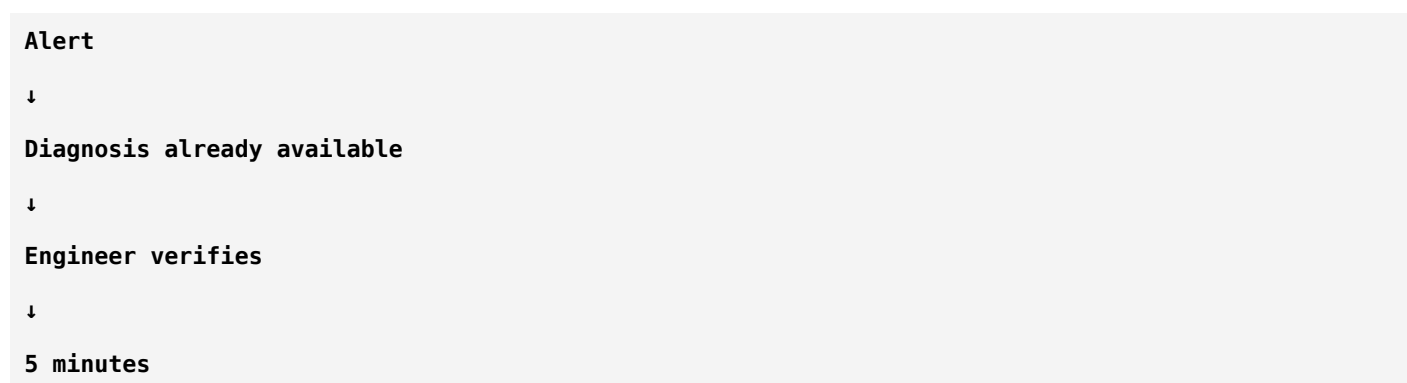
Today

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Engineer spends



Your operator



Notice

It doesn't eliminate engineers.

It eliminates repetitive investigation.

One important design change I would make

I actually wouldn't call it **Restart Analyzer Operator** anymore.

Because after thinking through it with you, the scope is broader.

It shouldn't analyze only restarts.

It should analyze **pod failures** in general.

A pod can fail without accumulating many restarts. It might be stuck in **Pending**, **ImagePullBackOff**, **CreateContainerConfigError**, or fail once and never restart. Those are just as important operationally.

I'd rename it something like:

- **Pod Failure Analyzer Operator** (my favorite)
- Kubernetes Incident Analyzer
- Kubernetes Failure Investigator
- Pod Diagnosis Operator

That gives you room to support many failure modes while still using the same controller architecture.

Before you write code

I would spend time becoming the "human analyzer" first.

Take one failure at a time and learn how to diagnose it manually.

For example:

1. OOMKilled
2. CrashLoopBackOff
3. ImagePullBackOff
4. ErrImagePull
5. CreateContainerConfigError
6. CreateContainerError
7. FailedMount
8. FailedScheduling
9. Readiness Probe Failure
10. Liveness Probe Failure
11. DNS Resolution Failure
12. Secret Missing
13. ConfigMap Missing
14. PVC Pending
15. Node NotReady

For each one, answer four questions:

- **What exactly happened inside Kubernetes?**
- **How does an engineer identify it manually?**
- **Which Kubernetes objects, events, logs, or metrics provide the evidence?**
- **What recommendation should the operator produce?**

If you can answer those questions for each failure mode, implementing the operator in Go becomes much more straightforward. The hard part isn't the Go code—it's encoding operational knowledge into deterministic reconciliation logic. That's exactly what makes operators valuable.

yeah i will do that, but first i want to know whether applying conditionals is really the best way because in our condition we may miss a lot of errors which we are not aware because in production only these 15 types of error not used to happen, there may be a lot of different types of errors also, how we would be able to capture all types of scenario?
 Show moreShow less

Real talk:

This question is exactly where the project becomes interesting.

You just asked the same question that separates a **toy project** from a tool that people could actually use.

The short answer is:

No, a giant if-else chain is NOT the best solution.

But...

Neither is trying to automatically understand every possible production failure.

Let's understand why.

First misconception

Many people think:

Production has infinite errors.
 How can we handle them all?

Actually, production has **infinite root causes**, but a **finite number of observable symptoms**.

That's a huge difference.

For example

Application bug

```
Go
panic("nil pointer")
```

Application bug

```
Go
panic("index out of range")
```

Application bug

```
Go
database returned nil
```

Three completely different bugs.

Kubernetes sees

```
Exit Code = 1
Container Restarted
```

To Kubernetes,

all three look almost identical.

Your operator should **not** try to understand application logic.

Think like Kubernetes

Kubernetes itself doesn't know

```
Java bug
Go bug
Python bug
React bug
```

It only knows

```
Container exited
Exit code 1
```

That's enough for Kubernetes.

Your operator should also think this way.

The Operator is NOT replacing developers

Suppose logs say

```
panic:
invalid customer id
```

Your operator should never try

```
Customer service has invalid IDs.
```

Impossible.

Instead

```
Application crashed.
Exit code 1.
Panic detected in logs.
Recommendation:
Inspect application logs.
```

That's already useful.

So what should the operator diagnose?

Not business logic.

Infrastructure failures.

These are much more finite.

Example categories

```
Storage
Networking
Scheduling
Memory
CPU
Configuration
Secrets
Image
Node
Container lifecycle
Kubernetes API
Security
Admission
Probes
```

These categories cover most Kubernetes operational problems.

Let's compare

Application problem

```
Order calculation wrong
```

Operator cannot know.

Infrastructure problem

PVC failed to mount

Operator absolutely knows.

Application problem

Null pointer exception

Operator doesn't know why.

Infrastructure problem

OOMKilled

Operator knows immediately.

Application problem

SQL syntax error

Operator cannot fix.

Infrastructure problem

Secret missing

Operator knows.

Notice the boundary.

So should we use conditions?

Yes.

But not this

```
Go
if exitCode == 137 {
  ...
}

if exitCode == 1 {
  ...
}

if exitCode == 143 {
  ...
}
```

This becomes horrible.

Instead

build a **rule engine**.

Imagine

Instead of code

You define rules.

```
Rule
Name:
OOMKilled
Conditions
LastState.Reason == OOMKilled
Evidence
ExitCode 137
Diagnosis
Out of Memory
Confidence
99%
Recommendation
Increase memory limit.
```

Another

```
Rule
ImagePullBackOff
Condition
Waiting.Reason == ImagePullBackOff
Recommendation
Verify image tag.
```

Your controller becomes

```
Collect Facts
↓
Run Rules
↓
Return Diagnoses
```

Not

1000 if else statements

Even better

Assign confidence.

Example

OOMKilled

Confidence 99%

DNS issue

Confidence 60%

Application panic

Confidence 40%

Don't pretend certainty when you don't have it.

Think like a doctor

Patient

Fever

Doctor says

Possible viral infection

Confidence 70%

Not

You definitely have COVID.

Your operator should behave similarly.

Rule Engine Architecture

Instead of

Controller

↓

if else

↓

Result

Do

```

Controller
↓
Evidence Collector
↓
Rule Engine
↓
Diagnosis
↓
Status
↓
Events

```

Now adding new rules becomes easy.

Example

```

Go
type Rule interface {
    Match(Facts) bool
    Diagnose(Facts) Diagnosis
}

```

Now every diagnosis is independent.

```

OOM Rule
Image Rule
Probe Rule
PVC Rule
DNS Rule
Node Rule

```

Beautiful architecture.

But what about unknown errors?

Excellent question.

Unknown errors should exist.

That's not failure.

Imagine

```

Collected

```

Events
Logs
Exit Code
Restart Count
Conditions
Node Status

None of the rules match.

Return

Diagnosis
Unknown
Evidence
Exit Code 42
Logs collected
Recommendation
Manual investigation required.

That's exactly what production software does.

Never guess.

Can we make it smarter later?

Absolutely.

Version 1

Rules

Version 2

Rules
+
Pattern Matching

Version 3

Rules
+
Statistical scoring
+
Historical incidents

Rules

+

LLM-assisted summaries

Notice something.

The rules never disappear.

Even if you later add AI,

the deterministic rules remain because they're fast, explainable, and reliable.

What do companies actually do?

This is important.

Most observability platforms **don't** try to understand every error either.

For example:

Prometheus

- Collects metrics.
- Doesn't diagnose.

Loki

- Stores logs.
- Doesn't diagnose.

Grafana

- Visualizes data.
- Doesn't diagnose.

Datadog

- Detects anomalies and correlates telemetry.
- Has some rule-based insights, but still expects engineers to investigate many issues.

Dynatrace

- Uses topology and AI-assisted root-cause analysis.
- Still doesn't magically understand arbitrary application bugs.

Splunk

- Searches and correlates logs.
- Doesn't inherently know business logic.

Notice the pattern?

They help you **narrow the search space**. They don't promise to identify every possible failure.

Here's how I'd scope your project

Don't advertise it as:

"Automatically finds the root cause of every Kubernetes problem."

That's not realistic.

Instead say:

"A Kubernetes operator that automatically collects evidence, correlates common infrastructure failure signals, and provides explainable diagnoses for the most frequent Kubernetes operational issues."

That's an honest claim—and it's a technically strong one.

One design idea I'd encourage from day one

Don't think in terms of **errors**.

Think in terms of **evidence**.

Your operator's first responsibility is to build a complete picture:

```
Pod Status
Container Status
Restart Count
Last State
Exit Code
Events
Logs
Previous Logs
Node Conditions
Resource Requests/Limits
Probe Results
PVC Status
Secret References
ConfigMap References
```

Then your second responsibility is to interpret that evidence.

That separation is powerful because even when the diagnosis is "Unknown," you've already saved the engineer from running ten different commands. Sometimes **collecting all the evidence in one place is 80% of the value**, even if the final conclusion still requires human judgment. That mirrors how experienced platform engineers actually work in production.

help to know about these failures and each information i should be knowing about them, taking 2 or 3 at a time, so that i may have enough content to deeply understand each one

Real talk:

This is exactly how you should approach this project.

Don't try to learn all 15 failures together. Learn **2-3 failures deeply**, then implement support for them in your operator, then move to the next ones.

I would recommend this order because each failure teaches new Kubernetes concepts.

Phase	Failures	Concepts You'll Learn
1	OOMKilled, CrashLoopBackOff, ImagePullBackOff	Container lifecycle, restart policy, events, logs
2	Liveness Probe Failure, Readiness Probe Failure, Startup Probe Failure	Kubelet, probes, health checking
3	CreateContainerConfigError, CreateContainerError, Secret/ConfigMap Missing	Config, volumes, environment variables
4	FailedScheduling, Pending Pods	Scheduler, taints, affinity, resources
5	FailedMount, PVC Pending	Storage, CSI, PV/PVC lifecycle
6	DNS Failure, Network Issues	Services, CoreDNS, networking

Today let's cover the **first two**. These are probably the most common failures you'll see in production.

1. OOMKilled

This is probably the single most common production issue.

What exactly is OOMKilled?

OOM = **Out Of Memory**

Imagine your container has this limit:

```
YAML
resources:
  limits:
    memory: 512Mi
```

Your application starts consuming memory

```
100Mi
↓
200Mi
↓
350Mi
↓
490Mi
↓
512Mi
```

↓

530Mi

As soon as it tries to exceed **512Mi**, Linux's **OOM Killer** terminates the process.

Notice something important.

Kubernetes does NOT kill the process.

Linux Kernel does.

Kubernetes only observes:

"Container exited because it was OOMKilled."

This distinction is important.

What happens internally?

Let's assume this Deployment

YAML

```
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 1
```

Container starts.

Memory keeps increasing.

Eventually

Memory > Limit

Linux Kernel

↓

Kills process

↓

Container exits

↓

Kubelet notices

↓

Container status becomes

OOMKilled

↓

Restart Policy = Always

↓

Container restarted

↓

Memory grows again

↓

Killed again

↓

Restart loop

What does engineer first notice?

Usually

```
Bash
kubectl get pods
```

Output

```
NAME
payment-api
STATUS
Running
RESTARTS
17
```

Interesting.

Pod is Running.

But restarts keep increasing.

Or

```
CrashLoopBackOff
```

How does engineer investigate?

Step 1

```
Bash
kubectl describe pod payment-api
```

Important section

Last State:

Terminated

Reason:

OOMKilled

Exit Code:

137

Immediately experienced engineers think

Memory problem.

Step 2

Verify memory graph

Usually in

- Grafana
- Prometheus
- Datadog

Graph

100Mi

↓

200Mi

↓

400Mi

↓

510Mi

↓

512Mi

↓

Killed

Root cause confirmed.

Step 3

Check resource limits

Bash

```
kubectl describe pod
```

or

```
Bash
kubectl get pod -o yaml
```

Find

```
YAML
resources:
requests:
memory: 256Mi
limits:
memory: 512Mi
```

Maybe limit too small.

Step 4

Read logs

```
Loading records...
Processing data...
Killed
```

Sometimes logs don't even mention memory.
Because kernel killed the process immediately.

How would your operator diagnose it?

Evidence collected

```
LastState.Reason = OOMKilled
ExitCode = 137
RestartCount = 15
MemoryLimit = 512Mi
```

Diagnosis

```
Cause
Out Of Memory
Confidence
99%
```

Recommendation

Increase memory limit
OR
Investigate memory leak

Notice

The operator shouldn't say

"Memory leak."

Maybe application legitimately needs more memory.

Only developer can know.

What concepts you learn

- Linux OOM Killer
- Resource Requests
- Resource Limits
- Kubelet
- Container Status
- Exit Codes
- Restart Policy

One failure teaches six Kubernetes concepts.

2. CrashLoopBackOff

This one confuses almost everyone.

Biggest misconception

People think

CrashLoopBackOff is the error.

It isn't.

It is Kubernetes saying

"I tried restarting this container multiple times. It keeps crashing. I'm slowing down before trying again."

It is a **state**, not a root cause.

Example

Application

```
Go
func main() {
    panic("database connection failed")
}
```

Container

↓

Starts

↓

Panic

↓

Exits

↓

Restart

↓

Panic

↓

Restart

↓

Panic

↓

Restart

Eventually Kubernetes says

CrashLoopBackOff

Notice

The real problem is

panic

CrashLoopBackOff is only the consequence.

Why "BackOff"?

Suppose Kubernetes restarted immediately forever.

```

Start
↓
Crash
↓
Start
↓
Crash
↓
Start
↓
Crash
↓
1000 times/sec

```

That would waste CPU.

Instead Kubernetes waits.

```

Restart
↓
Wait 10 sec
↓
Restart
↓
Wait 20 sec
↓
Restart
↓
Wait 40 sec
↓
Restart

```

This increasing wait time is called

Exponential Backoff.

What engineer sees

```

Bash
kubectl get pods

```

STATUS

`CrashLoopBackOff`

But still doesn't know

WHY.

Investigation

Step 1

Bash

```
kubectl describe pod
```

Events

Back-off restarting failed container

Not useful.

Still don't know cause.

Step 2

Current logs

Bash

```
kubectl logs payment-api
```

Sometimes

Starting...

Connecting...

Nothing useful.

Step 3 (Most Important)

Bash

```
kubectl logs payment-api --previous
```

Because previous container actually crashed.

Output

panic

database timeout

stack trace

Now engineer knows.

Step 4

Check Exit Code

Exit Code

1

Generic failure.

Need logs.

Sometimes

139

Segmentation fault.

Sometimes

137

Actually OOMKilled.

See?

CrashLoopBackOff itself isn't enough.

How would your operator work?

It sees

Container State

Waiting

Reason

CrashLoopBackOff

Not enough.

Operator automatically collects

- Previous Logs
- Exit Code
- Last State

- Events

Suppose

```
Last State
OOMKilled
```

Then

Operator says

```
CrashLoopBackOff caused by OOMKilled
```

Another example

```
Last State
Exit Code 1
Logs
panic
nil pointer dereference
```

Operator says

```
CrashLoopBackOff
Likely application panic
Recommendation
Inspect application stack trace
```

Notice

CrashLoopBackOff isn't diagnosed directly.

The operator **keeps digging**.

Exactly like an experienced engineer.

Difference between OOMKilled and CrashLoopBackOff

This is extremely important.

Imagine

```
Container exceeded memory
```

Result

```
OOMKilled
```

Kubernetes restarts it.

It exceeds memory again.

Again killed.

Again restarted.

After multiple retries

CrashLoopBackOff

Now

Question

What's the root cause?

Answer

OOMKilled

CrashLoopBackOff is only what Kubernetes reports after repeated failures.

Think of it like

Person slips

↓

Falls

↓

Hospitalized

Root cause

Slipped

Not

Hospitalized

How this changes your operator design

Initially, we discussed matching one rule per error. After understanding these two failures, you can already see that's not enough.

Your operator should work as a **multi-stage diagnosis pipeline**:

Pod restarted

↓

Collect evidence

↓

**Identify immediate state
(CrashLoopBackOff)**

↓

Investigate underlying cause
(OOMKilled, panic, probe failure, etc.)

↓

Generate diagnosis

↓

Suggest recommendation

This is much closer to how senior SREs debug incidents. They don't stop at the first status they see—they keep asking **"why?"** until they reach the most likely root cause.

For the next session, I'd cover:

1. **ImagePullBackOff** (registry, authentication, image tags)
2. **ErrImagePull** (how it differs from ImagePullBackOff)
3. **CreateContainerConfigError** (missing Secrets, ConfigMaps, bad configuration)

Those three build naturally on today's concepts and will give you a solid understanding of how Kubernetes brings containers from "scheduled" to "running."

yeah go ahead

Real talk:

These three failures happen **before your application even starts**.

That's why they're extremely important.

A beginner usually thinks:

"My application isn't working."

An experienced Kubernetes engineer asks:

"Did my application even get a chance to start?"

These failures teach exactly that.

First understand the container lifecycle

Before learning these failures, understand where they happen.

A Pod doesn't magically become Running.

It goes through multiple stages.

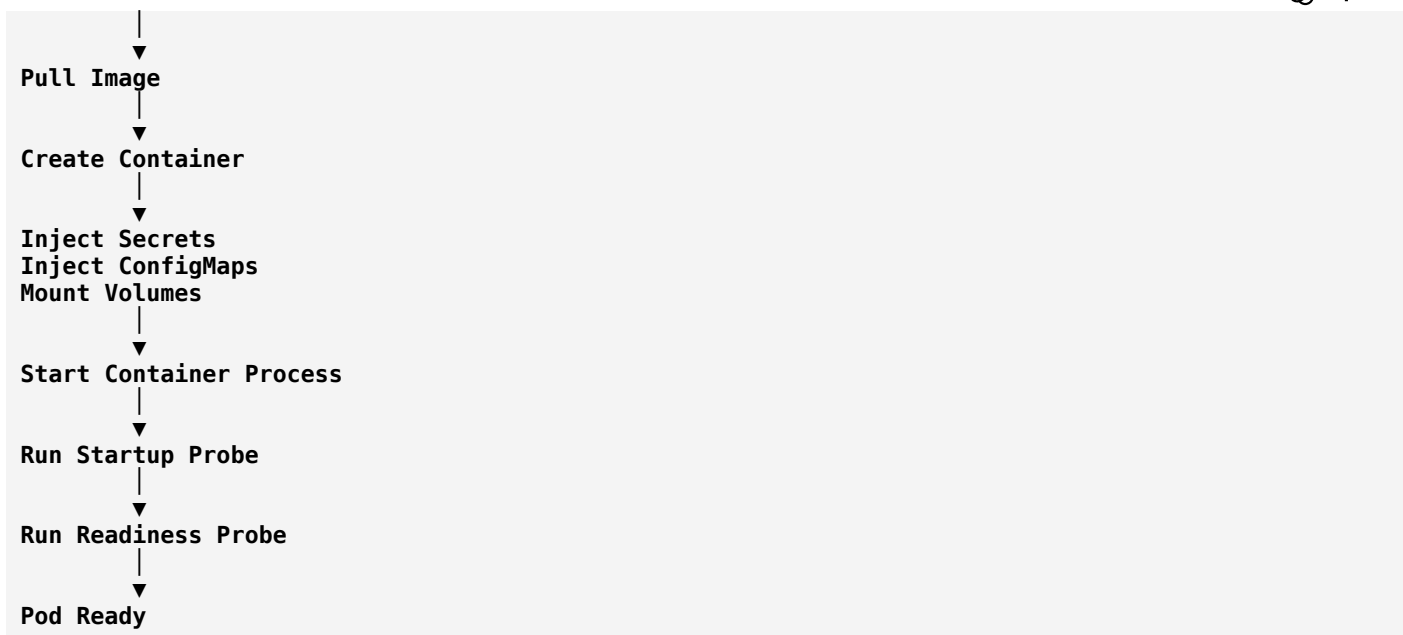
Deployment Created

↓

Scheduler selects a Node

↓

Kubelet receives Pod



Now notice something.

Different failures happen at different stages.

Stage	Possible Failure
Pull Image	ErrImagePull, ImagePullBackOff
Create Container	CreateContainerConfigError
Start Process	CrashLoopBackOff
Running	OOMKilled
Health Check	Readiness/Liveness Probe Failure

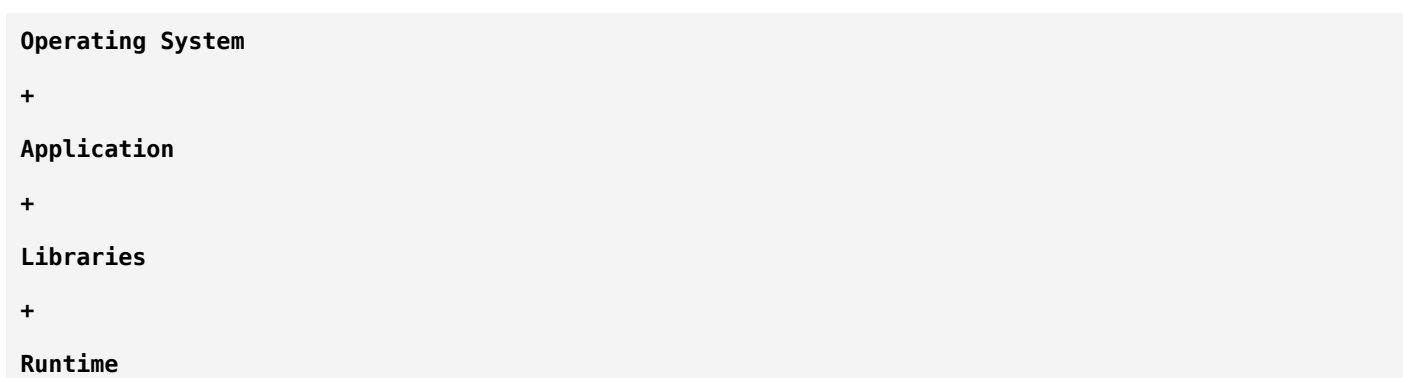
Knowing **where** a failure occurs immediately narrows your investigation.

1. ErrImagePull

This is the **first failure** that can happen.

What is an Image?

Think of Docker Image as



Example

```
YAML
image: nginx:1.27
```

or

```
YAML
image: mycompany/payment:v2
```

When Kubernetes schedules your Pod,
the Kubelet asks the container runtime

```
"Please download this image."
```

If downloading fails,
you get

```
ErrImagePull
```

Notice

Your application has **not started yet**.

Common reasons

Wrong image name

Example

```
YAML
image: nginx:latest
```

Notice typo.

Should be

```
YAML
image: nginx:latest
```

Wrong tag

```
YAML
image: nginx:v999
```

No such version exists.

Registry returns

404

Private Registry

Example

YAML

```
image: mycompany/backend:v1
```

Registry requires authentication.

No credentials.

Download denied.

Network issue

Node cannot reach Docker Hub.

Registry unavailable

Docker Hub outage.

AWS ECR outage.

GCR outage.

What engineer sees

Bash

```
kubectl get pods
```

Output

STATUS

```
ErrImagePull
```

Investigation

Step 1

Bash

```
kubectl describe pod
```

Events

```
Failed to pull image
```

```
rpc error
manifest unknown
```

or

```
authentication required
```

or

```
image not found
```

Notice

Events are much more useful here than logs.

Why?

Because

Container never started.

No logs exist.

Your operator

Collect

```
Pod Status
Waiting Reason
Events
```

Suppose Event says

```
manifest unknown
```

Diagnosis

```
Cause
Invalid Image Tag
Confidence
99%
Recommendation
Verify image tag.
```

Another example

```
authentication required
```

Recommendation

Check `imagePullSecrets`

Important lesson

Not every failure needs logs.
Some failures happen before logs exist.

2. ImagePullBackOff

This confuses almost everyone.

Biggest misconception

People think
`ErrImagePull`
and
`ImagePullBackOff`
are different problems.
They aren't.
One comes after the other.

Timeline

```

Pull image
↓
Failed
↓
ErrImagePull
↓
Retry
↓
Failed
↓
Retry
↓
Failed

```

```
↓
Retry
↓
ImagePullBackOff
```

Exactly like CrashLoopBackOff.

ImagePullBackOff means

Kubernetes has repeatedly failed to download the image and is now waiting before trying again.

Again

BackOff means

Exponential delay.

Example

```
Try
↓
Fail
↓
10 sec
↓
Try
↓
Fail
↓
20 sec
↓
Try
↓
Fail
↓
40 sec
↓
Try
```

Engineer investigates exactly the same way

```
Bash
kubectl describe pod
```

Events

```
Failed to pull image
Back-off pulling image
```

Operator

should not diagnose

```
ImagePullBackOff
```

Instead

look deeper

Events

↓

authentication

↓

DNS

↓

manifest unknown

↓

connection timeout

and diagnose

those.

Notice the pattern?

Exactly like

CrashLoopBackOff.

Difference

Status	Meaning
ErrImagePull	Pull failed first time
ImagePullBackOff	Kubernetes keeps retrying and backing off

Root cause

Same.

3. CreateContainerConfigError

This one teaches a completely different concept.

The image downloaded successfully.

Container has **still not started**.

Why?

Because Kubernetes needs to prepare container configuration.

Imagine Deployment

```
YAML
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: payment-secret
      key: password
```

Question

Does Secret exist?

Suppose

No.

Can Kubernetes start container?

No.

Because environment variable cannot be injected.

Result

```
CreateContainerConfigError
```

Notice

Application never started.

Again

No logs.

Other reasons

Missing ConfigMap

```
YAML
configMapRef:
  name: app-config
```

ConfigMap deleted.

Missing Secret

```
YAML
secretKeyRef
```

Secret deleted.

Invalid Volume Configuration

Bad environment variable reference

Engineer investigation

```
Bash
kubectl describe pod
```

Events

```
Secret "payment-secret" not found
```

Done.

Problem found.

Another

```
ConfigMap "app-config" not found
```

No need

for

```
Bash
kubectl logs
```

Because application never started.

Your operator

Collect

Events

Pod Status

Waiting Reason

Volumes

Secrets
ConfigMaps

Diagnosis

Cause
Missing Secret
Confidence
100%
Recommendation
Create Secret
payment-secret

Another

ConfigMap missing

Recommendation

Apply ConfigMap before Deployment.

Let's compare these three

Failure	Did Image Download?	Did Container Start?	Logs Available?	Main Investigation
ErrImagePull	✗ No	✗ No	✗ No	Events
ImagePullBackOff	✗ No	✗ No	✗ No	Events
CreateContainerConfigError	✓ Yes	✗ No	✗ No	Events + Spec

Notice a pattern?

For these failures,

Events are king.

Logs don't exist because the application never executed.

A better architecture for your operator

Originally, we thought about collecting everything (logs, events, metrics) for every failure. That would work, but it's inefficient.

A smarter operator would follow an investigation strategy similar to an experienced engineer:

Pod Failure
↓

What phase failed?

Image Pull?



Read Events

Container Creation?



Read Events + Pod Spec

Container Started?



Read Logs + Exit Code

Running?



Read Metrics + Node State

Now your operator isn't blindly collecting everything. It first determines **where in the Pod lifecycle the failure occurred**, then gathers only the evidence relevant to that phase.

This is more scalable, faster, and mirrors how senior Kubernetes engineers investigate incidents. It's also a much stronger design to explain during interviews because it shows you're modeling the Kubernetes lifecycle rather than writing a collection of unrelated rules.

Now next errors

Real talk:

Today we'll cover **probe failures**, because they are among the most misunderstood Kubernetes concepts. Almost every production Kubernetes application uses probes, and many outages are caused by incorrectly configured probes rather than bugs in the application itself.

Before understanding probe failures

You need to answer one question.

How does Kubernetes know whether your application is healthy?

It doesn't.

Kubernetes doesn't understand Java, Go, Python, Node.js, etc.

It only knows:

"I'll ask the application, and the application must tell me whether it's healthy."

This is exactly what probes do.

Three types of probes

Many beginners memorize them. Don't.

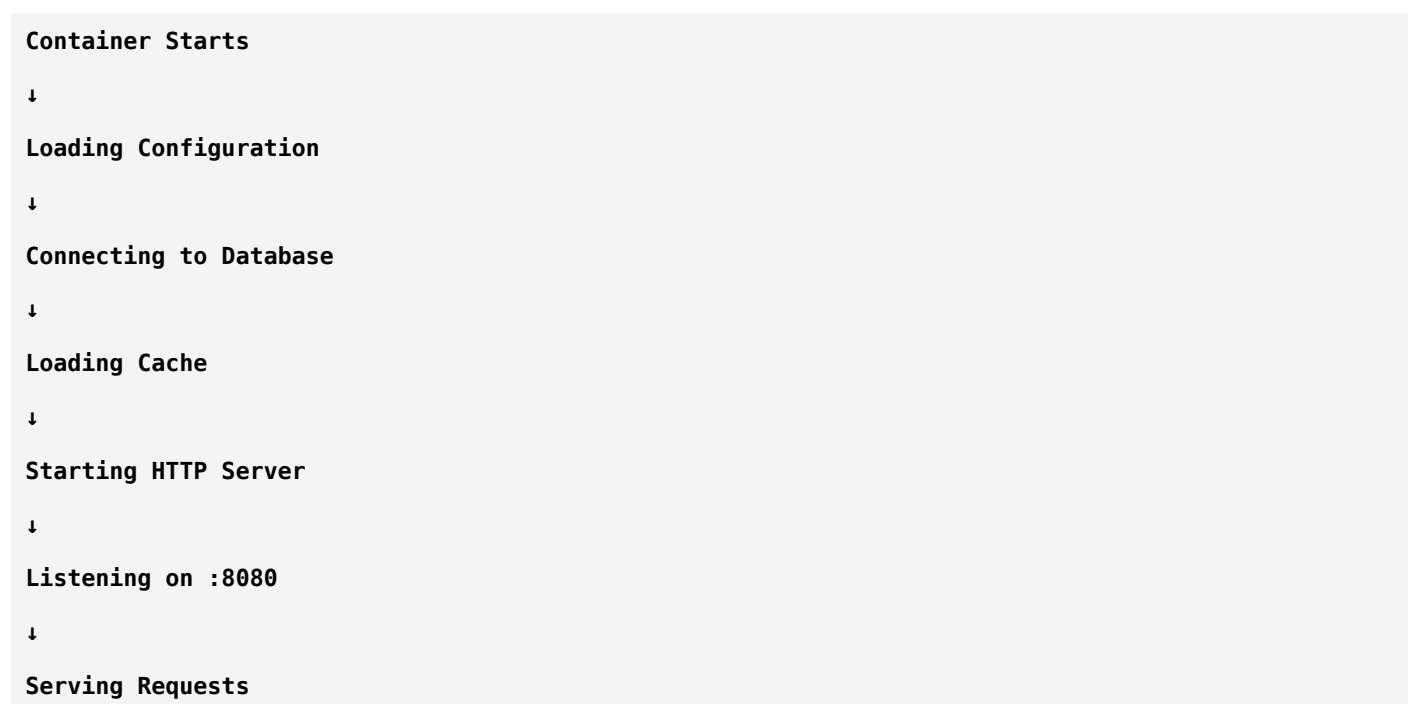
Instead, think about the questions Kubernetes is asking.

Probe	Kubernetes asks
Startup Probe	"Have you finished starting?"
Readiness Probe	"Can I send traffic to you?"
Liveness Probe	"Are you still alive?"

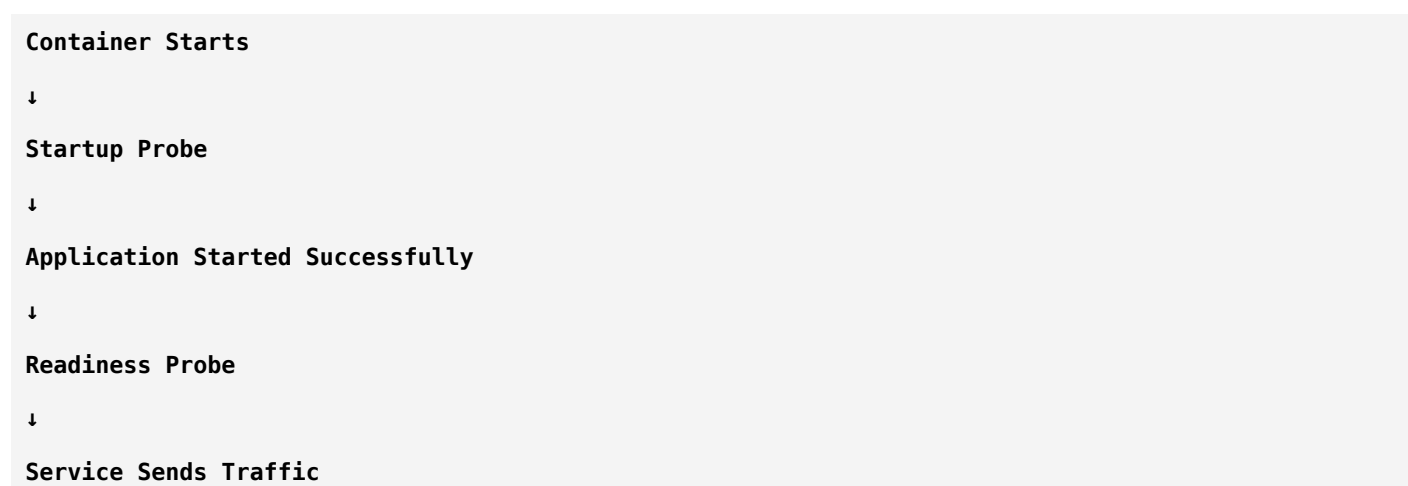
Those are three completely different questions.

Complete lifecycle

Imagine this Go application.



Now let's see where each probe runs.



```
↓
Liveness Probe
↓
Application Keeps Running
```

Notice

Readiness and Liveness don't even matter until Startup succeeds.

1. Startup Probe Failure

This is the easiest one.

What problem does Startup Probe solve?

Imagine your application takes **90 seconds** to start.

Why?

Maybe it

- loads a huge ML model
- warms cache
- performs database migrations
- downloads configuration
- initializes thousands of records

This is normal.

But Kubernetes doesn't know that.

Suppose Startup Probe is

```
YAML
startupProbe:
  httpGet:
    path: /health
    port: 8080

  periodSeconds: 10

  failureThreshold: 3
```

What does this mean?

Kubernetes waits

```
10 sec
↓
```

```
10 sec
```

```
↓
```

```
10 sec
```

Only 30 seconds total.

Application actually needs

90 seconds.

Result

```
Probe Failed
```

```
↓
```

```
Container Restarted
```

```
↓
```

```
Probe Failed
```

```
↓
```

```
Restarted
```

```
↓
```

```
CrashLoopBackOff
```

Nothing is wrong with the application.

Probe configuration is wrong.

What engineer sees

```
Bash
```

```
kubectl get pods
```

```
STATUS
```

```
CrashLoopBackOff
```

Looks like application is crashing.

Actually

It never got enough time to start.

Investigation

```
Bash
```

```
kubectl describe pod
```

Events

```
Startup probe failed
connection refused
```

Now engineer checks

```
YAML
startupProbe:
```

and realizes

Application startup time

```
90 sec
```

Probe timeout

```
30 sec
```

Problem solved.

Operator Diagnosis

Evidence

```
Events
Startup probe failed
Restart Count
High
Application eventually starts when timeout increased
```

Diagnosis

```
Cause
Startup probe timeout too aggressive
Confidence
95%
Recommendation
Increase failureThreshold
or
Increase periodSeconds
```

2. Readiness Probe Failure

This one is extremely important.

Biggest misconception

People think

Readiness failure means

Application crashed.

Wrong.

Application is still running.

Kubernetes simply stops sending traffic.

Imagine

Three Pods

Pod A
Running
Ready
✓
Pod B
Running
Ready
✓
Pod C
Running
Not Ready
x

Service

Client
↓
Service
↓
Pod A
↓
Pod B

Pod C receives

Zero traffic.

Notice

Pod C still exists.

Still consumes CPU.

Still consumes Memory.

It just doesn't receive requests.

Why?

Suppose application depends on

Database.

Database goes down.

Application decides

```
"I'm alive.
```

```
But I cannot serve requests."
```

Health endpoint returns

```
HTTP 503
```

Readiness probe fails.

Kubernetes removes Pod from Service endpoints.

Beautiful.

No restart needed.

Engineer investigation

```
Bash
```

```
kubectl describe pod
```

Events

```
Readiness probe failed
```

```
HTTP probe failed
```

```
503
```

Now engineer asks

Why is /health returning 503?

Maybe

- database down
- Redis unavailable
- Kafka unavailable
- cache loading

Now investigation shifts to dependencies.

Operator Diagnosis

Evidence

```
Pod Running
Container Running
Readiness Failed
HTTP 503
```

Diagnosis

```
Application unhealthy
Traffic removed
Investigate dependencies
```

Notice

Operator should NOT recommend restart.

Restart won't fix database outage.

3. Liveness Probe Failure

This one actually causes restarts.

Purpose

Imagine

Your Go application

has

Deadlock.

Example

```
Go
```

```
mutex.Lock()

...

mutex.Lock()
```

Application
doesn't crash.
CPU usage low.
Memory normal.
But
Every request hangs forever.
Kubernetes thinks
Container still running.
Actually
Application is dead.

Liveness Probe solves this.
Every few seconds
Kubernetes asks

```
GET /health
```

Application doesn't respond.
Timeout.
Probe fails.
After repeated failures
Kubelet restarts container.

Timeline

Application Running

↓

Deadlock

↓

Liveness Probe

↓

Timeout

↓

Failure

```
↓
Failure
↓
Failure
↓
Container Restarted
```

Engineer investigation

```
Bash
kubectl describe pod
```

Events

```
Liveness probe failed
Timeout
Container restarted
```

Now engineer checks

- application logs
- thread dump
- CPU usage
- deadlock
- infinite loop

Operator Diagnosis

Evidence

```
Restart Count Increasing
Liveness Probe Failed
Exit Code 143
```

Diagnosis

```
Container restarted due to failed liveness checks
Investigate application responsiveness
```

Notice

Operator cannot know

Whether
Deadlock
Infinite loop
Slow database
GC pause
caused timeout.
It should not guess.

Difference between all three

This table is worth memorizing because interviewers love this question.

Probe	Is Container Running?	Receives Traffic?	Restarted?
Startup Failed	✗ Usually No	✗ No	✓ Yes
Readiness Failed	✓ Yes	✗ No	✗ No
Liveness Failed	✓ Yes (initially)	Usually Yes	✓ Yes

Real production example

Imagine
Payment Service

```
Starting
↓
Loading configuration
↓
Connecting to PostgreSQL
↓
Loading 2GB fraud model
↓
Listening
↓
Serving requests
```

Correct probe behavior

```
Startup Probe
↓
Waits
```

```

↓
Application starts
↓
Readiness Probe
↓
Database available
↓
Traffic begins
↓
Liveness Probe
↓
Runs forever

```

Now imagine PostgreSQL goes down.

Startup?

Already completed.

No effect.

Liveness?

Application still responds.

Passes.

Readiness?

Returns 503.

Fails.

Kubernetes removes Pod from Service.

Exactly what you want.

No unnecessary restart.

How your operator should investigate probe failures

Instead of immediately saying:

"Readiness probe failed."

It should ask a series of questions, almost like a detective:

```

Probe Failure
↓
Which probe?
↓

```

```

Startup
Readiness
Liveness

↓

HTTP?
TCP?
Exec?

↓

Status Code?
Timeout?
Connection Refused?

↓

Is the container running?

↓

Was it restarted?

↓

What happened just before the failure?

```

That investigation tree is much more useful than a single rule like:

```

Go
if readinessFailed {
    ...
}

```

because the **same readiness failure** could be caused by:

- database unavailable
- Redis unavailable
- slow startup
- bad `/health` implementation
- network policy blocking traffic
- DNS resolution failure

Your operator should collect enough evidence to narrow it down without pretending it knows the exact application bug.

Your learning assignment before the next session

These concepts become much clearer if you reproduce them.

Create three tiny Go HTTP servers:

1. **Startup Probe Failure:** Sleep for 60 seconds before starting the HTTP server. Configure the startup probe to fail after 30 seconds and observe the restarts.

2. **Readiness Probe Failure:** Make `/health` return HTTP 503 when a boolean flag is false. Watch the Pod remain running while disappearing from the Service endpoints.
3. **Liveness Probe Failure:** Add an endpoint that intentionally blocks forever (simulating a deadlock), causing the liveness probe to fail and Kubernetes to restart the container.

Watching these behaviors in a Kind cluster will teach you far more than just reading about them, and they'll give you real scenarios to test your future operator against.

now next errors, i have to finish reading about all errors today only and from tomorrow we will start to build the architecture design and required planning and resources for this task.

Real talk:

Good plan. Today let's finish the failure modes. Tomorrow we'll design the operator like we're building a real CNCF-quality project, not just writing a controller.

From now on, don't memorize commands. Memorize the **Kubernetes component** responsible for the failure. That's how experienced engineers think.

1. CreateContainerError

This is commonly confused with `CreateContainerConfigError`.

They are **not the same**.

Error	Meaning
<code>CreateContainerConfigError</code>	Kubernetes couldn't prepare the configuration before creating the container.
<code>CreateContainerError</code>	Kubernetes had the configuration, but failed while creating the container.

Where does it happen?

Lifecycle

```

Image Pulled ✓
↓
Config Ready ✓
↓
Container Runtime Creates Container ✗

```

The application hasn't started yet.

Common reasons

Invalid command

Example

```
YAML
containers:
- image: nginx
  command:
  - /start.sh
```

But

```
/start.sh
```

doesn't exist.

Container runtime returns

```
exec: "/start.sh": no such file or directory
```

Invalid working directory

```
YAML
workingDir: /abc
```

Directory doesn't exist.

Invalid volume mount

Trying to mount

```
/app/config
```

onto a file instead of directory.

Container runtime problem

Docker

containerd

CRI-O

runtime failed internally.

Rare

but possible.

Investigation

```
Bash
kubectl describe pod
```

Events

```
Error: failed to create container
```

Sometimes

```
exec not found
```

or

```
permission denied
```

Operator diagnosis

Evidence

```
Container Waiting
Reason
CreateContainerError
Events
permission denied
```

Recommendation

```
Verify command
Verify working directory
Verify mounted volumes
```

2. FailedScheduling

This teaches Scheduler internals.

Very important for CKA.

What is scheduling?

Suppose

Three nodes

```
Node A
4 CPU
```

```
8GB
-----
Node B
4 CPU
8GB
-----
```

```
Node C
4 CPU
8GB
```

Deployment

```
YAML
resources:
requests:
cpu: 8
```

Question

Can scheduler place it?

No.

No node has

8 CPU.

Result

```
Pending
```

Events

```
FailedScheduling
```

Other reasons

Not enough Memory

```
0/5 nodes available
Insufficient memory
```

Taints

```
node-role=database
NoSchedule
```

Pod

doesn't tolerate it.

Node Selector

```
YAML
nodeSelector:
  disk: ssd
```

No node
has SSD label.

Affinity
Requires
zone

```
us-east-1a
```

No node there.

PVC waiting
Scheduler waits.

Engineer investigation

```
Bash
kubectl describe pod
```

Events

```
0/4 nodes available
Insufficient cpu
```

or

```
Node had taint
```

or

```
Node selector mismatch
```

Then

```
Bash
kubectl get nodes
```

Check
Capacity
Labels
Taints
Allocatable resources

Operator

Collect

```
Pod Spec
Node Labels
Node Resources
Events
```

Diagnosis

```
No node satisfies scheduling requirements
```

Recommendation

```
Reduce requests
OR
Add more nodes
OR
Fix nodeSelector
```

3. Pending Pod

Many people think
Pending
is an error.
It isn't.
It's a state.

Question

Why Pending?

Need investigation.

Possible reasons

Scheduler

PVC

ImagePull

Admission

Quota

Priority

Pending is

like

CrashLoopBackOff.

Symptom.

Not root cause.

Operator

should never say

Pending Pod

Instead

ask

Why Pending?

4. FailedMount

Very common.

Application needs storage.

PVC

postgres-data

Kubelet tries

Mount Volume

↓

Fails.

Reasons

StorageClass missing

CSI Driver missing

NFS unavailable
 Permission denied
 Read-only filesystem
 Volume already attached

Events

```
FailedMount
```

or

```
Unable to attach volume
```

Engineer

```
Bash
kubectl describe pod
```

Then

```
Bash
kubectl get pvc
```

Then

```
Bash
kubectl describe pvc
```

Then

```
Bash
kubectl get storageclass
```

Operator

Diagnosis

```
PVC exists
StorageClass missing
Cannot mount
```

Recommendation

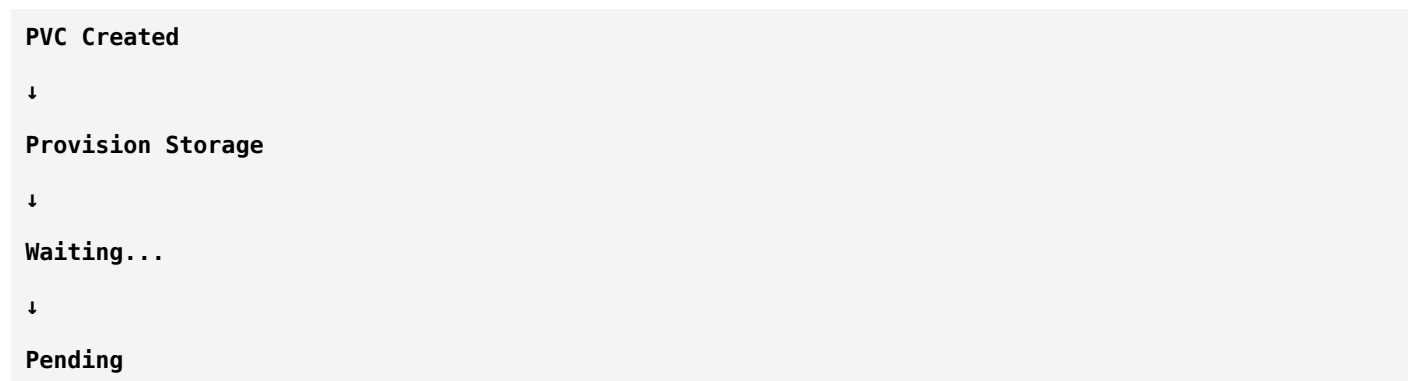
```
Install StorageClass
```

5. PVC Pending

Different from FailedMount.

PVC
itself
never got storage.

Lifecycle



Reasons

No StorageClass
Storage exhausted
CSI failure
Wrong access mode

Engineer

```

Bash
kubectl get pvc
  
```

```

STATUS
Pending
  
```

Describe

```

Bash
kubectl describe pvc
  
```

Events

```

Waiting for first consumer
  
```

or

```

No StorageClass
  
```

Operator

Recommendation

StorageClass missing

6. DNS Resolution Failure

Very common.

Application

tries

```
Go
db.default.svc.cluster.local
```

DNS

fails.

Application

cannot connect.

Engineer

```
Bash
kubectl exec
```

Run

```
Bash
nslookup db
```

Output

```
No such host
```

Then

Check

```
Bash
kubectl get svc
```

Maybe

Service missing.

Or

```
Bash
kubectl get pods --n kube-system
```

CoreDNS

crashed.

Operator

can detect

```
CoreDNS unavailable
```

```
DNS lookup failed
```

7. Secret Missing

Deployment

```
YAML
```

```
env:
```

```
secretKeyRef:
```

```
payment-secret
```

Secret

deleted.

Container

cannot start.

Actually

This usually becomes

CreateContainerConfigError.

Operator

Recommendation

```
Missing Secret
```

```
payment-secret
```

8. ConfigMap Missing

Same concept.

Deployment

references

```
app-config
```

ConfigMap deleted.

Container

never starts.

Usually

CreateContainerConfigError.

9. Node NotReady

Node itself

becomes unhealthy.

Reasons

Disk pressure

Memory pressure

Network lost

Kubelet stopped

Node powered off

Pods

may restart

be evicted

be rescheduled.

Engineer

```
Bash
kubectl get nodes
```

```
STATUS
NotReady
```

Then

```
Bash
kubectl describe node
```

Operator

can correlate

```
20 Pods restarted
Same Node
Node NotReady
```

Likely node failure

This is much more useful than saying

"20 applications failed."

Actually

Node failed.

10. Evicted Pods

Node

runs out of

Memory

Disk

Ephemeral storage.

Kubelet starts removing Pods.

Events

Evicted

Reason

Node had memory pressure

Operator

Recommendation

Node resource exhaustion

11. NetworkPolicy Blocking Traffic

Everything

looks healthy.

Application

cannot reach Database.

Reason

NetworkPolicy.

Engineer

tests

```
Bash
kubectl exec
```

```
Bash
curl
```

Fails.

Operator

can notice

```
NetworkPolicy exists
Connection timeout
Recommend checking ingress/egress rules
```

12. Admission Webhook Failure

Rare

but production clusters have this.

Example

Kyverno

OPA

Mutating Webhook

Validating Webhook

Webhook unavailable.

Pod creation blocked.

Events

```
admission webhook denied request
```

Operator

Recommendation

```
Admission webhook unavailable
```

Complete classification

Instead of memorizing 15 unrelated errors, classify them by **which Kubernetes component is responsible**.

Component	Failures
Scheduler	FailedScheduling, Pending
Kubelet	OOMKilled, CrashLoopBackOff, Probe Failures, FailedMount
Container Runtime	ErrImagePull, ImagePullBackOff, CreateContainerError
API Server / Configuration	CreateContainerConfigError, Secret Missing, ConfigMap Missing

Component	Failures
Storage (CSI)	PVC Pending, FailedMount
Networking	DNS Failure, NetworkPolicy, Service Discovery
Node	NodeNotReady, Evicted

This mental model is far more valuable than memorizing error names because when a new failure appears, your first question becomes:

"Which Kubernetes component owns this part of the lifecycle?"

That immediately narrows where to look.

Before we start designing tomorrow

There's one thing I want you to think about overnight because it will fundamentally shape the architecture.

You currently think your operator will diagnose **Pods**.

I want you to challenge that assumption.

Ask yourself:

What is the primary resource my controller should watch?

Should it watch:

- Pods?
- Events?
- ReplicaSets?
- Deployments?
- Nodes?
- Or a custom resource like **PodAnalysis**?

That single design decision changes the entire controller architecture, reconciliation logic, scalability, and extensibility. Tomorrow we'll start by answering that question before writing any Go or designing the CRD, because that's exactly the kind of architectural decision reviewers on CNCF projects care about.

tomorrow is here, now tell me what should be done

Real talk:

Now we stop thinking like someone building a GitHub project and start thinking like someone designing a real Kubernetes controller.

This phase is **more important than coding**.

A lot of people do this:

```
kubebuilder init
↓
Write Reconcile()
↓
Add if statements
↓
Done
```

That's backwards.

We first need to answer:

What problem are we solving?

Who is the user?

What is the architecture?

How does Kubernetes naturally solve this kind of problem?

Step 1 — Define the product

Not the code.

The product.

User

Who uses this?

Platform Engineers

SREs

DevOps Engineers

On-call Engineers

Their problem

They receive

```
Alert
↓
Pod restarting
↓
Need root cause
↓
```

Spend 20–40 minutes collecting evidence

↓

Finally find issue

Our goal

Alert

↓

Diagnosis already available

↓

Engineer verifies

↓

Fix

Notice

We are **not replacing engineers**.

We are reducing investigation time.

Step 2 — Define the scope

This is where many projects fail.

Your operator should NOT

✗ Restart Pods

✗ Auto-scale

✗ Fix ConfigMaps

✗ Restart Deployments

✗ Delete resources

✗ Auto-heal everything

Those become separate operators.

Your operator has **one responsibility**.

Observe → Analyze → Report

Nothing more.

This makes it easier to reason about.

Step 3 — What should the operator watch?

Yesterday I left you with this question.

Let's evaluate every option.

Option 1 — Watch Deployments

Deployment

↓

Reconcile

Bad.

Why?

Deployments don't restart.

Pods do.

You lose all runtime information.

Reject.

Option 2 — Watch ReplicaSets

Same issue.

Reject.

Option 3 — Watch Nodes

Too coarse.

One node contains many pods.

Reject.

Option 4 — Watch Events

Interesting.

Events tell us

OOMKilled

FailedMount

ImagePull

Probe Failed

But

Events disappear.

Events are noisy.

Events are not state.

Controllers generally reconcile desired state, not transient logs.

Not ideal as the primary watched resource.

Option 5 — Watch Pods

Much better.

Pods contain

- Status
- Restart Count
- Conditions
- Last State
- Container State
- OwnerReferences
- Node Name

Most investigation starts from Pod.

This is a strong candidate.

Option 6 — Watch Custom Resource

Example

```
kind: PodAnalysis
```

Interesting.

But who creates it?

If engineers have to manually create

```
kubectl apply
```

after every incident,

the tool becomes useless.

Best answer

The operator should primarily **watch Pods**.

Why?

Because Pods are where failures actually occur.

Everything else can be queried when needed.

Step 4 — Event-driven or polling?

Option A

Every 30 seconds

List Pods

↓

Check Restarts

No.

Wasteful.

Option B

Watch API

Pod Updated

↓

Reconcile immediately

Exactly how Kubernetes controllers work.

This is the Kubernetes way.

Step 5 — What triggers analysis?

Not every Pod update.

Example

Pod label changed.

Ignore.

Annotation changed.

Ignore.

Status changed

Interesting.

Restart count increased

Very interesting.

Container waiting reason changed

Analyze.

Pod phase changed

Analyze.

Trigger examples

Pending → Running

Running → Failed

RestartCount increased

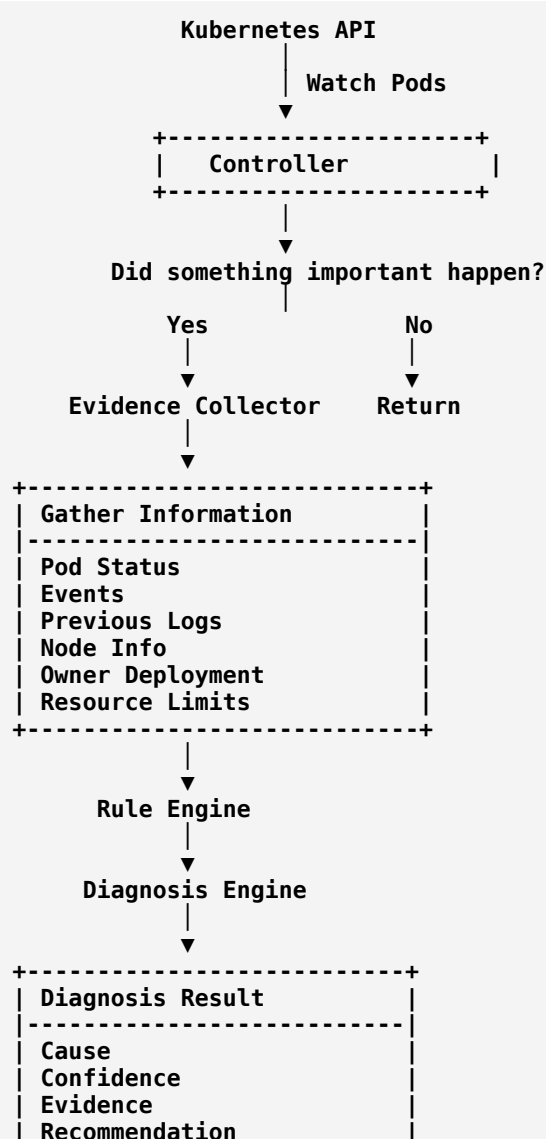
Waiting Reason changed

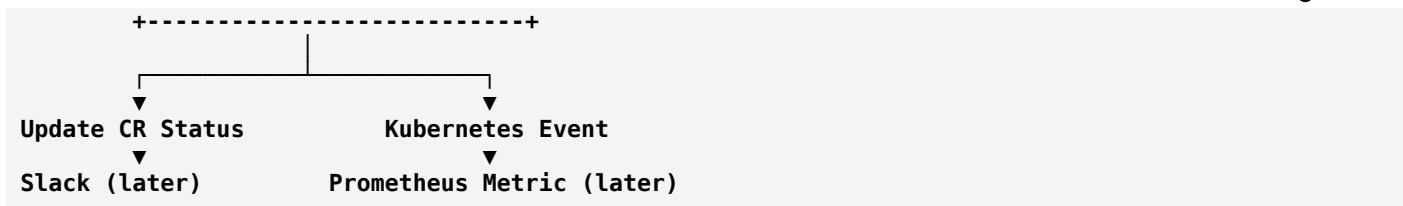
Container terminated

Pod deleted unexpectedly

Step 6 — High-level architecture

This is what I would build.





Notice

Every box has one responsibility.

That's good software engineering.

Step 7 — Should we have a CRD?

This is where many operator tutorials mislead people.

They start by saying

Every operator needs a CRD.

That's not always true.

Let's think.

What is a CRD for?

It lets users declare desired state.

Example

DatabaseBackup

↓

Schedule: Daily

User wants something.

Operator makes it happen.

Our operator

Does the user want to declare anything?

Not really.

The operator just observes Pods.

So why create

PodAnalysis

?

Option A — No CRD

Watch Pods.

Produce Events.

Done.

Very simple.

Option B — CRD

PodAnalysis

stores

Cause

Confidence

Evidence

Recommendations

History

Advantages

- Kubernetes-native
- Queryable
- Extensible
- Status tracking
- Great interview discussion

I would choose **Option B**, but **not** because the operator technically needs it. I'd choose it because it makes the tool more useful and showcases CRD design.

The CRD should represent **analysis results**, not configuration.

Step 8 — Internal packages

Don't put everything in `controllers/`.

I'd structure it like this:

```
api/
  v1alpha1/
    podanalysis_types.go

controllers/
  pod_controller.go

internal/
```

```
collector/
  pod.go
  events.go
  logs.go
  node.go

analyzer/
  engine.go

rules/
  oom.go
  imagepull.go
  probes.go
  scheduling.go

diagnosis/
  types.go

reporter/
  status.go
  events.go
  slack.go (future)

cache/
  restart_history.go
```

This looks like a production codebase instead of a tutorial project.

Step 9 — Think in pipelines, not conditionals

Instead of

```
Go
if reason == "OOMKilled" {
  ...
}
```

Think

```
Evidence
↓
Facts
↓
Rules
↓
Diagnosis
↓
Report
```

Every stage is replaceable.

That's a much cleaner architecture.

Step 10 — MVP (Minimum Viable Product)

Do **not** try to support all 15 failure modes initially.

Phase 1 should support only:

- OOMKilled
- CrashLoopBackOff
- ErrImagePull
- ImagePullBackOff
- CreateContainerConfigError

That's enough to prove the architecture.

Step 11 — Future roadmap

Once the architecture is stable, adding new rules becomes easy:

- Probe failures
- FailedScheduling
- PVC issues
- DNS failures
- Node failures
- NetworkPolicy issues
- Admission webhook failures

No architectural changes required—just new evidence collectors or rules.

This is what I would build

If this were my project, I'd position it as:

A Kubernetes operator that continuously observes Pod lifecycle events, correlates runtime evidence from multiple Kubernetes resources, and produces explainable diagnoses for common infrastructure failures.

That's a stronger description than:

"An operator that tells you why Pods restart."

The first describes an **architecture**. The second describes a single feature.

Before writing any code

There is one more design topic we should settle first because it will affect every package you create:

How should the Rule Engine work?

There are several viable designs:

1. A simple chain of Go rules (**Rule** interface with **Match()** and **Diagnose()**).
2. A weighted scoring engine where multiple rules contribute confidence.
3. A graph-based diagnosis engine that follows evidence to likely causes.
4. A hybrid approach combining deterministic rules with confidence scoring.

This choice will determine how easy it is to add new diagnoses later and whether your operator feels like a real diagnosis engine rather than a collection of **if** statements. I would finalize that before opening Kubebuilder.

first of all explain me the complete flow of request happening in backend and for user in detailed paragraphs form not in single word points because reading a word does not give clarity, explain like a story in easy language so it is easily digest in depth.

If we use CRD then what would the user create and how will this crd behave and work in real time. We have discussed approx 15 issues that occurs in production, and there would be a lot more issues, so would watching only pods be able to provide us the complete picture of what is happening and will be able to find where the problem is happening and what is the root cause, even though we are not able to provide root cause but we should be able to provide user a detailed report of what is happening in system, so that they might get an idea and get something valuable from our report

Explain each step in simple detailed paragraph.
[Show more](#)[Show less](#)

Real talk:

This is where most operator tutorials completely fail. They immediately start writing Go code without first answering **what actually happens inside Kubernetes from the moment a failure occurs until the engineer sees the diagnosis**.

Let's forget Go, Kubebuilder and controllers for a while.

Let's imagine you're a Platform Engineer working at Google or AWS.

First, let's understand the story from the user's perspective

Imagine you are a backend developer.

You have written a Go application.

```
Go
func main() {
    connectDatabase()
    startServer()
}
```

You build the Docker image.

Push it to your registry.

Create a Deployment.

YAML

kind: Deployment

You run

Bash

kubectl apply -f deployment.yaml

Everything seems fine.

You go for lunch.

After lunch, an alert comes.

Payment API is unavailable.

Now imagine your company has **our operator installed**.

You don't run any commands immediately.

Instead, you open your dashboard (or run **kubectl get podanalysis** later).

You immediately see something like

Application

payment-api

Status

Investigation Completed

Likely Cause

OOMKilled

Confidence

98%

Timeline

12:01 Pod Started

12:04 Memory Usage Increasing

12:05 OOMKilled

12:05 Restarted

12:06 OOMKilled Again

12:07 CrashLoopBackOff

Evidence

Exit Code 137

Reason OOMKilled

Memory Limit 512Mi

Restart Count 17

Recommendation

Increase memory limit or investigate memory leak.

Notice something.

The engineer hasn't fixed anything yet.

The operator hasn't fixed anything either.

But the engineer already knows **where to start**.

That alone saves a huge amount of time.

Now let's see the same story from Kubernetes' perspective

Now forget the human.

Imagine Kubernetes is talking.

You applied a Deployment.

The Deployment Controller creates a ReplicaSet.

ReplicaSet creates a Pod.

Scheduler chooses a Node.

Kubelet on that Node receives instructions.

Container Runtime pulls the image.

The container starts.

Everything is working.

For the next five minutes, Kubernetes is mostly idle. Nothing interesting is happening.

Then suddenly, your application consumes more than 512 MiB of memory.

Linux Kernel kills the process.

The container exits.

Kubelet notices

"The container I was managing just died."

Kubelet immediately updates the Pod Status inside the Kubernetes API Server.

The Pod object changes.

The restart count increases.

The last container state becomes

Reason: OOMKilled
ExitCode: 137

The kubelet also creates Kubernetes Events describing what happened.

The Deployment Controller sees the Pod died and expects one healthy replica.

Since restart policy is Always, the kubelet starts the container again.

It crashes again.

And again.

Eventually Kubernetes says

```
CrashLoopBackOff
```

Notice something very important.

Nobody called our operator.

Nobody sent an HTTP request.

Nobody executed our code.

The Kubernetes API simply changed.

That is the biggest mindset shift.

Controllers are not request-response systems.

They are **state observers**.

So where does our operator come into this story?

Imagine another person sitting inside the cluster.

That person constantly watches Kubernetes.

Not by polling every second.

Instead, Kubernetes itself tells him

```
Hey.
```

```
Pod payment-api changed.
```

```
Do you want to see the new version?
```

Our operator says

```
Yes.
```

Kubernetes sends the updated Pod object.

Now our operator starts thinking.

Not acting.

Thinking.

What is the first question our operator asks?

The operator doesn't immediately diagnose anything.

Instead it asks

"Should I even investigate this update?"

Because Pods change all the time.

Example

Label changed.

Ignore.

Annotation changed.

Ignore.

Image changed during rollout.

Ignore.

Restart Count increased.

Interesting.

Container entered Waiting state.

Interesting.

Container terminated.

Interesting.

Node changed.

Interesting.

Only then does the operator begin an investigation.

This is very similar to how an experienced engineer works.

Now the investigation begins

Think of our operator as a detective.

A detective never walks into a crime scene and immediately says

It was murder.

First, they collect evidence.

Exactly the same thing should happen here.

The operator first creates an internal investigation.

Something like

Investigation Started

Pod

payment-api

Reason

Restart Count Increased

Notice.

At this moment, the operator still has **no diagnosis**.

Only a reason to investigate.

What happens next?

The operator now starts collecting evidence.

Think of this as walking through a crime scene.

The first thing it already has is the Pod object because Kubernetes gave it to us.

Inside that Pod object, it finds things like

- current phase
- restart count
- container state
- waiting reason
- last terminated state
- exit code
- node name
- owner deployment
- resource requests
- resource limits

Already this is a lot of information.

But an experienced engineer never stops here.

The operator now says

"The Pod told me it died.

Now I want to know **what everyone else knows.**"

So it starts asking other Kubernetes resources.

It asks the Event API

Did anyone record something about this Pod?

Events reply

FailedMount

or

OOMKilled

```
or
Readiness Probe Failed
or
Image Pull Failed
```

Good.

More evidence.

Then it asks the Node

```
How are you feeling?
```

Node replies

```
Memory Pressure
Disk Pressure
Ready
NotReady
```

Interesting.

Maybe the Pod wasn't the real problem.

Maybe the Node was.

Then it asks the PersistentVolumeClaim

```
Did you mount correctly?
```

PVC replies

```
No.
StorageClass missing.
```

Interesting.

Maybe this isn't an application problem at all.

Then it asks

```
Does the Secret exist?
```

Secret API replies

```
No.
```

Interesting.

Then it asks

```
Give me previous logs.
```

Previous logs say

```
panic
database timeout
```

Now the picture is getting clearer.

Notice something beautiful happening.

The operator isn't looking at one object anymore.

It is **building a timeline** from multiple sources.

This is where I want to change one thing we discussed yesterday

Yesterday we said

Watch Pods.

Today I want to refine that.

The **Pod should be the trigger, not the only source of truth.**

Those are completely different ideas.

Think like this:

```
Pod changes
↓
Start investigation
↓
Collect information from
Pod
Events
Node
PVC
Deployment
ReplicaSet
ConfigMaps
Secrets
Logs
Metrics (future)
↓
```

Build timeline

↓

Diagnose

This is much more powerful.

Can watching Pods still detect every problem?

This is the exact question a senior interviewer would ask.

The answer is

No.

Watching Pods only tells us

"Something happened."

It does not tell us everything.

For example

Suppose the Node dies.

The Pod didn't cause that.

The Node did.

Suppose StorageClass is deleted.

PVC knows first.

Suppose CoreDNS crashes.

Pods are only victims.

Suppose NetworkPolicy blocks traffic.

Again

Pods only suffer.

They don't know why.

So our operator must think

Pod changed

↓

Who else might know something?

↓

Ask them.

That's much closer to how humans investigate.

Now let's discuss the CRD

This is the part that I think needs the biggest redesign.

Originally I suggested

```
kind: PodAnalysis
```

Today I think we can design something much better.

Imagine you are the engineer.

Do you really care about a Pod?

Not really.

You care about

```
The incident.
```

The Pod is only one piece of that incident.

So instead of thinking

```
One Pod
```

```
↓
```

```
One Analysis
```

Think

```
One Incident
```

```
↓
```

```
One Investigation Report
```

That changes everything.

Imagine our CRD becomes

```
YAML
```

```
apiVersion: diagnosis.karan.dev/v1alpha1
kind: IncidentReport
```

Now the user doesn't create it.

The operator creates it.

Exactly like ReplicaSets are created automatically by Deployments.

Example

Pod crashes.

Operator creates

```
YAML
```

```
kind: IncidentReport
metadata:
  name: payment-api-20260808-145500
```

Inside

```
YAML
spec:
  podName: payment-api
  namespace: production
```

Spec rarely changes.

Status continuously evolves.

Initially

```
YAML
status:
  phase: CollectingEvidence
```

A few seconds later

```
YAML
status:
  phase: Analyzing
```

Then

```
YAML
status:
  phase: Completed
  confidence: 96%
  summary:
    Likely OOMKilled
  timeline:
    12:01 Started
    12:03 Memory Growing
    12:04 OOMKilled
    12:05 Restart
  recommendation:
    Increase memory limit
```

Notice something.

The CRD itself becomes the investigation report.

Not the configuration.

This feels much more Kubernetes-native.

Why is this better?

Because engineers can now run

```
Bash
kubectl get incidentreports
```

or

```
Bash
kubectl describe incidentreport payment-api-145500
```

and read the complete investigation.

Exactly like they read a Pod.

The report becomes another Kubernetes resource.

That's elegant.

The biggest mindset shift

I think this is the most important thing we've discovered today.

Originally we thought

```
Pod
↓
Rule Engine
↓
Diagnosis
```

I no longer think that's enough.

I think the real architecture should look like this:

```
Pod changes
↓
Create Investigation
↓
Collect evidence from the entire cluster
↓
Correlate everything into one timeline
↓
```

Generate one Incident Report

↓

Store it as a Kubernetes resource

↓

Engineer reads it

See the difference?

We're no longer trying to recognize 15 errors.

We're trying to answer one question:

"What happened?"

That's a much more powerful problem statement.

And because you're answering **what happened**, not just **which error occurred**, your project becomes much closer to how commercial observability platforms like Datadog, Dynatrace, or New Relic think internally. They don't just classify errors—they correlate evidence from many sources into a coherent story.

One thing I would actually change before we write a single line of code

If I were reviewing this project at a CNCF meetup, I would suggest one rename.

Don't call it **Pod Failure Analyzer** anymore.

Call it something like:

Kubernetes Incident Investigator

or

Kubernetes Incident Correlator

because what we're designing is no longer a Pod analyzer. It's an **incident investigation engine** where the Pod is merely the event that starts the investigation. That framing is technically more accurate, scales naturally to future features (Node incidents, PVC incidents, DNS incidents), and better reflects the architecture we've evolved over these discussions.

how kubernetes is giving information to our operator, how kubernetes itself tells our operator do you want to see the new version?

i asked to provide simple language so that you describe each and every thing but it does not mean you give incorrect information.

I want to know how when an incident happen then what happens exactly, what part of our investigator does what in a flow, so that i can map everything.

Regarding CRD, you mean when any update happen in pod then automatically an incidentReport resource is made by kubernetes incident investigator and complete analysis is present in this report, right? if it is the way then when user would right kubectI get incidentReports then what he would get as in output and like i want the complete picture

Show moreShow less

Real talk:

These are exactly the questions you should ask before writing code. And you're right to call out my earlier wording—I simplified too much. Kubernetes does **not** "tell" the operator in a conversational sense. There is a real mechanism behind it, and understanding it is fundamental to Kubernetes.

This answer will be long because this is probably the **most important concept** for your entire project.

Part 1 - How does Kubernetes actually notify our operator?

Let's forget operators for a moment.

Imagine you have two applications.

Application A

and

Application B

Suppose Application A updates a database.

How does Application B know?

Three possibilities exist.

Method 1 - Polling

Application B asks every second.

Has anything changed?

No.

Has anything changed?

No.

Has anything changed?

No.

Eventually

Has anything changed?

Yes.

This is called polling.

It wastes resources because most of the time nothing changes.

Method 2 - Webhooks

Application A sends HTTP request.

```
POST /event
Something changed.
```

Application B immediately knows.

Method 3 - Watch Stream

This is what Kubernetes uses.

Neither polling.

Nor webhooks.

Instead imagine a phone call.

You call your friend.

```
Don't disconnect.
Just tell me whenever something changes.
```

Your friend says

```
Okay.
```

Now one hour passes.

Nobody speaks.

Connection stays open.

Then

```
Friend:
Something changed.
Here is the new data.
```

Connection stays open.

Ten minutes later

```
Friend:
Something else changed.
```

Still same connection.

This is called a **Watch**.

Who is the friend?

The Kubernetes API Server.

Everything in Kubernetes goes through the API Server.

Not kubelet.

Not scheduler.

Not controller-manager.

Everything updates the API Server.

The API Server is the source of truth.

Let's see the real story.

Suppose

```
Deployment
```

```
↓
```

```
ReplicaSet
```

```
↓
```

```
Pod
```

Pod starts.

Everything fine.

Now application crashes.

Linux kills process.

Kubelet notices.

Question

Where does kubelet save this information?

Answer

Into the Kubernetes API Server.

The Pod object changes.

Before

```
YAML
```

```
status:
  restartCount: 2
```

After

```
YAML
status:
  restartCount: 3
```

The API Server now has a newer version of the Pod object.

Notice

The object itself changed.

So where is our operator?

Imagine our operator started one hour ago.

During startup it says

```
Dear API Server,
I want to watch Pods.
```

API Server replies

```
Okay.
I'll keep this connection open.
Whenever any Pod changes,
I'll send you the updated object.
```

That's literally what happens.

The operator isn't asking every second.

The API Server is streaming updates over one long-lived connection.

This is why Kubernetes scales so well.

What does the operator actually receive?

This is important.

The operator does **not** receive

```
OOMKilled
```

or

```
Restart happened
```

Instead it receives

the **entire updated Pod object**.

Imagine

Old Pod

```
YAML
restartCount: 2
```

New Pod

```
YAML
restartCount: 3

lastState:
  reason: OOMKilled
```

The operator receives the **new version**.

It then compares

Old

vs

New

and thinks

```
Restart count increased.
Interesting.
Let's investigate.
```

So what is Reconcile()?

Many tutorials say

Reconcile() is called.

But why?

Let's understand.

Imagine

API Server sends

```
Pod Updated
```

controller-runtime receives this.

It puts a request into a work queue.

```
Queue
payment-api
```

Worker thread

takes one item.

Calls

```
Reconcile(payment-api)
```

Notice

Reconcile()

isn't magical.

It's simply a worker processing queued events.

Let's follow one incident from beginning to end.

Suppose

Your application exceeds memory.

We'll go extremely slowly.

Step 1

Application allocates memory.

```
100Mi
```

```
↓
```

```
300Mi
```

```
↓
```

```
450Mi
```

```
↓
```

```
512Mi
```

```
↓
```

```
530Mi
```

Step 2

Linux Kernel says

```
Memory limit exceeded.
```

```
Kill process.
```

Container dies.

Step 3

Kubelet notices.

```
Container exited.
```

Kubelet updates Pod Status.

```
restartCount = 5
↓
restartCount = 6
Reason = OOMKilled
```

Step 4

API Server stores new Pod object.

Now

API Server knows

Pod changed.

Step 5

Our operator already has an open Watch connection.

API Server immediately sends

```
Updated Pod Object
```

to the operator.

Not because it is "calling" the operator.

Because operator subscribed to changes earlier.

Step 6

controller-runtime receives update.

Puts request into queue.

```
Queue
payment-api
```

Step 7

Worker thread picks queue item.

Runs

```
Reconcile()
```

Now our code finally starts executing.

Everything before this was Kubernetes.

Everything after this is our software.

What happens inside our Reconcile()?

This is where your architecture begins.

Imagine

```
Reconcile()
↓
Should I investigate?
↓
Yes
↓
Create Investigation
↓
Collect Evidence
↓
Analyze
↓
Create Report
↓
Done
```

Let's expand each one.

Phase 1 - Trigger Check

Operator asks

```
Why was I called?
```

Pod label changed?

Ignore.

Annotation changed?

Ignore.

Restart count changed?

Investigate.

Container state changed?

Investigate.

Waiting reason changed?

Investigate.

This phase filters noise.

Phase 2 - Investigation Object

Instead of immediately analyzing,

I would create an in-memory object.

Something like

```
Investigation
```

```
Pod
```

```
payment-api
```

```
StartedAt
```

```
19:22
```

```
Evidence
```

```
(empty)
```

Think of this like opening a police case.

Nothing analyzed yet.

Phase 3 - Evidence Collection

Now every collector works independently.

Imagine

```
Pod Collector
```

asks

```
Give me Pod Status.
```

Stores

Restart Count
Container State
Exit Code
Reason

Event Collector

asks

Give me Events.

Stores

FailedMount
Probe Failed
OOMKilled

Node Collector

asks

How is the Node?

Stores

Ready
Memory Pressure
Disk Pressure

PVC Collector

asks

PVC healthy?

Stores

Status.

Secret Collector

asks

Secret exists?

Stores result.

Notice

Nobody diagnoses anything.

Collectors only collect.

Phase 4 - Analysis

Now analyzer receives
one Investigation object.

Example

```
Restart Count = 8
Reason = OOMKilled
Exit Code = 137
Memory Limit = 512Mi
Events = OOMKilled
```

Now

Rule Engine says

```
Rule 1
Matches.
```

Diagnosis

```
Likely Cause
OOMKilled
Confidence
99%
```

Phase 5 - Report Generation

Now we have

Evidence

Diagnosis

Timeline

Recommendations

Everything combined.

Operator now creates

```
IncidentReport
```

inside Kubernetes.

Let's talk about the CRD now.

This is where things become really interesting.

You asked

Does the operator automatically create IncidentReport?

Yes.

Not Kubernetes.

Our operator creates it.

Very important difference.

Kubernetes knows nothing about IncidentReport.

Our operator is responsible.

The flow is

```
Pod changed
↓
Operator investigated
↓
Operator creates IncidentReport
```

Exactly like

Deployment Controller creates ReplicaSet.

ReplicaSet Controller creates Pod.

Controllers create resources.

What would IncidentReport look like?

Imagine

```
kubectl get incidentreports
```

Output

NAME	POD	STATUS	CAUSE	AGE
payment-api-145530	payment-api	Completed	OOMKilled	2m
redis-150210	redis	Completed	FailedMount	5m
frontend-153122	frontend	Investigating	ImagePullBackOff	20s

Already useful.

Now

```
kubectl describe incidentreport payment-api-145530
```

Imagine output.

Name
payment-api-145530
Namespace
production

Incident
Pod restarted repeatedly.

Status
Completed

Timeline
14:52
Pod Started
14:54
Memory Usage Increasing
14:55
OOMKilled
14:55
Restarted
14:56
OOMKilled Again
14:57
CrashLoopBackOff

Evidence
Restart Count
17
Exit Code
137
Container Reason
OOMKilled
Node
worker-2
Memory Limit
512Mi

```

-----
Diagnosis
Likely Cause
Out Of Memory
Confidence
98%
-----
Recommendation
Increase memory limit
Investigate memory leak
-----
Collected By
Kubernetes Incident Investigator

```

Notice something.

The engineer never had to execute

```

kubectl describe pod
kubectl logs
kubectl get events
kubectl describe node

```

The report has already gathered everything.

But I want to improve this design even further

After thinking about it more, I actually wouldn't create an **IncidentReport** for **every** Pod update.

Imagine a healthy Deployment during a rolling update. Pods are constantly changing. Creating reports for every status change would flood the cluster.

Instead, I'd treat the report as something created only when an **investigation-worthy trigger** occurs:

- Restart count increased.
- Pod entered a failure state (**CrashLoopBackOff**, **ImagePullBackOff**, etc.).
- Scheduling repeatedly failed.
- Mount failures persist.
- Probe failures cross a threshold.

Normal lifecycle events (like a successful rollout) would simply be ignored.

That makes **IncidentReport** a meaningful operational resource rather than a noisy audit log.

The architecture I now believe is the right one

After all these discussions, I think the complete flow should look like this:



This is the first point in our design where I think we have a solid, production-style architecture. From here, we can start talking about the actual Go packages, CRD schema, controller behavior, and implementation with much more confidence because we now understand **who does what, when, and why**.

i understand your explanation, it is very good. I have some questions to ask like what is reconcile in our investigator like is it something that we would be creating or it is already there in kubernetes which calls our investigator. When i searched on internet i got this: In Kubernetes, reconcile means to force the actual state of your cluster to match your desired state. Kubernetes works on a continuous automation loop. You declare how you want things to look (the Desired State), and Kubernetes looks at how things actually are (the Actual State). If there is a difference, it fixes it. This

correction process is called reconciliation.

How does it relate to our setup.

Show moreShow less

Real talk:

Excellent question.

This is probably **the single most important Kubernetes concept** to understand. Once you truly understand reconciliation, 70% of Kubernetes controllers will suddenly make sense.

The definition you found is **correct**, but it is also **incomplete**. Most articles stop there, which is why people get confused when they start writing operators.

Let's build this concept from the ground up.

First understand why reconciliation exists

Imagine Kubernetes didn't have reconciliation.

You create

```
YAML
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
```

You apply it.

Kubernetes creates

```
Pod-1
Pod-2
Pod-3
```

Everything looks good.

Then someone accidentally deletes

```
Pod-2
```

Question:

Who will create the new Pod?

Nobody.

Because Kubernetes would have simply executed your request once and forgotten about it.

That would make Kubernetes just another deployment tool.

Instead, Kubernetes does something much smarter.

It **never forgets** what you wanted.

It continuously remembers

"User wanted 3 Pods."

Every time something changes, it asks

"Do I still have 3 Pods?"

If yes

Nothing to do.

If no

Create another Pod.

This continuous checking and correction is called **reconciliation**.

The real meaning of Reconcile()

The word itself means

Bring two things back into agreement.

Example

Desired

```
Replicas = 3
```

Actual

```
Replicas = 2
```

Not equal.

Need reconciliation.

After reconciliation

Actual

```
Replicas = 3
```

Now they match.

Done.

Who performs reconciliation?

This is where people get confused.

Kubernetes itself doesn't have one giant reconciliation loop.

Instead,

every controller has its own reconciliation loop.

For example

Deployment Controller

reconciles Deployments.

ReplicaSet Controller

reconciles ReplicaSets.

Job Controller

reconciles Jobs.

StatefulSet Controller

reconciles StatefulSets.

Each controller is responsible for only one resource.

Now let's look at Deployment Controller

Imagine

Deployment

```
YAML
replicas: 3
```

Current state

```
2 Pods
```

Deployment Controller wakes up.

It runs its Reconcile function.

Internally, it thinks something like this:

```
Desired Replicas = 3
Actual Replicas = 2
Difference = 1
Action = Create one Pod
```

That's reconciliation.

Notice

The controller isn't "running Pods."

It is comparing desired state with actual state and deciding what action is needed.

So what is Reconcile() in code?

When you create an operator with Kubebuilder, you'll see something like:

```
Go
func (r *PodAnalysisReconciler) Reconcile(
    ctx context.Context,
    req ctrl.Request,
) (ctrl.Result, error) {
}
```

Many beginners think

"Kubernetes calls this magical function."

Not exactly.

Let's see who actually calls it.

The actual flow

Your operator starts.

Inside it, **controller-runtime** starts running.

controller-runtime says to the API Server:

"I'm interested in Pods."

The API Server opens a watch stream.

Later, a Pod changes.

The API Server sends the updated Pod event.

controller-runtime receives it.

Then **controller-runtime** puts a request into a work queue.

Worker thread takes the request.

Finally,

controller-runtime calls your Reconcile() function.

So the call chain is:

API Server

↓

Watch Event

↓

controller-runtime

↓

Work Queue

```
↓
Worker
↓
Your Reconcile()
```

Notice

Kubernetes itself never directly calls your function.

`controller-runtime` does.

`controller-runtime` is just a Go library that makes writing controllers much easier.

Now comes the most important question.

What is the desired state in our project?

For Deployment Controller

Desired State

```
3 Pods
```

Easy.

But our investigator?

Let's think.

First idea

Desired state

```
Every Pod has an IncidentReport.
```

Sounds reasonable.

But...

Suppose Pod is perfectly healthy.

Do we want an IncidentReport?

No.

That would create millions of useless reports.

So that's not our desired state.

Second idea

Desired state

Every interesting incident has one completed investigation.

Now we're getting somewhere.

Imagine

Pod crashes.

Current state

Pod failed.

No investigation exists.

Desired state

Pod failed.

Investigation exists.

Evidence collected.

Diagnosis completed.

That's reconciliation!

Let's visualize it.

Current state

Pod Restarted

IncidentReport = none

Desired state

Pod Restarted

↓

Investigation Completed

↓

IncidentReport exists

Difference?

Investigation missing.

Our controller performs it.

This is where our operator differs from Deployment Controller

Deployment Controller changes the cluster.

Our operator mostly changes **knowledge**.

Deployment Controller

Desired
3 Pods
Actual
2 Pods
↓
Create Pod

Our Investigator

Desired
Incident investigated
Actual
Incident not investigated
↓
Collect Evidence
↓
Analyze
↓
Create IncidentReport

Notice

We aren't creating Pods.

We're creating information.

But wait...

There is an even deeper insight.

Yesterday we said

Our controller watches Pods.

Today I think that's only half the story.

Let's imagine this.

Pod restarts.

API Server sends update.

Reconcile starts.

Question:

Should Reconcile immediately analyze?

Maybe not.

What if another restart happens one second later?

Or another Event arrives?

Or Node becomes NotReady?

If we analyze immediately, we might produce an incomplete report.

I think our architecture should evolve slightly.

Instead of

Reconcile

↓

Analyze

Think

Reconcile

↓

Open investigation

↓

Collect evidence for a short window

↓

Then analyze

Why?

Because incidents are not instantaneous.

Example

12:00:01
Pod Restart

12:00:02
OOMKilled Event

12:00:03
Node Memory Pressure

12:00:04
CrashLoopBackOff

If you analyzed at

12:00:01

you'd miss the rest of the story.

A small but important correction to the project

This leads to one architectural challenge.

The traditional Kubernetes reconciliation model assumes:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

Observe

↓

Compare

↓

Fix

But our project is actually:

Observe

↓

Collect

↓

Correlate

↓

Analyze

↓

Report

That's not a perfect fit for reconciliation.

So our **Reconcile()** function should be **very small**.

Its job should be something like:

A Pod changed.

↓

Is this interesting?

↓

Yes

↓

Ensure an investigation exists.

↓

Update investigation with latest evidence.

↓

If enough evidence has been collected,
generate or update the IncidentReport.

Notice how **Reconcile()** is **not** the diagnosis engine.

It's the **orchestrator**.

That's a much cleaner design.

This is the mental model I want you to keep

Forget the word **Reconcile** for a moment.

Think of it like this:

Every Kubernetes controller has a manager.

Whenever something changes, the manager wakes up and asks:

"Given the current state of the cluster, what should exist that doesn't exist yet?"

For a Deployment:

"There should be three Pods."

For our investigator:

"There should be a completed investigation for this failure."

Everything else—watches, work queues, **Reconcile()**, **controller-runtime**—exists to help that manager answer that one question repeatedly and reliably. Once that mental model clicks, operators stop feeling magical and start feeling like well-structured event-driven programs.

will you elaborate this whole new thing, you said Ensure an investigation exists but how because in previous steps we were doing investigation here itself and now you investigation ready by this time, so how is it possible, if yes then when it will happen and how would the complete flow be happening in real situation starting from start
Show moreShow less

Real talk:

Yes — I changed the architecture in the previous answer, and I didn't explain the transition clearly enough. Your confusion is justified.

The important correction is this:

Reconcile() does not need to "finish the investigation" in one call.

It can start an investigation, update it as more evidence arrives, and eventually mark it complete.

And "ensure an investigation exists" simply means:

When an interesting incident is detected, make sure there is a persistent Kubernetes object representing that investigation. If one already exists, use/update it instead of creating another one.

Let's walk through the **entire real flow from zero**, with one concrete incident.

1. Start with a normal Kubernetes application

Suppose your company has:

```
Namespace: production
Deployment: payment-api
Replicas: 3
```

Kubernetes currently has:

```
payment-api-abc    Running
payment-api-def    Running
payment-api-ghi    Running
```

Your Incident Investigator is also running in the cluster.

Its job is:

Watch Kubernetes resources and investigate meaningful failures.

At this point, nothing is happening.

2. What has our Investigator already done?

When our Investigator started, its controller registered a **watch** for Pods through **controller-runtime**.

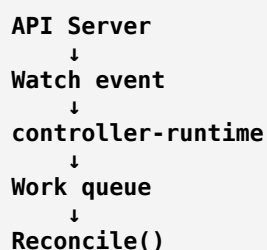
Conceptually:



The API Server maintains that watch connection.

Again, the API Server isn't calling our Go function directly.

The flow is:



Our application is basically saying:

"Keep me informed when Pods I'm watching change."

3. Now something goes wrong

Suppose `payment-api-abc` starts consuming too much memory.

Eventually:

```
Memory usage
  ↓
512 MiB
  ↓
Limit exceeded
  ↓
Linux OOM Killer kills process
```

The application process dies.

Kubernetes hasn't "diagnosed" the incident yet.

The Kubelet notices that its container died.

It updates the Pod's status.

For example:

```
restartCount: 4 → 5
lastState:
  terminated:
    reason: OOMKilled
    exitCode: 137
```

It may also generate relevant Kubernetes Events.

4. API Server receives the updated Pod

The Kubelet sends the status update to the Kubernetes API Server.

The API Server stores the new Pod state.

Now the API Server has:

```
payment-api-abc
restartCount: 5
reason: OOMKilled
exitCode: 137
```

This is important:

The API Server is not running our diagnosis.

It is simply storing and serving Kubernetes state.

5. Our watch receives the change

Remember our Investigator's watch?

The API Server sees:

"A Pod that this controller is watching changed."

So it sends an event through the watch stream.

Conceptually:



The event contains information identifying the changed Pod.

`controller-runtime` then puts a reconciliation request into its work queue.

Something conceptually like:

```

Queue:
production/payment-api-abc
  
```

6. Now Reconcile() runs

A worker takes that queue item.

It calls our code:

```

Go
Reconcile(ctx, req)
  
```

This is the first point where our own investigation code starts executing.

Everything before this was Kubernetes/controller-runtime infrastructure.

Now our code gets involved.

7. What does Reconcile do first?

It retrieves the current Pod from the Kubernetes API.

It asks:

"What does this Pod look like right now?"

It gets something like:

```

Pod: payment-api-abc
Phase: Running
Container:
  State: Running
Last State:
  Terminated
Reason:
  OOMKilled
  
```

```
Exit Code:
137

Restart Count:
5

Node:
worker-2
```

Now our Investigator knows:

"This Pod experienced something worth investigating."

8. But does it immediately investigate everything?

This is where the architecture needs to be precise.

There are actually **two reasonable approaches**.

Approach A — Investigate immediately

`Reconcile()` can immediately collect:

```
Pod
Events
Logs
Node
PVC
ConfigMap
Secret
```

and generate a report.

This is simpler.

Approach B — Create an investigation first

`Reconcile()` creates an `IncidentReport` representing:

"I have detected an incident and I'm investigating it."

Then later reconciliation updates that report as more information becomes available.

For the project we're designing, I prefer Approach B.

Why?

Because an incident is not necessarily a single instant.

You may have:

```
14:01 Pod restarted
14:02 OOMKilled
14:02 Restarted
14:03 Node memory pressure
14:04 CrashLoopBackOff
```

We want to correlate those things.

9. So what does "ensure investigation exists" mean?

This phrase sounds complicated, but it isn't.

Suppose the Pod has just had a significant failure.

Our controller asks:

"Do I already have an IncidentReport representing this incident?"

If no:

No IncidentReport exists.

Then the controller creates one.

Something like:

```
YAML
apiVersion: diagnosis.karan.dev/v1alpha1
kind: IncidentReport

metadata:
  name: payment-api-abc-xxxxx

spec:
  targetPod:
    name: payment-api-abc
    namespace: production

status:
  phase: Investigating
```

That is what **ensure** means.

It doesn't mean:

"Finish the investigation."

It means:

"Make sure the Kubernetes object representing this investigation exists."

10. Who creates this IncidentReport?

Our Investigator.

Not Kubernetes.

This distinction is extremely important.

Kubernetes doesn't know that **IncidentReport** has any meaning unless we install its CRD.

After we install our CRD:

```
Kubernetes API
now understands:
kind: IncidentReport
```

But Kubernetes still doesn't know **when** to create one.

Our controller decides that.

So:

```
Kubernetes
  ↓
provides API + watches
Our Controller
  ↓
decides when IncidentReport should exist
```

11. What happens after the IncidentReport is created?

Now something interesting happens.

Our Investigator has created a Kubernetes resource.

The API Server stores it.

For example:

```
YAML
status:
  phase: Investigating
```

The controller can then continue its work.

It might collect the first evidence:

```
Restart count = 5
Exit code = 137
Reason = OOMKilled
Memory limit = 512Mi
Node = worker-2
```

It updates the IncidentReport's status.

Now:

```
YAML
status:
  phase: CollectingEvidence

evidence:
  restartCount: 5
  exitCode: 137
  reason: OOMKilled
  memoryLimit: 512Mi
  node: worker-2
```

The report is now becoming a persistent record of the investigation.

12. Why not just keep the investigation in memory?

This is an important engineering question.

Suppose we did:

```
Go
investigation := Investigation{}
```

and kept everything in RAM.

Then our Investigator Pod crashes.

Or gets upgraded.

Or the node dies.

What happens?

All investigation state disappears.

That's bad.

By storing the investigation as a Kubernetes resource:

```
IncidentReport
```

the investigation itself becomes persistent cluster state.

Our Investigator can restart and recover.

That's one reason CRDs are useful here.

13. Now let's collect more evidence

The controller may ask Kubernetes for Events associated with the Pod.

It discovers:

```
OOMKilled
Container restarted
Back-off restarting failed container
```

It adds that to the report.

Then it looks at the Node.

Node says:

```
Ready: True
MemoryPressure: False
```

Interesting.

So maybe this was **container-specific memory exhaustion**, rather than the entire node running out of memory.

The controller records that.

14. What about logs?

The Investigator can retrieve the previous container logs.

For example:

```
Processing 10,000 transactions...
Loading customer records...
Killed
```

Now the report contains:

```
Logs:
Application was processing large dataset immediately
before termination.
```

The operator doesn't necessarily know that this is a memory leak.

But it has useful evidence.

15. Now the analysis engine receives the evidence

At this point our internal architecture might look like:

```
Reconcile
  ↓
Investigation
  ↓
Evidence Collector
  ↓
Evidence
  ↓
Analyzer
```

The analyzer sees:

```
Reason = OOMKilled
ExitCode = 137
Memory limit = 512Mi
Restart count = 5
Node MemoryPressure = False
```

It applies its rules.

The OOM rule matches strongly.

The analyzer produces:

```
Likely Cause:
Container exceeded memory limit.
```

Confidence:
99%

It might also say:

Evidence:

- Container terminated with OOMKilled
- Exit code 137
- Memory limit was 512Mi
- Node itself was not under memory pressure

Recommendation:
Investigate memory consumption and consider increasing the container memory limit.

Notice the wording.

We don't say:

"Your application has a memory leak."

We don't have enough evidence to claim that.

16. Now the report becomes Completed

The controller updates:

```
YAML
status:
  phase: Completed

  diagnosis:
    cause: OOMKilled
    confidence: 0.99

  recommendations:
    - "Investigate memory consumption"
    - "Consider increasing memory limit"

  evidence:
    ...
```

Now the investigation is finished.

17. What happens if another restart happens?

Suppose the application gets restarted again 10 minutes later.

The Pod changes again.

The watch catches it.

Another `Reconcile()` happens.

Now the controller asks:

"Is this the same incident or a new incident?"

This is an important design problem we will have to solve.

For example:

```
Incident #1
payment-api-abc
14:01-14:04
OOMKilled
```

Then at

```
15:30
```

another restart happens.

We probably don't want to create a completely unrelated report if it's clearly part of the same ongoing failure.

This is where **incident correlation** becomes one of the harder parts of the project.

We will design this carefully later.

18. Now imagine a completely different failure

Suppose instead the image doesn't exist.

Pod changes to:

```
Waiting:
Reason: ImagePullBackOff
```

Watch fires.

Reconcile() runs.

Controller determines:

"This is investigation-worthy."

It ensures an IncidentReport exists.

Then evidence collection happens.

Events say:

```
Failed to pull image
manifest unknown
```

Analyzer determines:

```
Likely Cause:
Container image/tag does not exist.

Confidence:
99%
```

Report becomes:

Completed

Same architecture.

Different evidence.

Different rule.

19. Now imagine something the operator doesn't understand

Suppose:

Exit Code: 42

Logs:

Application exited unexpectedly.

Events:

Container terminated.

No known infrastructure failure matches.

Our analyzer should **not invent an answer**.

It should produce:

Diagnosis:

Unknown application failure

Confidence:

Low

Evidence collected:

Exit code: 42

Container logs: available

Pod events: available

Node: healthy

Recommendation:

Manual application-level investigation required.

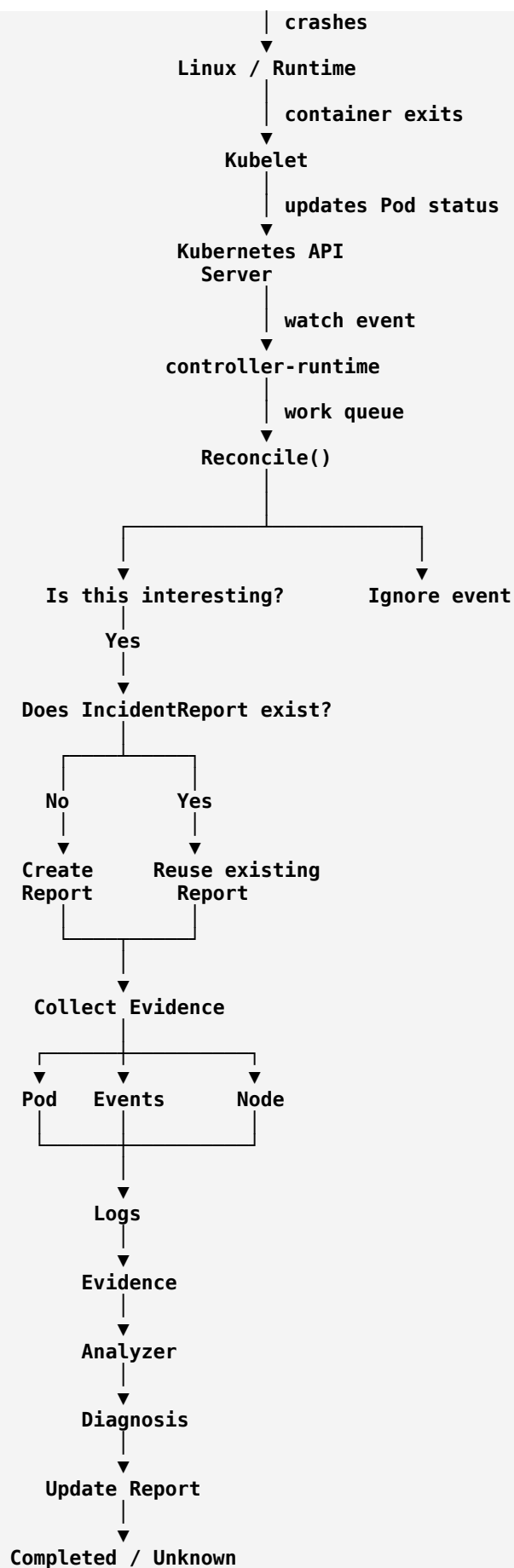
This is still valuable.

Because the engineer gets the evidence in one place.

20. Now you can see the entire real flow

Let's put the whole story together.

APPLICATION



That's the complete mental model.

One subtle but important correction

There is **no guarantee that all of this happens in one `Reconcile()` call.**

That's actually a feature.

Kubernetes controllers are designed to be **eventually consistent**.

For example:

First reconciliation

```
Pod changed
↓
Create IncidentReport
↓
Phase = Investigating
↓
Return
```

Second reconciliation

Maybe the report changed or another relevant event arrived.

```
Collect more evidence
↓
Update report
↓
Return
```

Third reconciliation

```
Enough evidence collected
↓
Analyze
↓
Update report
↓
Phase = Completed
```

The exact implementation will depend on how we design the investigation lifecycle.

And this answers your original question

You asked:

"How can Reconcile ensure an investigation exists if the investigation isn't already ready?"

It doesn't mean the investigation is already complete.

It means:

Reconcile creates and maintains the persistent object representing the investigation.

Think about a hospital.

Patient arrives.

Doctor doesn't immediately have a diagnosis.

They create a **patient case**.

Then:

```

Patient case created
↓
Tests ordered
↓
Results arrive
↓
More tests
↓
Diagnosis
↓
Case completed

```

Our **IncidentReport** is essentially the Kubernetes-native **case file**.

The controller creates and maintains the case.

The evidence collectors gather information.

The analyzer produces the diagnosis.

That is the cleanest mental model for this project.

One thing I would not design yet

Don't start deciding today exactly how many **Reconcile()** calls, queues, timers, or CRD fields we need.

We now understand the **behavioral flow**.

The next architectural question should be:

What exactly constitutes an "incident", how do we create one, and how do we know when an incident is the same incident versus a new incident?

That decision affects the CRD design, deduplication, correlation, investigation window, status lifecycle, and controller logic. It is the next thing I'd design before touching Kubebuilder.

previously we were investigating first and then creating incident report and now we are creating first or reusing previously created incident report and then investigating, why we changed this and

what improvement it is taking in?

if the same pod faces again some other error or same error or something incorrect then the same incident report would be used to update the current incident, right? is this is correct then would be appending the new investigations or replacing current investigation with previous one? if i am wrong then what will be happening clearly state it

Show moreShow less

Real talk:

You caught an important inconsistency in my earlier explanation. **We changed the order because I introduced a persistent IncidentReport as the case file**, but I didn't clearly explain the tradeoff.

More importantly, your assumption that **the same Pod should reuse the same IncidentReport for every future failure is wrong**. We should **not** do that.

Let's establish the correct model.

1. Why did we change from "investigate → create report" to "create report → investigate"?

There are two valid designs.

Design A — Investigate first

```
Pod failure
  ↓
Collect evidence
  ↓
Analyze
  ↓
Create IncidentReport
```

This is perfectly valid.

For a simple MVP, I actually prefer this because it's easier.

The problem is that the investigation exists only in our process memory while we're collecting evidence.

If our controller crashes halfway through:

```
Pod failure
  ↓
Collect Pod ✓
Collect Events ✓
Collect Logs ✓
Controller crashes 💥
```

we could lose the in-progress investigation.

Design B — Create a case first

```
Pod failure
  ↓
Create IncidentReport
  phase = Investigating
```

```

↓
Collect evidence
↓
Update IncidentReport
↓
Analyze
↓
phase = Completed

```

Now the investigation has a persistent Kubernetes object from the beginning.

If the operator crashes:

```

IncidentReport
phase: Investigating

```

still exists.

When the operator starts again, it can find it and continue.

That's the improvement.

We're trading some complexity for **durability and resumability**.

It's the same idea as opening a support ticket before investigating the issue rather than keeping the entire investigation in someone's notebook.

2. But this does NOT mean one Pod = one IncidentReport

This is the important correction.

Suppose:

```

payment-api-abc

```

has an OOM problem at 10:00.

We create:

```

IncidentReport #1

```

```

payment-api-abc-incident-001

```

```

Cause:
OOMKilled

```

```

Status:
Completed

```

At 11:30 the **same Pod** has another problem:

```

Readiness probe failed

```

That should be:

```

IncidentReport #2

```

because these are two different incidents.

3. What if the same Pod keeps restarting because of the same problem?

This is different.

Suppose:

```
10:00 OOMKilled
10:01 OOMKilled
10:02 OOMKilled
10:03 OOMKilled
10:04 CrashLoopBackOff
```

These are probably **not five incidents**.

They're one incident:

```
Incident #1

Started: 10:00
Status: Ongoing

Evidence:
10:00 OOMKilled
10:01 OOMKilled
10:02 OOMKilled
10:03 OOMKilled
10:04 CrashLoopBackOff
```

So we need **incident correlation**.

The controller needs to decide:

"Is this new event part of an existing incident, or should I create a new incident?"

That's a separate problem from diagnosis.

4. So what exactly is an incident?

We need a definition.

I'd define it approximately as:

An incident is a continuous period of abnormal behavior involving a workload, where multiple related failure signals may belong to the same underlying problem.

For example:

```
OOMKilled
↓
Container restart
↓
OOMKilled
↓
CrashLoopBackOff
```

One incident.

Whereas:

```
10:00 OOMKilled
...
Pod healthy for 2 hours
...
12:30 ImagePullBackOff
```

Two incidents.

5. How would the controller know?

We can use a combination of:

Target identity

Which Pod/workload?

```
namespace
pod
container
```

Time

When did the failure happen?

Failure signature

What kind of failure?

```
OOMKilled
ImagePullBackOff
Probe failure
FailedMount
```

Recovery state

Did the workload recover?

For example:

```
Failure
↓
Failure
↓
Failure
↓
Healthy
```

Once it becomes healthy and stays healthy for some period, we can consider the incident closed.

Then a later failure becomes a new incident.

6. Important complication: Pods are replaceable

There is an even deeper Kubernetes issue.

Suppose Deployment has:

```
payment-api
```

Pod:

```
payment-api-abc123
```

gets deleted.

Deployment creates:

```
payment-api-def456
```

Now these are **different Pods**.

If we identify incidents only by Pod name, we lose the relationship between them.

The real application is:

```
Deployment
├── Pod abc123
├── Pod def456
└── Pod ghi789
```

So our investigator should understand the **workload owner**.

For example:

```
Pod
↓
ReplicaSet
↓
Deployment
```

Then an incident can say:

```
Workload:
Deployment/payment-api
Affected Pod:
payment-api-abc123
```

This becomes much more useful.

7. Now your question about "appending vs replacing"

You asked:

Would we append the new investigation or replace the previous one?

Neither, exactly.

We should treat each incident as its **own immutable-ish case**.

Suppose:

Incident 001

```
Payment API
10:00-10:05
OOMKilled
Status: Resolved
```

We don't overwrite it.

Then later:

Incident 002

```
Payment API
13:20-13:25
Readiness probe failure
Status: Resolved
```

That's another report.

So conceptually:

```
Payment API
├── Incident 001
│   └── OOMKilled
├── Incident 002
│   └── Readiness failure
└── Incident 003
    └── ImagePullBackOff
```

This gives us **incident history**.

8. But what gets updated inside an ongoing incident?

Suppose Incident 001 is still happening.

Initial report:

```
Incident 001
Status: Investigating
```

```
Cause: Unknown
Evidence:
Restart count: 1
```

Then another failure occurs.

We update the same IncidentReport:

```
Status: Investigating
Evidence:
Restart count: 2
OOMKilled
Exit code: 137
```

Then another:

```
Restart count: 3
OOMKilled
```

Then analyzer has enough evidence:

```
Status: Completed
Cause:
OOMKilled
Confidence:
99%
```

So **while an incident is ongoing, its report gets updated.**

Once the incident is closed, we don't keep replacing it with unrelated future incidents.

9. Think of it like Git commits

This analogy may help.

Your application has:

```
Incident #1
```

The investigation evolves:

```
Initial state
↓
Evidence added
↓
More evidence
↓
Diagnosis
↓
Resolved
```

The report represents that one investigation.

Then later:

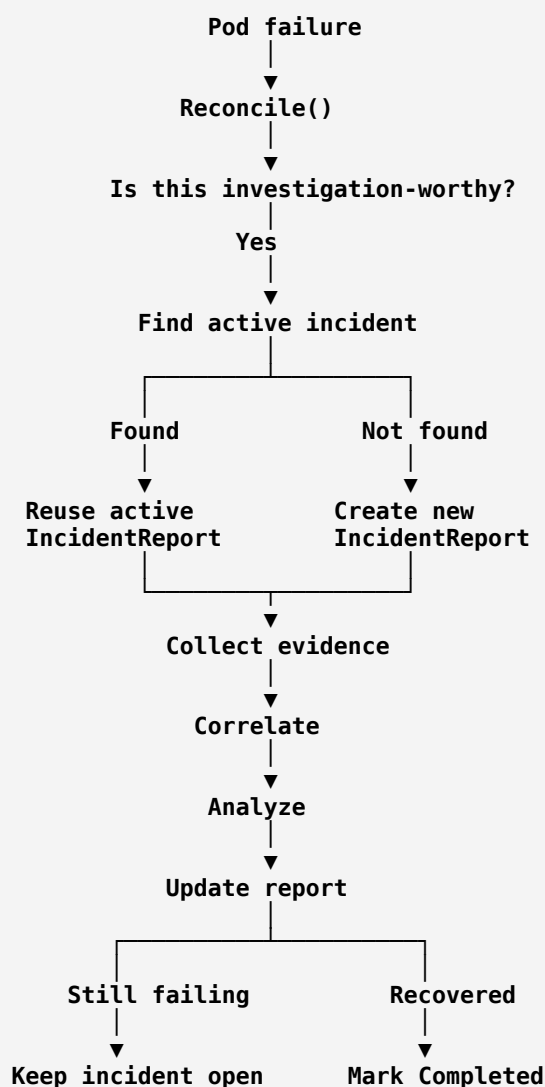
Incident #2

You don't modify Incident #1 to pretend Incident #2 happened inside it.

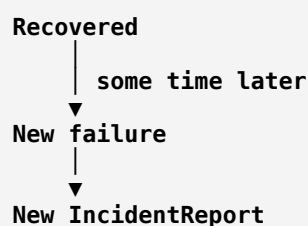
You create another record.

10. The final model

So our flow should actually be:



And later:



11. One important architectural consequence

This means our CRD isn't simply:

"Analysis of a Pod."

It represents:

"A specific incident involving a workload, containing the evidence and analysis accumulated during that incident."

That's a much better abstraction.

For example:

```
YAML
kind: IncidentReport

metadata:
  name: payment-api-incident-001

spec:
  workload:
    kind: Deployment
    name: payment-api

  affectedPod:
    name: payment-api-abc123

status:
  phase: Completed

  startedAt: ...

  resolvedAt: ...

  diagnosis:
    cause: OOMKilled
    confidence: 0.99

  evidence:
    ...

  timeline:
    ...
```

Then you can have:

```
kubectl get incidentreports
```

NAME	WORKLOAD	STATUS	CAUSE	AGE
payment-api-incident-001	payment-api	Completed	OOMKilled	3h
frontend-incident-001	frontend	Completed	ImagePull	2h
payment-api-incident-002	payment-api	Investigating	ProbeFailure	15m

That's actually useful.

One final correction to our architecture

I wouldn't make the **Pod itself** the identity of the incident.

I'd use:

Workload + affected Pod + time + failure signature

because Pods are ephemeral.

The Deployment/StatefulSet/DaemonSet is the thing the engineer ultimately cares about; the Pod is the concrete piece of evidence that experienced the failure.

So tomorrow, before designing the CRD fields, I would first design **incident lifecycle and correlation**:

```
Detected
↓
Investigating
↓
Diagnosed
↓
Resolved
```

and define exactly:

- when an incident starts
- when events belong to the same incident
- when an incident is considered resolved
- when a new incident is created
- how Pod replacement affects correlation

That is the hard part of this project. The Go controller itself is comparatively straightforward once these semantics are nailed down.

i got your explanation, now first list down what are the questions which are remained to be analyzed? what are the different part of this task ?, list down each. Then we will focus on how we are going to resolve the questions and how the hard parts is going to be resolved. Also List down if any gaps we are missing in our architecture?

Before writing single line of code, i want to 100% clarity on each aspect so that once we start implementing then it become easier for us because then we would be having answer for each question.

Show moreShow less

Real talk:

Yes. **Don't write code yet.** We have enough understanding of the problem, but we do **not** yet have a complete architecture.

And there are a few gaps in our current design that we should fix now rather than discover them halfway through implementation.

The right way to proceed is to treat this as a small system-design exercise. We should answer every important question first, then turn those answers into architecture, then implementation.

1. First: What exactly are we building?

Our current definition is:

Kubernetes Incident Investigator is a Kubernetes-native controller that detects meaningful workload failures, collects evidence from multiple Kubernetes sources, correlates that evidence over time, produces an explainable diagnosis when possible, and stores the investigation as a persistent IncidentReport resource.

The important words are:

detect → collect → correlate → analyze → report

Those are essentially the five major parts of the system.

2. The complete list of areas we need to design

I would break the project into **15 major areas**.

We should eventually have a concrete answer for every one.

Area 1 — Problem Definition

We need to precisely define:

What problem are we solving?

Not:

"Kubernetes Pods fail."

That's too broad.

We need something like:

"Engineers spend significant time manually collecting and correlating Kubernetes state, events, logs, and resource information when investigating workload failures."

Then we need to define:

What is our actual value?

Is it:

Reduce investigation time

or

Provide a unified incident timeline

or

Automatically diagnose common failures

or all three?

We need to establish the primary value proposition.

Area 2 — What constitutes an incident?

This is currently one of our biggest unanswered questions.

We need precise rules for:

When does an incident start?

For example:

```
Restart count increased
```

Is that enough?

What about:

```
Pod Pending for 10 seconds
```

What about:

```
One failed readiness probe
```

What about:

```
One failed HTTP request
```

We need thresholds.

When does an incident end?

For example:

```
Pod becomes healthy
```

Is that enough?

Or should it remain healthy for:

```
1 minute?  
5 minutes?  
10 minutes?
```

When is something the same incident?

Example:

```
OOMKilled  
OOMKilled
```

```
OOMKilled
```

Same incident.

But:

```
OOMKilled
...
2 hours healthy
...
ImagePullBackOff
```

Different incident.

We need a correlation strategy.

Area 3 — What resources do we watch?

We currently think:

Pod is the primary trigger.

But we need to formally decide whether we watch:

```
Pods
Events
Nodes
Deployments
ReplicaSets
PVCs
Services
Endpoints / EndpointSlices
Secrets
ConfigMaps
```

And importantly:

Which resources trigger reconciliation?

versus

Which resources are only consulted during investigation?

These should not necessarily be the same.

Area 4 — What information do we collect?

This is our **Evidence Collection layer**.

We need to define exactly what information we collect.

For a Pod:

```
Pod phase
Container state
Previous state
Exit code
```

```
Restart count
Image
Command
Arguments
Resources
Probes
Volumes
Environment references
OwnerReferences
Node
Conditions
```

Events:

```
Event reason
Message
Timestamp
Source
Related object
```

Node:

```
Ready
MemoryPressure
DiskPressure
PIDPressure
Conditions
```

And potentially:

```
PVC
ConfigMap
Secret
Service
EndpointSlice
```

We need to decide the MVP evidence set.

Area 5 — Logs

This deserves its own design because logs are fundamentally different from Kubernetes API objects.

We need to answer:

How does our operator obtain logs?

Kubernetes API can provide container logs through the kubelet API.

But:

- current logs?
- previous logs?
- how many lines?
- how much data?
- what if logs are huge?
- what if container has multiple containers?

- what if logs are unavailable?
- what if the application has already been deleted?

We need limits.

We absolutely cannot say:

"Collect all logs."

That's a production disaster waiting to happen.

Area 6 — Metrics

This is currently a major gap in our architecture.

We talked about Prometheus earlier, but we haven't actually decided whether metrics are part of the MVP.

Metrics can provide extremely valuable evidence:

```
Memory usage
CPU usage
Network traffic
Container restarts
```

For example:

```
OOMKilled
+
Memory steadily increased
+
Memory reached limit
```

is much stronger than simply:

```
OOMKilled
```

So we need to decide:

MVP includes Prometheus?

or

MVP only uses Kubernetes API + logs?

I'd personally keep Prometheus as an **optional evidence source**, not a hard dependency for v1.

Area 7 — Evidence Correlation

This is one of the hardest parts.

We may have:

Pod event
Node event
Container termination
Previous logs
PVC event
Network failure

How do we decide they are related?

Example:

14:00:01 Pod restarted
14:00:02 OOMKilled
14:00:03 Node MemoryPressure
14:00:04 Another Pod evicted

These are probably related.

But:

14:00:01 Pod restarted
15:30:00 Node MemoryPressure

Probably unrelated.

So we need:

- time windows
- resource relationships
- ownership relationships
- causal relationships
- event severity

This is a major design area.

Area 8 — Diagnosis Engine

We need to decide how evidence becomes a diagnosis.

We discussed:

Evidence
↓
Rules
↓
Diagnosis

But we haven't finalized:

What does a Rule look like?

For example:

Go

```
type Rule interface {
    Match(Evidence) bool
    Diagnose(Evidence) Diagnosis
}
```

But maybe we need:

```
Match
Evidence
Confidence
Explanation
Recommendation
```

We also need to handle:

Multiple rules matching.

Example:

```
OOMKilled
+
Node MemoryPressure
+
Pod restart
```

Does one rule win?

Do we combine them?

Do we increase confidence?

Area 9 — Unknown / Unrecognized Failures

This is extremely important.

What happens when none of our rules match?

We must not produce bullshit.

The system should be able to say:

```
Diagnosis: Unknown
Evidence collected: Yes
Automatic diagnosis: Not available
Manual investigation required.
```

But even then, we should provide:

```
timeline
evidence
logs
events
resource state
related objects
```

So we need to design an **unknown diagnosis path**.

Area 10 — IncidentReport CRD

We haven't finalized the CRD.

We need to decide:

What belongs in spec?

Probably relatively immutable information:

```
workload
namespace
initial affected Pod
```

What belongs in status?

Dynamic investigation state:

```
phase
startedAt
updatedAt
resolvedAt
diagnosis
confidence
evidence
timeline
recommendations
```

We need to decide whether storing all evidence directly in the CRD is even appropriate.

This is another important gap.

A huge CRD is a bad design.

Kubernetes objects are stored in etcd.

We don't want:

```
IncidentReport
+
20 MB logs
+
thousands of events
```

inside etcd.

So we need an **evidence storage strategy**.

This is a significant architectural question.

Area 11 — Incident Lifecycle

We need exact states.

For example:

```
Detected
↓
Investigating
```

```

↓
Diagnosed
↓
Resolved

```

But maybe we need:

```

Detected
Investigating
Diagnosed
Unknown
Resolved
Failed

```

And we need to define exactly what causes each transition.

Area 12 — Pod Replacement / Workload Identity

This is another major gap.

Pods are ephemeral.

Suppose:

```

payment-api-abc

```

dies.

Deployment creates:

```

payment-api-def

```

Should the new Pod continue the old incident?

Maybe.

Maybe not.

We need to understand:

```

Pod
↓
ReplicaSet
↓
Deployment

```

and determine how incidents attach to workloads.

Our incident identity probably needs:

```

Workload
+
Affected Pod(s)
+
Time
+
Failure signature

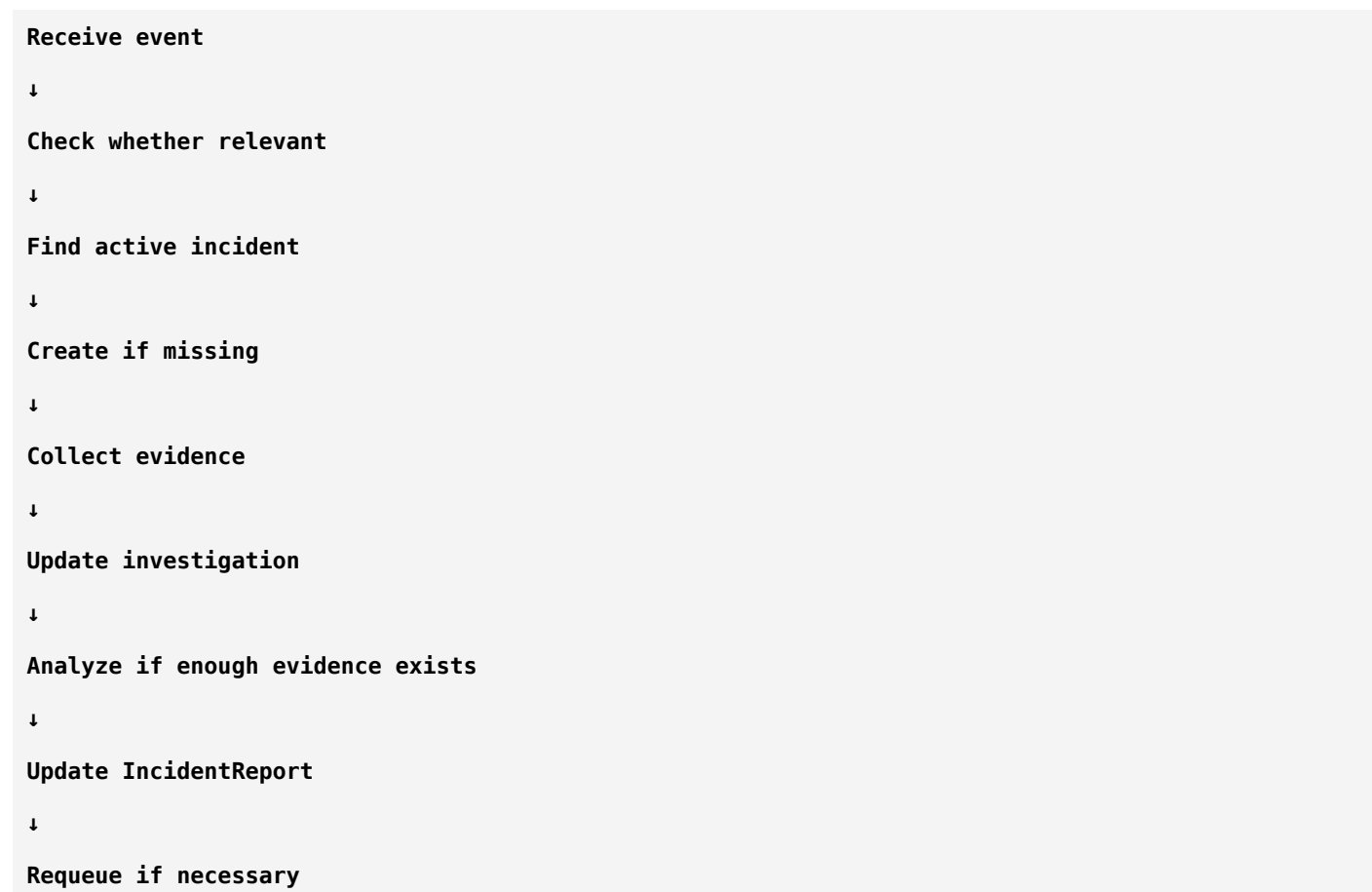
```

rather than simply Pod name.

Area 13 — Controller Behavior / Reconciliation

We need to design exactly what `Reconcile()` does.

For example:



But we need to determine:

When does it queue?

How long?

What happens if evidence isn't available yet?

What happens if API calls fail?

What happens if the operator crashes?

What happens if two reconcile calls happen simultaneously?

These are controller-runtime concerns.

Area 14 — Reliability and Failure Handling

This is where we move from "demo" toward production quality.

What happens if:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

API Server unavailable
 Prometheus unavailable
 Pod deleted
 Node deleted
 Logs unavailable
 CRD update fails
 RBAC denies access
 Network fails
 Operator restarts

Our operator itself will fail sometimes.

We need to design it so:

The investigator failing does not affect the applications it is investigating.

This is critical.

The investigator should be **read-mostly** and non-invasive.

Area 15 — Security / RBAC / Performance

Three things belong here.

RBAC

What can our operator read?

Potentially:

pods
 events
 nodes
 deployments
 replicaset
 pvcs
 configmaps
 secrets

We should request the **minimum permissions necessary**.

Secrets are particularly sensitive.

Do we actually need to read Secret contents?

Probably not.

We only need to know:

Secret exists?

That distinction matters.

Performance

Imagine a cluster with:

```
10,000 Pods
```

and thousands of events per minute.

Our operator cannot:

```
fetch everything
for every event
```

We need caching, filtering, rate limiting, and bounded investigation.

Multi-tenancy

Do we investigate:

```
all namespaces?
```

or

```
selected namespaces?
```

Can one team see another team's incident information?

This becomes important in real clusters.

16. Reporting / User Experience

We haven't fully designed how users consume the result.

Potential interfaces:

```
kubectl get incidentreports
kubectl describe incidentreport
```

Later:

```
Slack
Prometheus metrics
Grafana
```

Eventually maybe:

```
Web UI
```

But v1 should probably remain Kubernetes-native.

17. Alerting

Do we send alerts?

If yes:

Slack?
PagerDuty?
Kubernetes Event?

But there's a danger.

If we send a Slack message for every restart:

☀ Pod restarted
☀ Pod restarted
☀ Pod restarted
☀ Pod restarted

we've created another operational problem.

Alerting needs deduplication and severity rules.

I'd keep external alerting out of the first MVP.

18. Testing Strategy

We haven't designed this yet.

We need:

Unit tests

Test individual rules:

OOM evidence → OOM diagnosis
Image evidence → Image diagnosis

Controller tests

Test:

Pod failure
↓
IncidentReport created

Integration tests

Using:

envtest

Real Kubernetes tests

Using:

Kind

Failure simulation

We should intentionally create:

```
OOMKilled
ImagePullBackOff
CrashLoopBackOff
Probe failure
PVC failure
```

and verify the report.

19. Deployment / Packaging

We need to decide how users install it.

Potentially:

```
Helm
```

or:

```
kubectl apply
```

Eventually:

```
OCI image
GHCR
Helm chart
```

But this is downstream of architecture.

20. Observability of the Investigator itself

This is a gap I don't want us to forget.

We're building an observability tool.

The tool itself must be observable.

We should expose metrics like:

```
incidents_detected_total
incidents_completed_total
incidents_unknown_total
investigation_duration_seconds
evidence_collection_errors_total
rule_matches_total
reconcile_errors_total
```

Otherwise we won't know whether our investigator is actually working.

The architecture gaps I see right now

These are the things I would explicitly mark as **UNRESOLVED** today.

● Critical

1. **What exactly constitutes an incident?**
2. **How do we distinguish one incident from another?**
3. **When does an incident start?**
4. **When does it end?**
5. **How do Pod replacements affect incidents?**
6. **Exactly which resources trigger investigation?**
7. **Exactly which resources are evidence sources?**
8. **How do we correlate evidence?**
9. **How does the diagnosis engine work?**
10. **How do we handle multiple possible causes?**
11. **How do we handle unknown failures?**
12. **What exactly goes inside IncidentReport?**
13. **Where do large logs/evidence go?**

● Important

14. How long should evidence collection run?
15. When does the controller requeue?
16. What happens when evidence collection fails?
17. How do we handle operator restarts?
18. How do we prevent duplicate investigations?
19. How do we handle concurrent reconciliations?
20. How do we limit API usage?
21. What RBAC permissions are required?
22. Do we support all namespaces or selected namespaces?
23. Do we require Prometheus?
24. How do we expose the results?
25. What metrics should the investigator expose?

● Later

26. Slack/PagerDuty integration
 27. Grafana dashboard
 28. UI
 29. LLM-assisted analysis
 30. Historical pattern detection
 31. Cross-incident correlation
 32. Automatic remediation
-

The actual development plan I recommend

Don't tackle those 32 questions randomly.

We'll resolve them in this order:

Phase A — Define the problem

1. Exact problem
2. User
3. Value
4. Scope

Phase B — Define incident semantics

5. What is an incident?
6. Start
7. End
8. Correlation
9. Pod replacement

Phase C — Design evidence

10. What triggers?
11. What evidence?
12. Evidence relationships
13. Logs
14. Metrics

Phase D — Design intelligence

15. Rule model
16. Multiple causes
17. Confidence
18. Unknown diagnosis
19. Recommendations

Phase E — Design Kubernetes API

20. CRD
21. Status
22. Incident lifecycle
23. Persistence
24. Large evidence

Phase F — Design controller

- 25. Watch
- 26. Queue
- 27. Reconcile
- 28. Requeue
- 29. Concurrency
- 30. Failure recovery

Phase G — Production concerns

- 31. RBAC
- 32. Performance
- 33. Multi-tenancy
- 34. Observability
- 35. Security

Phase H — Implementation

Only **after all of that**:

```

Kubebuilder
  ↓
CRD
  ↓
Controller
  ↓
Evidence collectors
  ↓
Correlation
  ↓
Rules
  ↓
IncidentReport
  ↓
Tests
  ↓
Kind

```

One thing I strongly recommend

Don't try to answer all 32 questions in one sitting.

We'll take them **one architectural decision at a time**, and for each one I'll explain:

1. The problem.
2. What happens in a real Kubernetes cluster.
3. Possible designs.
4. What existing production tools do.
5. Tradeoffs.
6. What I recommend for this project.
7. Exactly how that decision affects the next components.

Then we'll lock the decision and move to the next one.

By the time we finish, we should be able to draw the complete architecture **without writing any code**. Only then should we touch Kubebuilder. That's the right order if the goal is to build something you can defend in a serious Platform/SRE interview rather than merely get running on Kind.

I have read all 20 areas and these needs to be cleared first, So take 5 areas each time and come up with your answers in details and tell what should be done for each specific area, what would be the best design possible...

Real talk:

Yes. We'll lock the architecture **five areas at a time**. I won't jump to code or Kubebuilder until these decisions are settled.

For this first batch, the correct order is:

1. Problem definition
2. Incident definition
3. Resources we watch
4. Evidence we collect
5. Logs

These five establish **what the system is trying to investigate, when an investigation starts, where it gets information, and how logs fit into the evidence**.

1. Problem Definition

What are we actually building?

Our original idea was:

"An operator that tells engineers why Pods are restarting."

That's too narrow.

We've now discovered that a restart is only one symptom. A workload can also be:

- stuck Pending
- unable to pull an image
- unable to mount storage
- failing probes
- unable to start because of configuration
- affected by a broken node
- unable to communicate because of networking
- repeatedly crashing

And often the real problem isn't inside the Pod at all.

So I would define the problem as:

Kubernetes engineers currently have to manually collect information from multiple Kubernetes resources, logs, events, nodes, and observability systems to understand why a workload is failing. The investigator automatically collects and correlates that evidence and produces a structured, explainable incident report.

That's our actual problem.

What are we NOT trying to do?

This is extremely important.

The investigator should **not initially**:

- restart Pods
- modify Deployments
- increase resources
- modify Services
- fix NetworkPolicies
- recreate PVCs
- change configuration
- automatically remediate incidents

Why?

Because diagnosis and remediation are different problems.

If our tool starts modifying production resources, one incorrect diagnosis could make an outage worse.

Our first responsibility is:

```
Observe
↓
Investigate
↓
Explain
```

Not:

```
Observe
↓
Guess
↓
Change production
```

What is the actual value?

There are three levels.

Level 1 — Evidence aggregation

Instead of the engineer doing:

```
kubectl describe pod
kubectl logs
kubectl logs --previous
kubectl get events
kubectl describe node
kubectl describe pvc
...
```

the investigator presents relevant evidence together.

This alone has value.

Level 2 — Timeline reconstruction

Instead of showing unrelated pieces of information:

```
OOMKilled
Node MemoryPressure
Restart
CrashLoopBackOff
```

we show:

```
14:02:01 - Memory usage increased
14:02:08 - Container exceeded memory limit
14:02:08 - Container terminated: OOMKilled
14:02:09 - Kubelet restarted container
14:02:20 - Container terminated again
14:02:21 - Pod entered CrashLoopBackOff
```

Now the engineer understands the sequence.

Level 3 — Diagnosis

Where evidence is sufficient:

```
Likely cause:
Container exceeded memory limit.

Confidence:
99%

Evidence:
Exit code 137
OOMKilled
Memory limit 512Mi
Node healthy

Recommendation:
Investigate application memory consumption.
```

This is the highest-value part.

What happens when we cannot diagnose?

We **still provide value**.

For example:

```
Diagnosis:
Unknown application failure

Evidence collected:
✓ Pod status
✓ Container state
✓ Exit code
✓ Previous logs
✓ Kubernetes events
✓ Node status

Evidence:
Exit code 42
Application exited unexpectedly.

Automatic root-cause identification:
Not available.

Recommended next step:
Investigate application-level logs and stack trace.
```

That's much better than pretending:

"Root cause: database issue."

when we don't know.

Decision #1

Our product is:

An evidence-first Kubernetes incident investigation system that automatically collects, correlates, and explains failure information without modifying the affected workload.

That's the foundation.

2. What exactly is an Incident?

This is one of our hardest design decisions.

We cannot say:

Every Pod update = incident.

That would be terrible.

Pods constantly change.

Example

Pod starts:

```
Pending
```

Then:

```
Running
```

That's normal.

We don't create an incident.

Pod gets restarted once:

```
RestartCount: 0 → 1
```

Is that an incident?

Not necessarily.

Applications can legitimately restart.

We need context.

So what should trigger an incident?

I recommend we distinguish between:

Normal lifecycle changes

Don't investigate.

Examples:

```
Pod created  
Pod scheduled  
Pod becomes Ready  
Normal rollout  
Normal termination
```

Suspicious signals

Potentially investigate.

Examples:

```
Restart count increases  
CrashLoopBackOff  
ImagePullBackOff  
FailedMount  
FailedScheduling  
Container terminated unexpectedly  
Repeated probe failures
```

Pod remains Pending abnormally long
Eviction
Node-related failures

Severe signals

Immediately investigate.

Examples:

OOMKilled
CrashLoopBackOff
NodeNotReady affecting workload
Repeated FailedMount
Repeated scheduling failures

We therefore need a trigger policy

Instead of saying:

"Every Pod failure is an incident."

we say:

An incident starts when one or more configured failure signals cross a meaningful threshold.

For MVP, we can hardcode sensible thresholds.

For example:

CrashLoopBackOff
→ immediate incident

OOMKilled
→ immediate incident

ImagePullBackOff
→ immediate incident

FailedMount repeated
→ incident

Readiness probe failure once
→ don't create incident

Readiness failures repeatedly
→ incident

This prevents noise.

When does an incident end?

This is another important decision.

We shouldn't mark:

OOMKilled
↓

```
container restarted
↓
Resolved
```

immediately.

The application might crash again 5 seconds later.

Instead:

```
Failure
↓
Investigating
↓
Evidence collected
↓
Application becomes healthy
↓
Healthy for stabilization period
↓
Resolved
```

For MVP, we could use something like a configurable **stability window**.

Example:

```
No relevant failure for 5 minutes
+
Pod Ready
+
Container running
```

→ incident resolved.

The exact default can be tuned later.

Same incident or new incident?

Suppose:

```
10:00 OOMKilled
10:01 OOMKilled
10:02 OOMKilled
10:03 CrashLoopBackOff
```

One incident.

But:

```
10:00 OOMKilled

Pod healthy for 2 hours

12:00 ImagePullBackOff
```

Two incidents.

So the incident needs:

```
Workload identity
+
```

```
failure relationship
+
time continuity
+
recovery state
```

What happens when a Pod is replaced?

Suppose:

```
Deployment/payment-api
```

has:

```
payment-api-abc
```

which fails.

Deployment creates:

```
payment-api-def
```

We shouldn't blindly say:

New Pod = new incident.

Because the workload might still be experiencing the same problem.

So the incident should primarily belong to the **workload**, while affected Pods are evidence.

Conceptually:

```
Incident #17

Workload:
Deployment/payment-api

Affected Pods:
- payment-api-abc
- payment-api-def
```

This is much stronger.

Decision #2

Our incident model is:

An incident represents a continuous period of abnormal behavior affecting a workload. Multiple related Pod failures during that period belong to the same incident. Once the workload has recovered and remained stable, a later failure becomes a new incident.

This decision will later drive our correlation algorithm.

3. What resources should we watch?

This needs careful distinction.

There are two concepts:

Trigger sources

Resources whose changes can tell us:

"Something might be wrong."

Evidence sources

Resources we query to understand:

"Why is it wrong?"

They don't have to be the same.

Primary trigger: Pods

I still recommend **Pods as the primary trigger**.

Why?

Because most workload failures eventually manifest at the Pod level.

For example:

```
OOMKilled
CrashLoopBackOff
ImagePullBackOff
Probe failure
FailedMount
Container failure
Eviction
```

All become visible through Pods.

But should we ONLY watch Pods?

No.

That would be a serious architectural mistake.

Consider:

```
Node becomes NotReady.
```

A Pod may not immediately change in a way that gives us the complete story.

Or:

PVC provisioning fails.

The PVC itself knows the problem.

Or:

CoreDNS fails.

Many Pods are victims.

Therefore:

Pod is the primary workload trigger, but other Kubernetes resources can generate supporting signals and, in some cases, independent incidents.

What should we watch?

For MVP, I'd use:

Primary

Pods

Secondary triggers

Nodes
PVCs
Events

But there's an important nuance.

Events shouldn't be treated like normal stateful resources.

Events are transient observations.

We should consume them as signals, not build our whole controller around them.

What should we query as evidence?

Potentially:

Pod
Events
Node
Deployment
ReplicaSet
PVC
ConfigMap
Secret
Service
EndpointSlice

And later:

Prometheus

Why Deployment/ReplicaSet?

Because the Pod alone doesn't tell us enough about ownership.

We want:

```
Pod
↓
ReplicaSet
↓
Deployment
```

Then our report can say:

```
Workload:
Deployment/payment-api

Affected Pod:
payment-api-abc123
```

rather than making the Pod the entire identity.

Why ConfigMap and Secret?

Suppose:

```
CreateContainerConfigError
```

Pod says:

```
Secret payment-secret not found
```

We need to verify:

```
Does payment-secret actually exist?
```

We don't need Secret contents.

This is important for security.

Why Service / EndpointSlice?

Suppose:

```
Readiness probe succeeds.
Application is running.
But traffic isn't reaching it.
```

We may need to investigate:

Service
EndpointSlice
Pod labels

This becomes useful for networking-related incidents.

Decision #3

For v1:

TRIGGERS

Pods
Nodes
PVCs
Relevant Events

EVIDENCE

Pod
Events
Owner workload
Node
PVC
ConfigMap existence
Secret existence
Service
EndpointSlice

Prometheus is optional and comes later.

This keeps the first version Kubernetes-native and avoids making Prometheus a mandatory dependency.

4. What exactly should we collect as Evidence?

Now we're getting into the actual investigation engine.

The biggest mistake would be:

"Whenever something happens, collect everything."

No.

That is expensive and noisy.

Instead, we should collect **layered evidence**.

Layer 1 — Primary object

If Pod triggered the incident:

Pod

Collect:

```
Phase
Conditions
Container states
Previous container states
Restart count
Exit code
Termination reason
Image
Command
Arguments
Resources
Probes
Volumes
Environment references
Node
OwnerReferences
```

This is our base evidence.

Layer 2 — Events

Collect relevant events associated with:

```
Pod
Node
PVC
```

Events can tell us:

```
FailedScheduling
FailedMount
FailedPull
Probe failed
OOMKilled
Evicted
```

Events are extremely useful because they often contain the **human-readable explanation** Kubernetes generated.

Layer 3 — Owner/workload

Walk the ownership chain:

```
Pod
↓
ReplicaSet
↓
Deployment
```

or:

```
Pod
↓
StatefulSet
```

or:

```
Pod
↓
DaemonSet
```

This tells us what workload is actually affected.

Layer 4 — Node

If Pod is running on:

```
worker-2
```

we check:

```
Ready
MemoryPressure
DiskPressure
PIDPressure
Conditions
```

This lets us distinguish:

```
Application-specific problem
```

from:

```
Node-wide problem
```

Layer 5 — Storage

Only if relevant.

For example:

```
FailedMount
PVC Pending
```

Then investigate:

```
PVC
PV
StorageClass
```

We shouldn't query storage for every CrashLoopBackOff.

Layer 6 — Configuration

If evidence indicates configuration issues:

```
Secret
ConfigMap
```

We check existence and metadata.

Again:

never dump Secret values into IncidentReport.

Layer 7 — Networking

Only when relevant:

```
Service
EndpointSlice
NetworkPolicy
```

Potentially later.

Layer 8 — Metrics

Later:

```
Prometheus
```

This can answer things Kubernetes objects cannot.

For example:

```
Memory increased from 200Mi
→
350Mi
→
500Mi
→
512Mi
→
OOMKilled
```

That's extremely strong evidence.

The key design principle

Evidence collection should be **adaptive**.

Example:

OOMKilled

Collect:

```
Pod
Events
Node
Resources
Metrics
Logs
```

ImagePullBackOff

Collect:

```
Pod
Events
Image configuration
ImagePullSecrets existence
Node/network information
```

Don't waste time querying:

```
PVC
Service
EndpointSlice
```

unless evidence indicates they're relevant.

Decision #4

Our evidence system will be:

```
Trigger
↓
Basic evidence
↓
Determine failure category
↓
Collect relevant additional evidence
↓
Build evidence package
```

This is better than a giant collector that blindly fetches everything.

5. How should Logs work?

Logs deserve special treatment.

Why?

Because logs can be **huge**.

Imagine a container produces:

```
10 GB/day
```

We cannot put that into a Kubernetes CRD.

And we shouldn't even try.

What do we actually need?

Usually:

Current logs

Useful when container is still running.

Previous logs

Extremely important for crashes/restarts.

For example:

```
Bash
kubectl logs --previous
```

This is often the most valuable piece of information after a restart.

How much should we collect?

For MVP I'd impose hard limits.

Something like:

```
Last 200 lines
```

or:

```
Maximum 64 KB
```

We can make these configurable later.

The important thing is:

Never allow one broken application to cause our Investigator to download gigabytes of logs.

What about multiple containers?

Pods can contain:

```
app
sidecar
init-container
```

We need to distinguish them.

For example:

```
Pod
├─ envoy
└─ payment-api
```

If **payment-api** crashes, we shouldn't automatically treat Envoy logs as application logs.

The report should identify:

```
Container:
payment-api
```

and separately collect relevant sidecar information when useful.

What if logs don't exist?

That's normal.

For:

```
ImagePullBackOff
CreateContainerConfigError
```

the application never started.

There may be no application logs.

The report should explicitly say:

```
Application logs unavailable because container did not start.
```

That's useful information.

What if the Pod disappears?

This is important.

Suppose:

```
Pod crashes
↓
Deployment deletes it
↓
New Pod created
```

If we haven't collected logs yet, they're gone.

Therefore, evidence collection must happen **quickly after the trigger**.

We should prioritize:

```
Pod state
Events
Previous logs
```

before doing slower correlation.

Should logs go into IncidentReport?

No, not unrestricted raw logs.

The CRD should contain:

Small relevant excerpts

For example:

YAML

```
evidence:
  logs:
    previous:
      container: payment-api
      excerpt: |
        Processing transaction batch...
        Killed
```

But we should enforce strict size limits.

For larger evidence, a future version could store it externally and put only a reference in the IncidentReport.

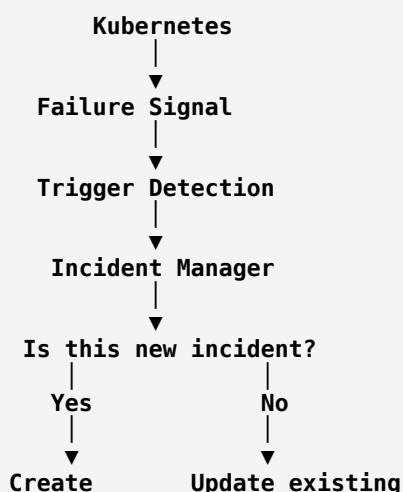
Decision #5

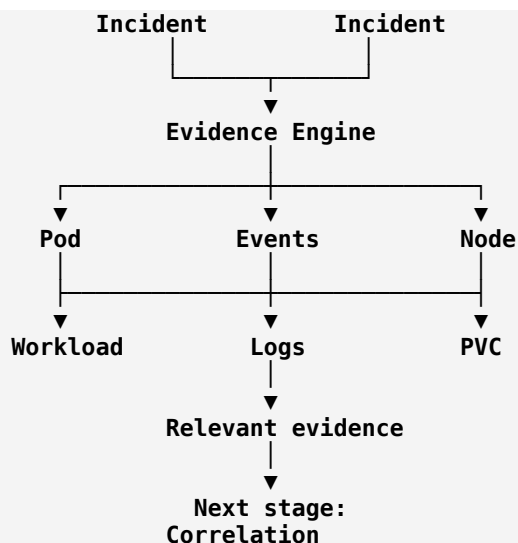
For v1:

- Collect current logs when useful.
- Collect previous logs for restart/crash investigations.
- Limit log size.
- Identify container explicitly.
- Never store unlimited logs in etcd.
- Never store Secret contents.
- If logs aren't available, report why.
- Collect logs early because Pods can disappear.

Where we stand after these five decisions

We now have a much clearer system:





And we have locked five important decisions:

Area	Decision
Problem	Evidence-first incident investigation, not remediation
Incident	Continuous abnormal behavior affecting a workload
Triggers	Pods primarily; Nodes/PVCs/Events as supporting triggers
Evidence	Layered and adaptive, not "collect everything"
Logs	Bounded excerpts, especially previous logs; never unlimited CRD storage

The most important architectural principle so far

We are not building an "error detector."

We're building an **incident investigation pipeline**:

```

Something suspicious happened
↓
Is it actually an incident?
↓
Which workload is affected?
↓
What evidence is relevant?
↓
What happened over time?
↓
What can we confidently conclude?
↓
What don't we know?
↓
Give the engineer a useful report
  
```

That's the system we're designing.

Next, the five areas I would tackle are **Evidence Correlation, Diagnosis Engine, Multiple Possible Causes, Unknown Failures, and Recommendations**. Those are the "intelligence" layer and, frankly, the technically hardest part of the project.

first answer these
CrashLoopBackOff

→ immediate incident

OOMKilled

→ immediate incident

ImagePullBackOff

→ immediate incident

FailedMount repeated

→ incident

Readiness probe failure once

→ don't create incident

Readiness failures repeatedly

→ incident

What does incident and immediate incident mean because investigation record is created when it is worthy threshold and would happen after it's record is created and data updated in that record as we get it. We can put all the threshold that we want to track here to track only them.

When we are not providing the logs into report then what is being done with logs.

then go with next 5 more

Show moreShow less

Real talk:

Your understanding is correct. I was using "**immediate incident**" too loosely.

There are two separate concepts we need to distinguish:

1. **Incident detection / creation**

2. **Investigation**

And yes, the clean design is:

Failure signal

↓

Does it cross our incident threshold?

↓

YES

↓

Create IncidentReport

↓

Start / continue investigation

↓

Keep updating the report as evidence arrives

There is no separate concept of "immediate investigation" that bypasses the report.

1. What does "immediate incident" actually mean?

It should mean:

The signal is strong enough that we don't wait for additional failures before creating an IncidentReport.

For example:

OOMKilled

You see:

```
Container terminated
Reason: OOMKilled
```

That's already strong evidence of a failure.

So:

```
OOMKilled
↓
Threshold crossed
↓
Create IncidentReport immediately
↓
Investigate
```

It does **not** mean:

```
OOMKilled
↓
Immediately know root cause
```

We don't.

We only know:

"Something significant happened; start an investigation."

Readiness probe failed once

You see:

```
Readiness probe failed
```

This is weak evidence.

Applications can temporarily fail a readiness check during startup or a short dependency issue.

So:

```
One readiness failure
↓
Below threshold
↓
No IncidentReport
```

But if we see:

```
failure
failure
failure
failure
failure
```

within our configured window:

```
5 failures within 2 minutes
```

then:

```
Threshold crossed
↓
Create IncidentReport
↓
Investigate
```

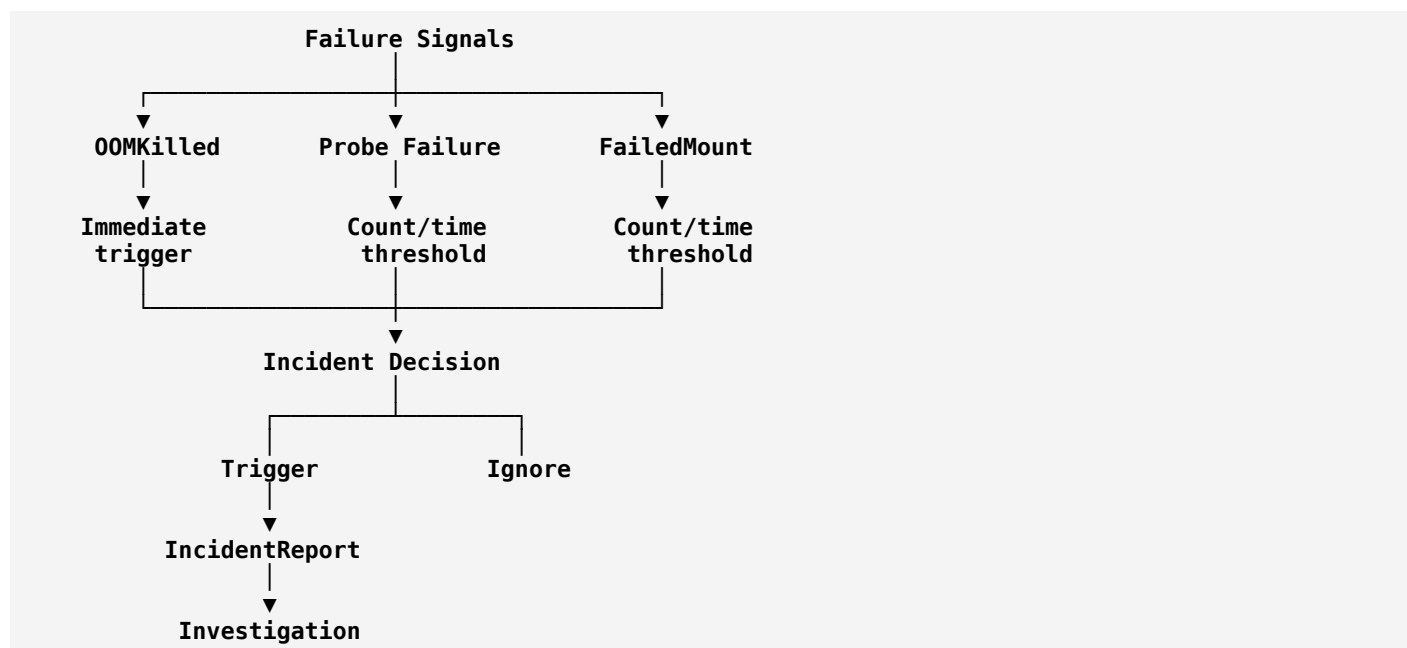
That's the distinction.

2. Yes, we should have a trigger/threshold policy

I agree with your idea.

We should have a **central incident trigger policy** rather than scattering arbitrary `if` conditions throughout the controller.

Conceptually:



This is much cleaner.

3. What thresholds should we initially track?

For the MVP, I'd define something like:

Signal	Initial policy	Why
OOMKilled	Immediate	Strong failure signal
CrashLoopBackOff	Immediate	Workload is repeatedly failing
ImagePullBackOff	Immediate	Container cannot start
CreateContainerConfigError	Immediate	Configuration prevents startup
FailedMount	Repeated	One transient mount failure may recover
FailedScheduling	Repeated/time-based	Scheduling can briefly fail during cluster changes
Readiness probe failure	Repeated	One failure is common
Liveness probe failure	Repeated	One failure can be transient
Evicted	Immediate	Pod was forcibly removed
Node NotReady	Immediate	Potential cluster/node incident

But these should eventually be **configurable**, rather than permanently hardcoded.

One important distinction: **we should not expose every threshold as a CRD configuration field in v1**. That would make the CRD unnecessarily complicated. Start with sensible defaults; later allow configuration.

4. What happens after the threshold is crossed?

This is the important part.

Suppose:

```
14:00:00
Readiness failure #1

14:00:10
Readiness failure #2

14:00:20
Readiness failure #3

14:00:30
Readiness failure #4

14:00:40
Readiness failure #5
```

Suppose our policy says:

```
5 failures within 2 minutes
```

At 14:00:40:

```
Threshold crossed
↓
Create IncidentReport
↓
status.phase = Investigating
```

The report now becomes the **persistent case file**.

Then we collect evidence.

5. What if more failures happen after that?

Suppose:

```
14:00:50 readiness failure #6
14:01:00 readiness failure #7
14:01:10 readiness failure #8
```

We **do not create three more IncidentReports**.

We determine:

"These are related to the currently active incident."

Then:

```
IncidentReport #17
events:
  failure #1
  failure #2
  failure #3
  ...
  failure #8
```

The same report is updated.

So the report is a **living case file while the incident is active**.

6. What if the same Pod later has a completely different failure?

Suppose the readiness problem is resolved.

Then two hours later:

```
OOMKilled
```

That is probably a **new incident**.

So:

```
IncidentReport #17
  Readiness failure
  Resolved
```

and:

```
IncidentReport #18
OOMKilled
Investigating
```

We don't overwrite #17.

We don't append OOMKilled to #17.

We create a new incident because the previous incident ended.

This is why **incident lifecycle and correlation** are so important.

7. What if the same failure continues for hours?

Then the same IncidentReport remains active.

For example:

```
Incident #17

Started:
10:00

Status:
Investigating

10:00 OOMKilled
10:01 OOMKilled
10:03 OOMKilled
10:05 CrashLoopBackOff
10:10 OOMKilled
10:20 OOMKilled
```

The report keeps getting updated.

Eventually:

```
10:30 Pod becomes healthy
10:31 healthy
10:32 healthy
...
stability window satisfied
```

Then:

```
status.phase = Resolved
```

8. So the report is not the final report only

This is another important correction.

I previously sometimes called it an "IncidentReport" as though it only existed after analysis.

Instead, think:

IncidentReport = persistent record of an incident and its investigation.

It starts incomplete.

At creation

```
status:
  phase: Investigating

  startedAt: 10:00

diagnosis:
  state: Unknown
```

During investigation

```
status:
  phase: Investigating

evidence:
  restartCount: 5
  reason: OOMKilled
  events: [...]
```

After diagnosis

```
status:
  phase: Diagnosed

diagnosis:
  cause: OOMKilled
  confidence: 0.99
```

After recovery

```
status:
  phase: Resolved
```

That's the model I'd lock in.

9. What happens to logs if they're not fully stored in the report?

Good question.

There are actually **three different things** we can do with logs.

Option 1 — Use logs only during investigation

The investigator retrieves:

```
kubectl logs --previous
```

internally.

Analyzer reads them.

It extracts useful information:

```
panic
database timeout
connection refused
```

Then we store only the relevant conclusion/evidence in the report.

The raw logs are **not permanently stored by our operator**.

This is the simplest MVP.

Option 2 — Store a small excerpt

For example:

```
YAML
evidence:
  logs:
    container: payment-api
    source: previous
    excerpt: |
      Processing transaction batch...
      database connection timeout
      process terminated
```

Strict size limit.

This gives the engineer some context without putting megabytes into etcd.

I prefer this for v1.

Option 3 — External evidence storage

Later, we could store larger evidence externally:

```
S3
Object Storage
Loki
Elasticsearch
```

and put only a reference in the IncidentReport:

```
YAML
evidence:
  logs:
    externalReference: ...
```

But **don't build this for v1**.

It introduces another dependency and turns a Kubernetes controller project into a storage-system project.

10. What happens if the logs disappear?

This is why evidence collection needs to happen quickly.

Imagine:

```
Pod A
↓
crashes
↓
previous logs available
↓
Investigator collects them
↓
Pod A deleted
↓
Pod B created
```

Our report still contains the relevant excerpt we captured.

That's another reason for storing a **small evidence snapshot**, rather than depending on being able to retrieve the old Pod forever.

Now the next five areas

We've already covered the first five. Let's move to:

6. Evidence Correlation
7. Diagnosis Engine
8. Multiple Possible Causes
9. Unknown Failures
10. Recommendations

These five are the **brain of the system**.

6. Evidence Correlation

This is probably the hardest part of the entire project.

Collecting information is relatively easy.

Understanding whether the information is related is difficult.

Imagine we get:

```
Pod payment-api-abc
OOMKilled

Node worker-2
MemoryPressure=True

Pod another-api-xyz
Evicted

Event:
Node has memory pressure
```

We have multiple pieces of information.

The question is:

Are these all part of the same incident?

Probably yes.

Correlation should use three dimensions

1. Time

Events close together in time are more likely to be related.

For example:

```
14:01:01 OOMKilled
14:01:02 Node MemoryPressure
14:01:03 Pod Evicted
```

Strong relationship.

But:

```
14:01 OOMKilled
18:00 Node MemoryPressure
```

Probably unrelated.

2. Kubernetes relationships

We know:

```
Pod
↓
Node
```

and:

```
Pod
↓
ReplicaSet
↓
Deployment
```

and:

```
Pod
↓
PVC
```

These relationships give us context.

3. Failure semantics

Some failures naturally relate.

For example:

```
PVC Pending
↓
FailedMount
↓
Container never starts
```

That's a logical chain.

Whereas:

```
PVC Pending
+
unrelated ImagePullBackOff
```

might be two independent problems.

So I would NOT build a complicated AI correlation engine initially

That's overengineering.

Use deterministic correlation:

```
time window
+
resource relationship
+
failure type
+
causal relationship
```

This is explainable.

And explainability matters because we're giving engineers operational recommendations.

Example

Suppose:

```
14:00:01
Pod FailedMount

14:00:02
PVC Pending

14:00:03
StorageClass provisioning failed
```

Correlation engine says:

```
These events are related because:

1. Same workload
2. Same storage dependency
3. Occurred within 2 seconds
4. Causal chain exists
```

Now the analyzer has a coherent evidence set.

Decision #6

Use a **deterministic correlation engine**, not AI.

Correlation is based on:

```
time
+
resource relationship
+
failure semantics
```

AI/LLM can be a future enhancement, but it should not be the foundation of the diagnosis system.

7. Diagnosis Engine

Now we have:

Correlated Evidence

We need to answer:

What does this evidence mean?

I'd use a **rule-based diagnosis engine** initially.

But not a giant:

```
Go
if x && y && z {
    ...
}
```

Instead, each diagnosis is a separate rule.

Conceptually:

```
Rule
├── Name
├── Match()
└── Diagnose()
```

Example:

```
OOMRule

Match:
    container terminated
    AND reason == OOMKilled

Diagnosis:
    Container exceeded memory limit
```

Why rules?

Because operational diagnosis needs to be:

- deterministic
- testable
- explainable
- predictable

If the same evidence comes in twice, we want the same diagnosis.

Diagnosis should produce more than a cause

A rule should produce something like:

```

Diagnosis:

Cause:
Container exceeded memory limit

Confidence:
Very High

Evidence:
- OOMKilled
- Exit code 137
- Memory limit 512Mi

Explanation:
The container exceeded its configured memory limit
and was terminated by the kernel.

Recommendation:
Investigate memory consumption and memory limit.
```

This is much better than:

```
OOMKilled
```

Decision #7

Use a **modular deterministic rule engine**.

Each failure diagnosis is independently implemented and tested.

No giant conditional block.

No LLM dependency.

8. What if multiple causes are possible?

This absolutely will happen.

Suppose:

```
OOMKilled
```

and:

```
Node MemoryPressure=True
```

Possible explanations:

Cause A

Container itself exceeded its memory limit.

Cause B

Node is under severe memory pressure.

Both may be true.

So we should not force the engine to choose only one.

Instead, the diagnosis result should support:

```
Primary finding
+
Contributing factors
```

For example:

```
Primary cause:
Container exceeded memory limit.

Confidence:
95%

Contributing factor:
Node was also experiencing memory pressure.

Confidence:
80%
```

Another example

Suppose:

```
ImagePullBackOff
```

Events:

```
Failed to pull image
unauthorized
```

and:

```
imagePullSecret missing
```

We might say:

Primary cause:
Registry authentication failure.

Supporting evidence:
Referenced imagePullSecret does not exist.

What if two completely different causes are possible?

Then we should represent both.

For example:

Possible causes:

1. Registry authentication failure – 70%
2. Incorrect image reference – 30%

But be careful:

Those percentages should not be fake mathematical probabilities.

If we're not running a statistically trained model, calling 70% a probability is misleading.

Better:

Confidence:
High

or:

Confidence:
Medium

For MVP I'd actually use:

High
Medium
Low

or a bounded confidence score with a clearly defined meaning.

Decision #8

Diagnosis supports:

Primary cause
+
Contributing factors
+
Alternative hypotheses
+
Evidence
+
Confidence level

It does not force a single root cause when evidence doesn't justify one.

9. Unknown Failures

This is one of the most important features.

Suppose:

```
Exit code: 42
```

No known rule matches.

Bad system:

```
Root cause: Application bug
```

That's speculation.

Good system:

```
Diagnosis:
Unknown

Reason:
No known infrastructure failure pattern matched.

Evidence collected:
Pod state
Container state
Events
Previous logs
Node state

Relevant evidence:
Application exited with code 42.
No Kubernetes-level failure was identified.

Recommendation:
Investigate application logs and application-level stack traces.
```

That is honest and useful.

Unknown doesn't mean useless.

The report can still save the engineer time by presenting:

```
Timeline
+
Evidence
+
What was ruled out
+
What remains unknown
```

For example:

```
Node:
Healthy

PVC:
Healthy

Image:
```

Successfully pulled

Container:
Exited with code 42

Logs:
Application panic detected

Now the engineer immediately knows:

"This probably isn't a Kubernetes infrastructure issue. I should go into the application."

That's valuable.

Decision #9

Unknown diagnosis is a **first-class outcome**, not an error.

Our possible results are:

Diagnosed
Partially Diagnosed
Unknown

rather than pretending everything has a root cause.

10. Recommendations

This is another place where we need to be careful.

The recommendation should come from the evidence.

Not generic garbage like:

Check your Kubernetes configuration.

That's useless.

Good recommendation

Evidence:

OOMKilled
Memory limit = 512Mi
Memory usage reached limit

Recommendation:

Inspect application memory growth.

Consider increasing the container memory limit
if the workload legitimately requires more memory.

Another example

Evidence:

```
ImagePullBackOff
Event:
manifest unknown
Image:
payment-api:v42
```

Recommendation:

```
Verify that image tag payment-api:v42 exists
in the configured registry.
```

FailedMount

Evidence:

```
PVC:
Pending

StorageClass:
gp3

CSI:
provisioning failed
```

Recommendation:

```
Investigate CSI provisioning and StorageClass configuration.
```

Recommendations should have severity

For example:

```
Immediate:
Restore missing image.

Recommended:
Investigate memory growth.

Informational:
Consider reviewing resource limits.
```

Most important rule

Our investigator must **not automatically perform the recommendation**.

Report:

```
Recommended action:
Increase memory limit.
```

Not:

I increased memory limit.

The latter becomes remediation.

That's outside our initial scope.

Decision #10

Recommendations are:

Evidence-backed
Specific
Explainable
Non-destructive

They are suggestions for the engineer, not automatic actions.

So now our architecture has two clear halves

Investigation side

```

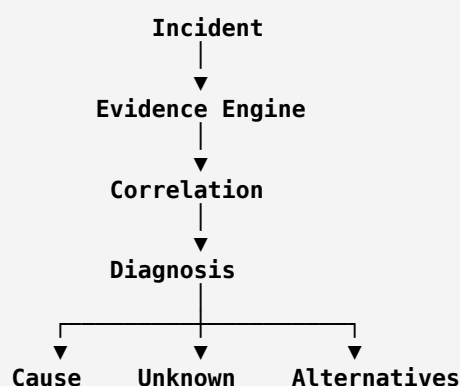
Trigger
↓
Incident
↓
Evidence Collection
↓
Correlation
  
```

Intelligence side

```

Correlated Evidence
↓
Diagnosis Rules
↓
Primary Cause
↓
Contributing Factors
↓
Confidence
↓
Recommendations
  
```

Then:





The key architectural decision from these five:

Our system should never be designed around "recognizing 15 errors."

Instead:

The system collects evidence first. Rules interpret evidence second.

That means when production gives us a failure we've never seen before, we don't completely break. We can still produce a useful evidence report, even though our diagnosis engine doesn't yet have a rule for that failure.

And that's exactly the property you were concerned about earlier: **we don't need to know every possible Kubernetes failure in advance to make the tool useful.**

Today 8:26 AM

i am getting bored you are giving me a lot of this things to read, it is also taking huge amount of time to read all these and understand, take care of this as well, now next

Real talk:

Got it. **From now on, I'll keep these architecture discussions much shorter.** We need enough detail to make a decision, not a textbook.

We finished Areas 1–10. Let's do the **next 5: 11–15.**

11. IncidentReport CRD

The CRD is basically the **persistent case file** for an incident.

It should contain enough information for an engineer to understand what happened, but **not dump everything into etcd.**

A simplified structure:

```

YAML
kind: IncidentReport

spec:
  workload:
    kind: Deployment
    name: payment-api

status:
  phase: Investigating

  startedAt: ...

  affectedPods:
    - payment-api-abc

  timeline:
  
```

```
- ...

diagnosis:
  cause: OOMKilled
  confidence: High

evidence:
  summary: ...

recommendations:
  - ...
```

Important decision

Don't put huge raw logs or unlimited events inside the CRD.

The CRD should contain:

- summary
- important evidence
- timeline
- diagnosis
- recommendations
- small log excerpts

The raw/large evidence can remain outside the CRD later if needed.

Decision: CRD = structured investigation summary, not a data dump.

12. Incident Lifecycle

We need a simple lifecycle.

I'd use:

```
Investigating
↓
Diagnosed
↓
Resolved
```

And if we can't determine the cause:

```
Investigating
↓
Unknown
↓
Resolved
```

Example

```
OOMKilled detected
↓
Investigating
↓
Evidence collected
↓
```

```

Diagnosis = OOMKilled
  ↓
Diagnosed
  ↓
Pod healthy for stability period
  ↓
Resolved

```

The important thing:

Diagnosed does not mean the problem is fixed.

It only means:

"We have enough evidence to explain what happened."

The workload may still be broken.

13. Pod Replacement / Workload Identity

This is important because Pods are temporary.

Suppose:

```

Deployment/payment-api

Pod A → crashes
Pod A → deleted

Pod B → created
Pod B → also crashes

```

We don't want:

```

Incident A
Incident B

```

automatically.

They may be the **same workload incident**.

So the IncidentReport should primarily identify:

```

Deployment/payment-api

```

and maintain affected Pods:

```

Pods:
  payment-api-A
  payment-api-B

```

This lets us say:

"Deployment **payment-api** has been continuously failing across multiple replacement Pods."

That's much more useful than thinking only in terms of individual Pods.

Decision: Workload is the primary incident identity; Pods are evidence associated with it.

14. Controller Behavior / Reconciliation

Our `Reconcile()` should remain relatively simple.

Conceptually:

```

Event
↓
Reconcile()
↓
Is this relevant?
↓
Find/create active IncidentReport
↓
Collect relevant evidence
↓
Update report
↓
Can we diagnose?
↓
Update diagnosis
↓
Should we recheck later?

```

It doesn't have to perform the entire investigation in one call.

For example:

```

Reconcile #1
→ create report

Reconcile #2
→ collect more evidence

Reconcile #3
→ diagnose

Reconcile #4
→ detect recovery

```

This is normal controller behavior.

What causes another Reconcile?

Several things:

- watched Kubernetes resource changes
- our IncidentReport changes
- explicit requeue after some time

For example, if we need to check whether a Pod has recovered after 5 minutes, we can requeue.

Decision: Reconcile orchestrates the investigation; it isn't itself the entire investigation.

15. Reliability / Failure Handling

This is where we make the operator safe.

Our Investigator itself can fail.

For example:

```
API Server temporarily unavailable
Logs unavailable
Node disappears
Permission denied
Operator crashes
```

We should **not lose the incident**.

Because the IncidentReport already exists in Kubernetes:

```
IncidentReport
phase: Investigating
```

the restarted controller can find it and continue.

Example

```
10:00 incident detected
      ↓
IncidentReport created
      ↓
Evidence collection starts
      ↓
Investigator crashes 🌟
```

After restart:

```
Find IncidentReport
      ↓
phase = Investigating
      ↓
Continue investigation
```

That's the main reason we chose the "create case first" design.

What if evidence collection fails?

Don't mark the whole investigation as failed.

For example:

```
Pod information      ✓
Events              ✓
Node information     ✓
Logs                ✗
```

Report:

```
Evidence unavailable:
Container logs could not be retrieved.
```

Continue with everything else.

The investigator should be **best-effort**, not all-or-nothing.

The 5 decisions

Area	Decision
CRD	Persistent structured case file; no huge raw data
Lifecycle	Investigating → Diagnosed/Unknown → Resolved
Identity	Workload first, Pods attached as evidence
Reconcile	Orchestrates investigation across multiple calls
Reliability	Persistent reports + best-effort evidence collection

That's enough for this batch.

Next 5 should be 16–20: RBAC, performance, multi-tenancy, user experience/reporting, and alerting. Those are much more straightforward, so I'll keep that batch particularly concise.

next...

Real talk:

Next 5 are mostly **production-quality concerns**, so I'll keep them concise.

16. RBAC / Security

Our operator needs permission to **read** Kubernetes resources.

For example:

```
Pods
Events
Nodes
Deployments
ReplicaSets
PVCs
Services
EndpointSlices
```

But it should **not** automatically get permission to modify them.

Especially avoid:

```
delete pods
update deployments
read secret contents
```

unless there's a genuine requirement.

For Secrets, ideally we only need metadata/existence, not the actual secret value.

Decision

Read-only by default.

This is important because an investigation tool should not become a security risk or accidentally modify production.

17. Performance / Scalability

Imagine a cluster has:

10,000 Pods

We cannot investigate every Pod update deeply.

For example, this would be terrible:

```
Pod changes
↓
Fetch logs
↓
Fetch events
↓
Fetch node
↓
Fetch PVC
↓
Analyze
```

for every single Pod event.

Instead:

```
Normal event
↓
Ignore

Suspicious event
↓
Create/update incident
↓
Deep investigation
```

And during investigation:

- limit log size
- limit evidence collection frequency
- avoid duplicate API requests
- use controller-runtime's cache where appropriate
- avoid repeatedly investigating the same incident

Decision

Cheap filtering first, expensive investigation only after an incident is detected.

18. Multi-tenancy / Namespace Scope

Suppose a cluster has:

```
team-a
team-b
team-c
```

Should our operator investigate everything?

For the first version, I recommend:

Support cluster-wide installation, but allow namespace filtering.

Example:

```
--watch-namespaces=production,staging
```

Why?

Because a company may not want an internal tool investigating every namespace.

Also, teams may not want one team's incident data exposed to another team.

Decision

Start with namespace-scoped operation, with the architecture capable of cluster-wide operation later.

19. How does the user consume the result?

For v1, don't build a dashboard.

That would waste time.

Kubernetes already gives us a natural interface:

```
Bash
kubectl get incidentreports
```

Example:

NAME	WORKLOAD	STATUS	CAUSE
payment-api-001	payment-api	Resolved	OOMKilled
frontend-004	frontend	Investigating	ProbeFailure
orders-002	orders	Unknown	-

Then:

```
Bash
kubectl describe incidentreport payment-api-001
```

gives:

```
Timeline
Affected Pods
Evidence
Diagnosis
Confidence
Recommendations
```

This is actually a strong Kubernetes-native UX.

Later we can add:

Slack
Grafana
Web UI

but they're not needed for MVP.

Decision

`kubectl get/describe incidentreports` is the primary interface for v1.

20. Alerting

We should **not immediately build Slack/PagerDuty integration**.

Why?

Because our system itself needs to avoid alert noise first.

Imagine:

```
500 Pod restarts
  ↓
500 Slack messages
```

We've created another incident.

Instead, first make the IncidentReport reliable.

Later:

```
Incident created
  ↓
Severity evaluated
  ↓
Deduplication
  ↓
Alert
```

And alert only meaningful incidents.

For example:

```
OOMKilled
→ alert

Single readiness failure
→ no alert
```

Decision

No external alerting in MVP.

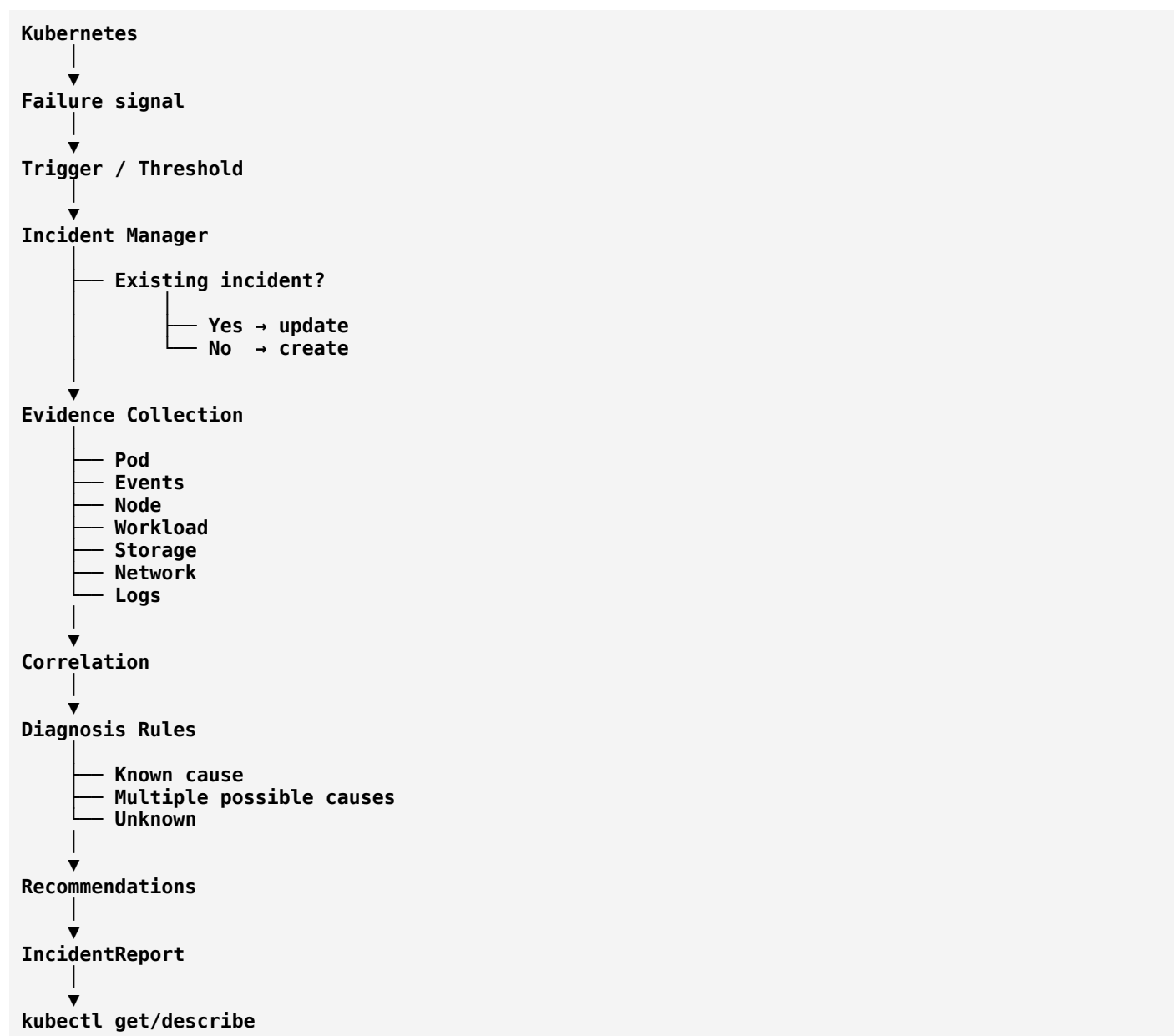
Build the investigation/reporting system first; add Slack/PagerDuty later.

Where we are now

We've now covered all **20 major architecture areas**.

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

The high-level system is:



And importantly, we've deliberately not built:

- automatic remediation
- LLM-based diagnosis
- dashboard
- Slack integration
- external evidence storage
- Prometheus dependency

Those are distractions until the core investigator works.

At this point, the next step shouldn't be more theory. We should take these decisions and turn them into the **actual architecture: components, their responsibilities, data flow, CRD structure, and exact lifecycle of one incident from detection → resolution.** That will be much easier to digest than continuing to add isolated concepts.

ChatGPT can make mistakes. Check important info.